



HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÀI GIẢNG THUẬT TOÁN VÀ ỨNG DỤNG

Biên soạn

**:TS. ĐỖ THỊ BÍCH NGỌC
TS. TRẦN QUÍ NAM
TS. NGUYỄN DUY PHƯƠNG**

Hà Nội, tháng 12/2019

LỜI NÓI ĐẦU

Thuật toán là thủ tục hay trình tự thực hiện các công việc của máy tính được định nghĩa rõ ràng, trong đó nó nhận vào giá trị hoặc một tập giá trị (gọi là input) và sinh ra giá trị hoặc một tập giá trị (gọi là output). Có thể hiểu thuật toán như là tấm bản đồ, chỉ dẫn để hoàn thành công việc. Thuật toán quan trọng trong tất cả các lĩnh vực khác nhau của khoa học máy tính. Một máy tính có tốc độ cao hay tốc độ thấp, một hệ điều hành thực hiện ổn định hay không ổn định, một ứng dụng chạy nhanh hay chạy chậm tất cả đều phụ thuộc vào thuật toán cài đặt trên công nghệ, hệ điều hành và phần mềm đó. Càng nhiều vấn đề được giải quyết thì càng có nhiều thuật toán. Thông qua việc hiểu biết về thuật toán người học sẽ có khả năng lựa chọn được giải pháp hợp lý cho vấn đề cần giải quyết và áp dụng nó một cách chính xác.

Tài liệu giảng dạy thuật toán và ứng dụng được xây dựng dành cho sinh viên năm thứ ba ngành công nghệ thông tin được tổ chức thành 11 chương. Trong đó chương 1 trình bày một số vấn đề liên quan đến thuật toán, độ phức tạp thuật toán, độ phức tạp chương trình và độ phức tạp của các cấu trúc lệnh. Chương 2 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán bằng mô hình thuật toán sinh. Chương 3 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán bằng mô hình thuật toán đệ qui và quay lui. Chương 4 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán bằng mô hình thuật toán tham lam. Chương 5 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán bằng mô hình thuật toán chia và trị. Chương 6 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán bằng mô hình thuật toán qui hoạch động. Chương 7 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán bằng mô hình thuật toán sắp xếp và tìm kiếm. Chương 8 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán dựa vào ngăn xếp. Chương 9 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán dựa vào hàng đợi. Chương 10 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán dựa vào cây nhị phân. Chương 11 trình bày những kỹ năng cơ bản và nâng cao giải quyết các bài toán dựa vào đồ thị.

Trong mỗi phần của tài liệu, chúng tôi cố gắng trình bày ngắn gọn trực tiếp vào bản chất của mô hình thuật toán nhằm đạt được ba mục tiêu chính: hiểu được bản chất của mô hình thuật toán, giải quyết được lớp các bài toán dựa vào mô hình thuật toán, nâng cao kỹ năng giải quyết vấn đề bằng thuật toán. Mặc dù đã rất cẩn trọng trong quá trình biên soạn, tuy nhiên tài liệu không tránh khỏi những thiếu sót và hạn chế. Chúng tôi rất mong được sự góp ý quý báu của tất cả bạn đọc.

Hà Nội, tháng 12 năm 2019

MỤC LỤC

.....	1
LỜI NÓI ĐẦU	2
MỤC LỤC.....	i
CHƯƠNG 1. THUẬT TOÁN VÀ MỘT SỐ VẤN ĐỀ LIÊN QUAN	4
1.1. Thuật toán và một số vấn đề liên quan.....	4
1.2. Biểu diễn thuật toán.....	5
1.3. Độ phức tạp thuật toán	7
1.3.1. Khái niệm độ phức tạp thuật toán	7
1.3.2. Một số qui tắc xác định độ phức tạp thuật toán.....	8
1.3.3. Một số dạng hàm được dùng xác định độ phức tạp thuật toán.....	9
1.4. Độ phức tạp của các cấu trúc lệnh.....	10
1.5. Qui trình giải quyết bài toán trên máy tính	12
CHƯƠNG 2. MÔ HÌNH THUẬT TOÁN SINH	14
2.1. Giới thiệu thuật toán sinh	14
2.2. Một số bài toán xử lý bằng thuật toán sinh cơ bản.....	16
2.3. Một số bài toán xử lý bằng thuật toán sinh nâng cao	19
2.4. Một số bài toán xử lý bằng thuật toán sinh tiên tiến	22
CHƯƠNG 3. MÔ HÌNH THUẬT TOÁN ĐỆ QUI & QUAY LUI.....	24
3.1. Mô hình thuật toán đệ qui	24
3.2. Mô hình thuật toán quay lui (Back-track Algorithm)	25
3.3. Một số bài toán xử lý bằng thuật toán đệ qui & quay lui cơ bản	26
3.4. Một số bài toán xử lý bằng thuật toán đệ qui và quay lui nâng cao	31
3.5. Một số bài toán xử lý bằng thuật toán đệ qui và quay lui tiên tiến	34
CHƯƠNG 4. MÔ HÌNH THUẬT TOÁN THAM LAM.....	39
4.1. Giới thiệu mô hình thuật toán tham lam (Greedy Algorithm)	39
4.2. Một số bài toán xử lý bằng thuật toán tham lam cơ bản	43
4.3. Một số bài toán xử lý bằng thuật toán tham lam nâng cao.....	47
4.4. Một số bài toán xử lý bằng thuật toán tham lam tiên tiến.....	51
CHƯƠNG 5. MÔ HÌNH THUẬT TOÁN CHIA VÀ TRỊ	55
5.1. Giới thiệu mô hình thuật toán chia và trị (Devide and Conquer Algorithm)	55
5.2. Một số bài toán xử lý bằng thuật toán chia và trị cơ bản	56
5.3. Một số bài toán xử lý bằng thuật toán chia và trị nâng cao.....	60
5.4. Một số bài toán xử lý bằng thuật toán chia và trị tiên tiến.....	63
CHƯƠNG 6. MÔ HÌNH THUẬT TOÁN QUI HOẠCH ĐỘNG.....	68
6.1. Giới thiệu mô hình thuật toán qui hoạch động	68
6.2. Một số bài toán xử lý bằng thuật toán qui hoạch động cơ bản.....	70
6.3. Một số bài toán xử lý bằng thuật toán qui hoạch động nâng cao	77

6.4. Một số bài toán xử lý bằng thuật toán qui hoạch động tiên tiến	81
CHƯƠNG 7. MÔ HÌNH THUẬT TOÁN SẮP XẾP VÀ TÌM KIẾM	86
7.1. Giới thiệu vấn đề	86
7.2. Một số bài toán xử lý bằng thuật toán sắp xếp và tìm kiếm cơ bản	86
7.3. Một số bài toán xử lý bằng thuật toán sắp xếp và tìm kiếm nâng cao	93
7.4. Một số bài toán xử lý bằng thuật toán sắp xếp tiên tiến	98
CHƯƠNG 8. MÔ HÌNH THUẬT TOÁN DỰA VÀO NGĂN XẾP	102
8.1. Giới thiệu ngăn xếp (stack)	102
8.2. Một số bài toán cơ bản dựa vào ngăn xếp	104
8.3. Một số bài toán nâng cao dựa vào ngăn xếp	107
8.4. Một số bài toán tiên tiến dựa vào ngăn xếp	110
CHƯƠNG 9. MÔ HÌNH THUẬT TOÁN DỰA VÀO HÀNG ĐỢI	117
9.1. Giới thiệu hàng đợi (queue)	117
9.2. Một số bài toán cơ bản dựa vào hàng đợi	118
9.3. Một số bài toán nâng cao dựa vào hàng đợi	120
9.4. Một số bài toán tiên tiến dựa vào hàng đợi	123
CHƯƠNG 10. MÔ HÌNH THUẬT TOÁN DỰA VÀO CÂY NHỊ PHÂN	126
10.1. Giới thiệu cây nhị phân (binary tree)	126
10.2. Một số bài toán cơ bản dựa vào cây nhị phân	127
10.3. Một số bài toán nâng cao dựa vào vào cây nhị phân	134
10.4. Một số bài toán tiên tiến dựa vào vào cây nhị phân	140
CHƯƠNG 11. MÔ HÌNH THUẬT TOÁN DỰA VÀO ĐỒ THỊ	145
11.1. Giới thiệu về đồ thị	145
11.1.1. Một số thuật ngữ trên đồ thị vô hướng	146
11.1.2. Một số thuật ngữ trên đồ thị có hướng	146
11.1.3. Một số loại đồ thị đặc biệt	147
11.1.4. Biểu diễn đồ thị	148
11.2. Một số bài toán cơ bản dựa vào đồ thị	150
11.3. Một số bài toán nâng cao dựa vào đồ thị	155
11.4. Một số bài toán tiên tiến dựa vào đồ thị	159
TÀI LIỆU THAM KHẢO	170

PDF

CHƯƠNG 1. THUẬT TOÁN VÀ MỘT SỐ VẤN ĐỀ LIÊN QUAN

Nội dung chính của chương này là cung cấp cho người đọc những vấn đề cơ bản liên quan đến thuật toán. Đối với một thuật toán, người học cần quan tâm đến 4 vấn đề cơ bản: biểu diễn thuật toán, ước lượng độ phức tạp thuật toán, cài đặt thuật toán, thử nghiệm thuật toán. Tài liệu sẽ sử dụng ngôn ngữ hình thức để biểu diễn thuật toán, sử dụng độ phức tạp của các cấu trúc lệnh & cấu trúc dữ liệu để ước lượng độ phức tạp thuật toán, sử dụng ngữ lập trình C++ để cài đặt thuật toán, sử dụng phương pháp thử nghiệm dựa vào dữ liệu để kiểm tra tính đúng đắn của thuật toán.

1.1. Thuật toán và một số vấn đề liên quan

Thuật toán là thủ tục hay trình tự thực hiện các công việc của máy tính được định nghĩa rõ ràng, trong đó nó nhận vào giá trị hoặc một tập giá trị (gọi là input) và sinh ra giá trị hoặc một tập giá trị (gọi là output). Ta có thể tổng quát hóa khái niệm thuật toán như sau.

Định nghĩa thuật toán (Algorithm): Thuật toán F giải bài toán P là dãy các thao tác sơ cấp $F_1, F_2, \dots, F_N$ trên tập dữ kiện đầu vào (*Input*) để đưa ra được kết quả ra (*Output*).

$F_1 F_2 \dots F_N(\text{Input}) \rightarrow \text{Output}$.

- $F = F_1 F_2 \dots F_N$ được gọi là thuật toán giải bài toán P . Trong đó, mỗi F_i là các phép toán sơ cấp.
- *Input* được gọi là tập dữ kiện đầu vào hay tập thông tin đầu vào.
- *Output* là kết quả nhận được sau khi thực hiện thuật toán F trên tập *Input*.

Ví dụ. Thuật toán tìm USCLN(a, b).

```
int    USCLN ( int a, int b) { //đầu vào là số nguyên a, b
    while (b!=0) { //lặp trong khi b khác 0
        x = a % b; //lấy x là a mode b
        a = b; //đặt a bằng b
        b = x; //đặt b bằng x
    }
    return(a); //kết quả thực thi thuật toán
}
```

Những đặc trưng cơ bản của thuật toán:

- **Tính đơn định.** Ở mỗi bước của thuật toán, các thao tác sơ cấp phải hết sức rõ ràng, không tạo ra sự lộn xộn, nhập nhằng, đa nghĩa. Thực hiện đúng các bước của thuật toán trên tập dữ liệu đầu vào chỉ cho duy nhất một kết quả.

- **Tính dừng.** Thuật toán không được rơi vào quá trình vô hạn. Thuật toán phải dừng lại và cho kết quả sau một số hữu hạn các bước.
- **Tính đúng.** Sau khi thực hiện tất cả các bước của thuật toán theo đúng qui trình đã định, ta phải nhận được kết quả mong muốn với mọi bộ dữ liệu đầu vào. Kết quả đó được kiểm chứng bằng yêu cầu của bài toán.
- **Tính phổ dụng.** Thuật toán phải dễ sửa đổi để thích ứng được với bất kỳ bài toán nào trong lớp các bài toán cùng loại và có thể làm việc trên nhiều loại dữ liệu khác nhau.
- **Tính khả thi.** Thuật toán phải dễ hiểu, dễ cài đặt, thực hiện được trên máy tính với thời gian cho phép.

Đối với thuật toán ta cần quan tâm đến những vấn đề sau:

- **Biểu diễn thuật toán:** xác định ngôn ngữ để biểu diễn thuật toán.
- **Đánh giá độ phức tạp thuật toán:** ước lượng thời gian và không gian nhớ khi thực hiện thuật toán.
- **Kiểm nghiệm thuật toán:** kiểm nghiệm thuật toán với các bộ dữ liệu thực khác nhau.
- **Cài đặt thuật toán:** cài đặt thuật toán bằng ngôn ngữ lập trình cụ thể.

1.2. Biểu diễn thuật toán

Có ba ngôn ngữ chính để biểu diễn thuật toán: ngôn ngữ tự nhiên, ngôn ngữ máy tính và ngôn ngữ hình thức.

- **Ngôn ngữ tự nhiên** là phương tiện giao tiếp giữa con người với con người. Ta có thể sử dụng chính ngôn ngữ này vào việc biểu diễn thuật toán.
- **Ngôn ngữ máy tính** là phương tiện giao tiếp giữa máy tính và máy tính. Trong trường hợp này ta có thể sử dụng bất kỳ ngôn ngữ lập trình nào để biểu diễn thuật toán (C, Pascal, Java...).
- **Ngôn ngữ hình thức.** Ngôn ngữ hình thức là phương tiện giao tiếp trung gian giữa con người và hệ thống máy tính. Ví dụ ngôn ngữ sơ đồ khối, ngôn ngữ tựa tự nhiên, ngôn ngữ đặc tả. Đặc điểm chung của các loại ngôn ngữ hình thức là việc sử dụng nó rất gần gũi với ngôn ngữ tự nhiên, rất gần gũi với ngôn ngữ máy tính. Tuy nhiên, ngôn ngữ hình thức lại không phụ thuộc vào ngôn ngữ tự nhiên, không phụ thuộc vào ngôn ngữ máy tính. Chính vì lý do này, ngôn ngữ hình thức được sử dụng phổ biến trong biểu diễn thuật toán.

Ví dụ dưới đây sẽ minh họa cho các ngôn ngữ biểu diễn thuật toán.

//Ví dụ 1.3.Biểu diễn thuật toán bằng ngôn ngữ tự nhiên

Đầu vào (Input). Hai số tự nhiên a, b .

Đầu ra (Output). Số nguyên u lớn nhất để a và b đều chia hết cho u .

Thuật toán (Euclidean Algorithm):

Bước 1. Đưa vào hai số tự nhiên a và b .

Bước 2. Nếu $b \neq 0$ thì chuyển đến bước 3, nếu $b=0$ thì thực hiện bước 4.

Bước 3. Đặt $r = a \bmod b$; $a = b$; $b = r$; Quay quay trở lại bước 2.

Bước 4 (Output). Kết luận $u=a$ là số nguyên cần tìm.

//Ví dụ 1.4.Biểu diễn thuật toán bằng ngôn ngữ máy tính (C++)

```
int USCLN( int a, int b) {  
    while ( b != 0 ) {//lặp trong khi b khác 0  
        r = a % b; //đặt r bằng phần dư của a/b  
        a = b; //đặt a bằng b  
        b = r; //đặt b bằng r  
    }  
    return(a);//trả lại giá trị a  
}
```

//Ví dụ 1.5.Biểu diễn thuật toán bằng ngôn ngữ hình thức

Thuật toán Euclide:

Đầu vào (Input): $a \in \mathbb{N}, a \in \mathbb{N}$.

Đầu ra (Output): $s = \max \{ u \in \mathbb{N} : a \bmod u = 0 \text{ and } b \bmod u = 0 \}$.

Format : $s = \text{Euclide}(a, b)$.

Actions :

```
while ( b  $\neq$  0 ) do //lặp trong khi b khác 0  
    r = a mod b; //đặt r bằng a mod b  
    a = b; //đổi giá trị của a thành b  
    b = r; // đổi giá trị của b thành r  
endwhile;//kết thúc cấu trúc lặp while  
return(a);//giá trị trả về của hàm
```

Endactions.

Một số lưu ý trong khi biểu diễn thuật toán bằng ngôn ngữ hình thức:

- Khi biểu diễn bằng ngôn ngữ hình thức ta được phép sử dụng cả ngôn ngữ tự nhiên hoặc ngôn ngữ máy tính thông dụng. Mỗi bước thực hiện của thuật toán không cần mô tả quá chi tiết mà chỉ cần mô tả một cách hình thức miễn là đầy đủ thông tin để có thể dễ dàng chuyển đổi sang ngôn ngữ lập trình.
- Đối với những thuật toán phức tạp nặng nề về tính toán, các công thức cần được mô tả một cách tường minh, có ghi chú rõ ràng.
- Đối với các thuật toán kinh điển thì ta cần phải học thuộc. Không bắt buộc phải chứng minh lại độ phức tạp của các thuật toán kinh điển. Tuy nhiên ta có thể sử dụng kết quả này để chứng minh độ phức tạp cho một thuật toán khác.

1.3. Độ phức tạp thuật toán

Một bài toán có thể thực hiện bằng nhiều thuật toán khác nhau. Lựa chọn giải thuật nhanh nhất để giải quyết bài toán là một nhu cầu của thực tế. Vì vậy ta cần phải có một ước lượng cụ thể bằng toán học để xác định mức độ nhanh chậm của mỗi giải thuật.

1.3.1. Khái niệm độ phức tạp thuật toán

Thời gian thực hiện một giải thuật bằng chương trình máy tính phụ thuộc vào các yếu tố:

- Kích thước dữ liệu đầu vào: một giải thuật hay một chương trình máy tính thực hiện trên tập dữ liệu có kích thước lớn hiển nhiên mất nhiều thời gian hơn thuật toán hoặc chương trình này thực hiện trên tập dữ liệu đầu vào có kích thước nhỏ.
- Phần cứng của hệ thống máy tính: hệ thống máy tính có tốc độ cao thực hiện nhanh hơn trên hệ thống máy tính có tốc độ thấp.

Tuy nhiên, nếu ta quan niệm thời gian thực hiện của một thuật toán là số các phép toán sơ cấp thực hiện trong thuật toán đó thì phần cứng máy tính không còn là yếu tố ảnh hưởng đến quá trình xác định thời gian thực hiện của một thuật toán. Với quan niệm này, độ phức tạp thời gian thực hiện của một thuật toán chỉ còn phụ thuộc duy nhất vào độ dài dữ liệu đầu vào.

Gọi độ dài dữ liệu đầu vào là $T(n)$. Khi đó, số lượng các phép toán sơ cấp để giải bài toán P thực hiện theo thuật toán $F = F_1 F_2 \dots F_n$ trên độ dài dữ liệu $T(n)$ là $F(T(n))$. Để xác định số lượng các phép toán sơ cấp $F_i (i = 1, 2, \dots, n)$ thực hiện trong thuật toán F ta cần phải giải bài toán đếm để xác định $F(T(n))$. Đây là bài toán vô cùng khó và không phải lúc nào cũng giải được []. Để đơn giản điều này, người ta thường tìm đến các phương pháp xấp xỉ để tính toán độ phức tạp thời gian của một thuật toán. Điều này có nghĩa, khi ta không thể xây dựng được công thức đếm $F(T(n))$, nhưng ta lại có khẳng định chắc chắn $F(T(n))$ không vượt quá một phiếm hàm biết trước $G(n)$ thì ta nói $F(T(n))$ thực hiện nhanh nhất là $G(n)$.

Tổng quát, cho hai hàm $f(x)$, $g(x)$ xác định trên tập các số nguyên dương hoặc tập các số thực. Hàm $f(x)$ được gọi là $O(g(x))$ nếu tồn tại một hằng số $C > 0$ và n_0 sao cho:

$$|f(x)| \leq C \cdot |g(x)| \text{ với mọi } x \geq n_0.$$

Điều này có nghĩa với các giá trị $x \geq n_0$ hàm $f(x)$ bị chặn trên bởi hằng số C nhân với $g(x)$. Nếu $f(x)$ là thời gian thực hiện của một thuật toán thì ta nói giải thuật đó có cấp $g(x)$ hay độ phức tạp thuật toán $f(x)$ là $O(g(x))$.

Ghi chú. Các hằng số C, n_0 thỏa mãn điều kiện trên là không duy nhất. Nếu có đồng thời $f(x)$ là $O(g(x))$ và $h(x)$ thỏa mãn $g(x) < h(x)$ với $x > n_0$ thì ta cũng có $f(x)$ là $O(h(x))$.

Ví dụ 1.5. Cho $f(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$; trong đó, a_i là các số thực ($i = 0, 1, 2, \dots, n$). Khi đó $f(x) = O(x^n)$.

Chứng minh. Thực vậy, với mọi $x > I$ ta có:

$$\begin{aligned} |f(x)| &= |a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0| \\ &\leq |a_n| x^n + |a_{n-1}| x^{n-1} + \dots + |a_1| x + |a_0| \\ &\leq |a_n| x^n + |a_{n-1}| x^n + \dots + |a_1| x^n + |a_0| x^n \\ &\leq x^n (|a_n| + |a_{n-1}| + \dots + |a_1| + |a_0|) \\ &\leq C \cdot x^n = O(x^n). \text{ Trong đó, } C = (|a_n| + |a_{n-1}| + \dots + |a_1| + |a_0|). \end{aligned}$$

1.3.2. Một số qui tắc xác định độ phức tạp thuật toán

Như đã đề cập ở trên, bản chất của việc xác định độ phức tạp thuật toán là giải bài toán đếm số lượng các phép toán sơ cấp thực hiện trong thuật toán đó. Do vậy, tất cả các phương pháp giải bài toán đếm thông thường đều được áp dụng trong khi xác định độ phức tạp thuật toán. Hai nguyên lý cơ bản để giải bài toán đếm là nguyên lý cộng và nguyên lý nhân cũng được mở rộng trong khi ước lượng độ phức tạp thuật toán.

Nguyên tắc tổng: Nếu $f_1(x)$ có độ phức tạp là $O(g_1(x))$ và $f_2(x)$ có độ phức tạp là $O(g_2(x))$ thì độ phức tạp của $(f_1(x) + f_2(x))$ là $O(\max(g_1(x), g_2(x)))$.

Chứng minh. Vì $f_1(x)$ có độ phức tạp là $O(g_1(x))$ nên tồn tại hằng số C_1 và k_1 sao cho $|f_1(x)| \leq |g_1(x)|$ với mọi $x \geq k_1$. Vì $f_2(x)$ có độ phức tạp là $O(g_2(x))$ nên tồn tại hằng số C_2 và k_2 sao cho $|f_2(x)| \leq |g_2(x)|$ với mọi $x \geq k_2$.

Ta lại có :

$$\begin{aligned} |f_1(x) + f_2(x)| &\leq |f_1(x)| + |f_2(x)| \\ &\leq C_1 |g_1(x)| + C_2 |g_2(x)| \\ &\leq C |g(x)| \text{ với mọi } x > k; \end{aligned}$$

Trong đó, $C = C_1 + C_2$; $g(x) = \max(g_1(x), g_2(x))$; $k = \max(k_1, k_2)$.

Tổng quát. Nếu độ phức tạp của $f_1(x), f_2(x), \dots, f_m(x)$ lần lượt là $O(g_1(x)), O(g_2(x)), \dots, O(g_m(x))$ thì độ phức tạp của $f_1(x) + f_2(x) + \dots + f_m(x)$ là $O(\max(g_1(x), g_2(x), \dots, g_m(x)))$.

Nguyên tắc nhân: Nếu $f(x)$ có độ phức tạp là $O(g(x))$ thì độ phức tạp của $f^n(x)$ là $O(g^n(x))$. Trong đó:

$$f^n(x) = f(x).f(x) \dots f(x). // n \text{ lần } f(x).$$

$$g^n(x) = g(x).g(x) \dots g(x). // n \text{ lần } g(x)$$

Chứng minh. Thật vậy theo giả thiết $f(x)$ là $O(g(x))$ nên tồn tại hằng số C và k sao cho với mọi $x > k$ thì $|f(x)| \leq C.g(x)$. Ta có:

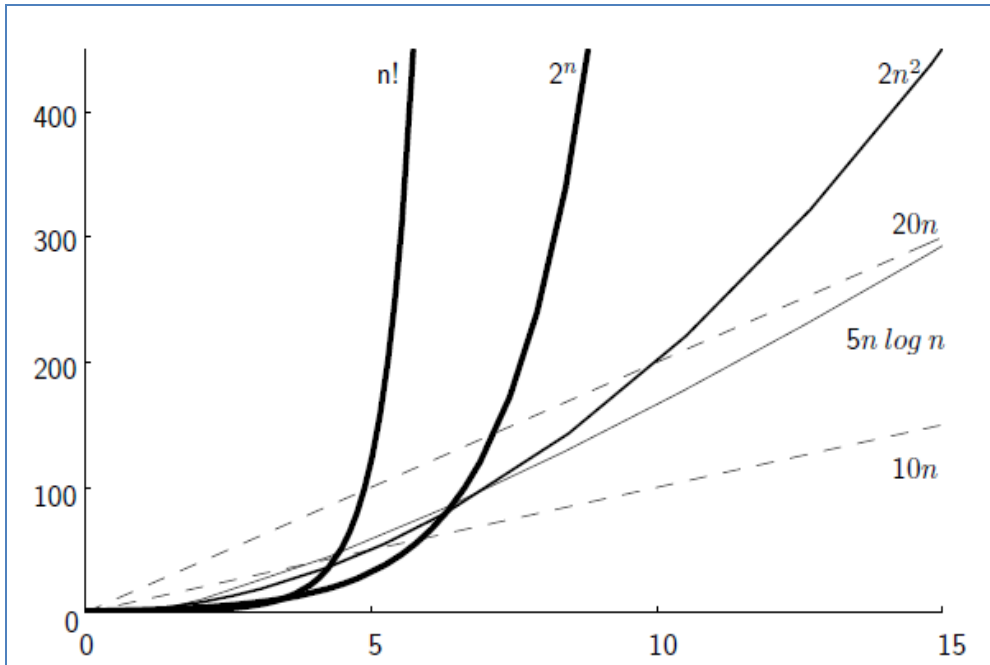
$$\begin{aligned} |f^n(x)| &= |f^1(x).f^2(x) \dots f^n(x)| \\ &\leq |C.g^1(x).C.g^2(x) \dots C.g^n(x)| \\ &\leq |C^n.g^n(x)| = O(g^n(x)) \end{aligned}$$

1.3.3. Một số dạng hàm được dùng xác định độ phức tạp thuật toán

Như đã đề cập ở trên, để xác định chính xác độ phức tạp thuật toán $f(x)$ là bài toán khó nên ta thường xấp xỉ độ phức tạp thuật toán với một phép hàm $O(g(x))$. Dưới đây là một số phép hàm của $O(g(x))$.

Bảng 1.1. Các dạng hàm xác định độ phức tạp thuật toán

Dạng phép hàm	Tên gọi
$O(1)$	Hằng số
$O(\log(\log(n)))$	Logarit của logarit
$O(\log(n))$	Logarithm
$O(n)$	Tuyến tính
$O(n^2)$	Bậc 2
$O(n^3)$	Bậc 3
$O(n^m)$	Đa thức (m là hằng số)
$O(m^n)$	Hàm mũ
$O(n!)$	Giai thừa



Hình 1.1. Độ tăng của các hàm theo độ dài dữ liệu

Dưới đây là một số qui tắc xác định $O(g(x))$:

- Nếu một thuật toán có độ phức tạp hằng số thì thời gian thực hiện thuật toán đó không phụ thuộc vào độ dài dữ liệu.
- Một thuật toán có độ phức tạp logarit của $f(n)$ thì ta viết $O(\log(n))$ mà không cần chỉ rõ cơ số của phép logarit.
- Với $P(n)$ là một đa thức bậc k thì $O(P(n)) = O(n^k)$.
- Thuật toán có độ phức tạp đa thức hoặc nhỏ hơn được xem là những thuật toán thực tế có thể thực hiện được bằng máy tính. Các thuật toán có độ phức tạp hàm mũ, hàm giai thừa được xem là những thuật toán thực tế không giải được bằng máy tính.

1.4. Độ phức tạp của các cấu trúc lệnh

Để đánh giá độ phức tạp của một thuật toán đã được mã hóa thành chương trình máy tính ta thực hiện theo một số qui tắc sau.

Độ phức tạp hằng số $O(1)$: đoạn chương trình không chứa vòng lặp hoặc lời gọi đệ qui có tham biến là một hằng số.

Ví dụ 1.7. Đoạn chương trình dưới đây có độ phức tạp hằng số.

```
for (i=1; i<=c; i++){
    <Tập các chỉ thị có độ phức tạp O(1)>;
}
```

Ví dụ 1.8. Đoạn code dưới đây có độ phức tạp tuyến tính $O(n)$.

```
for (i=1; i<=0; i +=c ) {
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}
for (i=n; i>0; i = i - c ){
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}
```

Độ phức tạp đa thức $O(n^c)$: Độ phức tạp của c vòng lặp lồng nhau, mỗi vòng lặp đều có độ phức tạp $O(n)$ là $O(n^c)$.

Ví dụ 1.9. Đoạn code dưới đây có độ phức tạp $O(n^2)$.

```
for (i=1; i<=n; i = i + c ) {
    for (j=1; j<=n; j = j + c ){
        <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
    }
}
for (i = n; i >0 ; i = i - c ) {
    for (j = i- 1; j>1; j = j -c ){
        <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
    }
}
```

Độ phức tạp logarit $O(\log(n))$: Độ phức tạp của vòng lặp là $\log(n)$ nếu biểu thức khởi đầu lại của vòng lặp được chia hoặc nhân với một hằng số c .

Ví dụ 1.10. Đoạn code dưới đây có độ phức tạp $\log(n)$.

```
for (i=1; i <=n; i = i * c ){
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}
for (j=n; j >0 ; j = j / c ){
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}
```

Độ phức tạp hằng số $O(\log(\log(n)))$: nếu biểu thức khởi đầu lại của vòng lặp được nhân hoặc chia cho một hàm mũ.

Ví dụ 1.11. Đoạn code dưới đây có độ phức tạp $\log \log(n)$.

```
for (i=1; j<=n; j*= Pow(i, c) ){
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}
for (j=n; j>=0; j = j- Function(j) ){ //Function(j) =sqrt(j) hoặc lớn hơn 2.
    <Tập các chỉ thị có độ phức tạp  $O(1)$ >;
}
```

}

Độ phức tạp của chương trình: độ phức tạp của một chương trình bằng số lần thực hiện một chỉ thị tích cực trong chương trình đó. Trong đó, một chỉ thị được gọi là tích cực trong chương trình nếu chỉ thị đó phụ thuộc vào độ dài dữ liệu và thực hiện không ít hơn bất kỳ một chỉ thị nào khác trong chương trình.

Ví dụ 1.12. Tìm độ phức tạp thuật toán sắp xếp kiểu Bubble-Sort?

```
Void Bubble-Sort ( int A[], int n ) {
    for ( i=1; i<n; i++) {
        for ( j = i+1; j<=n; j++) {
            if (A[i] > A[j]) {//đây chính là chỉ thị tích cực
                t = A[i]; A[i] = A[j]; A[j] = t;
            }
        }
    }
}
```

Lời giải. Sử dụng trực tiếp nguyên lý cộng ta có:

- Với $i = 1$ ta cần sử dụng $n-1$ phép so sánh $A[i]$ với $A[j]$;
- Với $i = 2$ ta cần sử dụng $n-1$ phép so sánh $A[i]$ với $A[j]$;
-
- Với $i = n-1$ ta cần sử dụng 1 phép so sánh $A[i]$ với $A[j]$;

Vì vậy tổng số các phép toán cần thực hiện là:

$$S = (n-1) + (n-2) + \dots + 2 + 1 = n(n-1)/2 \leq n^2 = O(n^2).$$

Ghi chú. Độ phức tạp thuật toán cũng là số lần thực hiện phép toán tích cực. Phép toán tích cực là phép toán thực hiện nhiều nhất đối với thuật toán.

1.5. Quy trình giải quyết bài toán trên máy tính

Để giải quyết một bài toán hoặc vấn đề trong tin học ta thực hiện thông qua 6 bước như sau:

Bước 1. Xác định yêu cầu bài toán. Xem xét bài toán cần xử lý vấn đề gì? Giả thiết nào đã được biết trước và lời giải cần đạt những yêu cầu gì? Ví dụ thời gian, hay không gian nhớ.

Bước 2. Tìm cấu trúc dữ liệu thích hợp biểu diễn các đối tượng cần xử lý của bài toán. Cấu trúc dữ liệu phải biểu diễn đầy đủ các đối tượng thông tin vào của bài toán. Các thao tác trên cấu trúc dữ liệu phải phù hợp với những thao tác của thuật toán được lựa chọn. Cấu trúc dữ liệu phải cài đặt được bằng ngôn ngữ lập trình cụ thể đáp ứng yêu cầu bài toán.

Bước 3. Lựa chọn thuật toán. Thuật toán phải đáp ứng được yêu cầu của bài toán và phù hợp với cấu trúc dữ liệu đã được lựa chọn Bước 1.

Bước 3. Cài đặt thuật toán. Thuật toán cần được cài đặt bằng một ngôn ngữ lập trình cụ thể. Ngôn ngữ lập trình sử dụng phải có các cấu trúc dữ liệu đã lựa chọn.

Bước 4. Kiểm thử chương trình. Thử nghiệm thuật toán (chương trình) trên các bộ dữ liệu thực. Các bộ dữ liệu cần phải bao phủ lên tất cả các trường hợp của thuật toán.

- **Tối ưu chương trình:** Cải tiến để chương trình tốt hơn.

.

PDF

CHƯƠNG 2. MÔ HÌNH THUẬT TOÁN SINH

Mô hình thuật toán sinh dùng để giải quyết lớp các bài toán liệt kê. Một số bài toán quan trọng của lý thuyết tổ hợp được giải quyết bằng mô hình thuật toán sinh như liệt kê các tập con của tập gồm n phần tử, liệt kê tổ hợp hay liệt kê hoán vị. Nội dung chính của chương này cung cấp cho người học những vấn đề cơ bản của mô hình thuật toán sinh. Trên cơ sở đó, người học có thể sử dụng mô hình thuật toán sinh để giải quyết các bài toán từ cơ bản đến nâng cao.

2.1 Giới thiệu thuật toán sinh

Mô hình thuật toán sinh được dùng để giải lớp các bài toán liệt kê, bài toán đếm, bài toán tối ưu, bài toán tồn tại thỏa mãn hai điều kiện:

- **Điều kiện 1:** có thể xác định được một thứ tự trên tập các cấu hình cần liệt kê của bài toán. Biết cấu hình đầu tiên, biết cấu hình cuối cùng.
- **Điều kiện 2:** từ một cấu hình chưa phải cuối cùng, ta xây dựng được thuật toán sinh ra cấu hình đứng ngay sau nó.

Mô hình thuật toán sinh được biểu diễn thành hai bước: bước khởi tạo và bước lặp. Tại bước khởi tạo, cấu hình đầu tiên của bài toán sẽ được thiết lập. Điều này bao giờ cũng thực hiện được theo giả thiết của bài toán. Tại bước lặp, quá trình lặp được thực hiện khi gặp phải cấu hình cuối cùng. Điều kiện lặp của bài toán bao giờ cũng tồn tại theo giả thiết của bài toán. Hai chỉ thị cần thực hiện trong thân vòng lặp là đưa ra cấu hình hiện tại và sinh ra cấu hình kế tiếp. Mô hình sinh kế tiếp được thực hiện tùy thuộc vào mỗi bài toán cụ thể. Mô hình thuật toán sinh được biểu diễn cụ thể trong hình 2.1.

Thuật toán Generation:

begin

Bước1 (Khởi tạo):

<Thiết lập cấu hình đầu tiên>;

Bước 2 (Bước lặp):

while(*<Lặp khi cấu hình chưa phải cuối cùng>*)**do**

<Đưa ra cấu hình hiện tại>;

<Sinh ra cấu hình kế tiếp>;

endwhile;

End.

Hình 2.1. Mô hình thuật toán sinh

Ví dụ . Vector $X = (x_1, x_2, \dots, x_n)$, trong đó $x_i = 0, 1$ được gọi là một chuỗi nhị phân có độ dài n . Hãy liệt kê các chuỗi nhị phân có độ dài n . Ví dụ với $n=4$, ta sẽ liệt kê được 24 chuỗi nhị phân độ dài 4 như trong Bảng 2.1.

Bảng 2.1. Các chuỗi nhị phân độ dài 4

STT	$X=(x_1, x_2, x_3, x_4)$	STT	$X=(x_1, x_2, x_3, x_4)$
0	0 0 0 0	8	1 0 0 0
1	0 0 0 1	9	1 0 0 1
2	0 0 1 0	10	1 0 1 0
3	0 0 1 1	11	1 0 1 1
4	0 1 0 0	12	1 1 0 0
5	0 1 0 1	13	1 1 0 1
6	0 1 1 0	14	1 1 1 0
7	0 1 1 1	15	1 1 1 1

Lời giải:

Điều kiện 1: Gọi thứ tự của chuỗi nhị phân $X=(x_1, x_2, \dots, x_n)$ là $f(X)$. Trong đó, $f(X)=k$ là số chuyển đổi chuỗi nhị X thành số ở hệ cơ số 10. Ví dụ, chuỗi $X = (1, 0, 1, 1)$ được chuyển thành số hệ cơ số 10 là 11 thì ta nói chuỗi X có thứ tự 11. Với cách quan niệm này, chuỗi đứng sau chuỗi có thứ tự 11 là 12 chính là chuỗi đứng ngay sau chuỗi $X = (1, 0, 1, 1)$. Chuỗi đầu tiên có thứ tự là 0 ứng với chuỗi có n số 0. Chuỗi cuối cùng có thứ tự là 2^n-1 ứng với chuỗi có n số 1. Như vậy, điều kiện 1 của thuật toán sinh đã được thỏa mãn.

Điều kiện 2: Về nguyên tắc ta có thể lấy $k = f(X)$ là thứ tự của một chuỗi bất kỳ theo nguyên tắc ở trên, sau đó lấy thứ tự của chuỗi kế tiếp là $(k + 1)$ và chuyển đổi $(k+1)$ thành số ở hệ cơ số 10 ta sẽ được chuỗi nhị phân tiếp theo. Chuỗi cuối cùng sẽ là chuỗi có n số 1 ứng với thứ tự $k = 2^n-1$. Với cách làm này, ta có thể coi mỗi chuỗi nhị phân là một số, mỗi thành phần của chuỗi là một bit và chỉ cần cài đặt thuật toán chuyển đổi cơ số ở hệ 10 thành số ở hệ nhị phân. Ta có thể xây dựng thuật toán tổng quát hơn bằng cách xem mỗi chuỗi nhị phân là một mảng các phần tử có giá trị 0 hoặc 1. Sau đó, duyệt từ vị trí bên phải nhất của chuỗi nếu gặp số 1 ta chuyển thành 0 và gặp số 0 đầu tiên ta chuyển thành 1. Ví dụ với chuỗi $X = (0, 1, 1, 1)$ được chuyển thành chuỗi $X = (1, 0, 0, 0)$, chuỗi $X = (1, 0, 0, 0)$ được chuyển thành chuỗi $X = (1, 0, 0, 1)$. Lời giải và thuật toán sinh chuỗi nhị phân kế tiếp được thể hiện trong chương trình dưới đây. Trong đó, thuật toán sinh chuỗi nhị phân kế tiếp từ một chuỗi nhị phân bất kỳ là hàm `Next_Bits_String()`.

```
#include <iostream>
#include <iomanip>
```

```
#define MAX 100
using namespace std;
int X[MAX], n, dem = 0; //sử dụng các biến toàn cục X[], n, OK, dem
bool OK =true;
void Init(void){ //khởi tạo xâu nhị phân đầu tiên
    cout<<"Nhập n="; cin>>n;
    for(int i = 1; i<=n; i++) //thiết lập xâu với n số 0
        X[i]=0;
}
void Result(void){ //đưa ra xâu nhị phân hiện tại
    cout<<"\n Xâu thứ "<<dem<<": ";
    for(int i=1; i<=n; i++)
        cout<<X[i]<<setw(3);
}
void Next_Bits_String(void){ //thuật toán sinh xâu nhị phân kế tiếp
    int i=n;
    while(i>0 && X[i]){ //duyet từ vị trí bên phải nhất
        X[i]=0; //nếu gặp X[i] = 1 ta chuyển thành 0
        i--; //lùi lại vị trí sau
    }
    if (i>0)X[i]=1; //gặp X[i] =0 đầu tiên ta chuyển thành 1
    else OK = false; //kết thúc khi gặp xâu có n số 1
}
int main(void){ //đây là thuật toán sinh
    Init(); //thiết lập cấu hình đầu tiên
    while(OK){ //lặp khi chưa phải cấu hình cuối cùng
        Result(); //đưa ra cấu hình hiện tại
        Next_Bits_String(); //sinh ra cấu hình kế tiếp
    }
}
```

2.2. Một số bài toán xử lý bằng thuật toán sinh cơ bản

Dưới đây là một số bài toán xử lý bằng thuật toán sinh cơ bản:

1. Cho xâu nhị phân $X[]$, nhiệm vụ của bạn là hãy đưa ra xâu nhị phân tiếp theo của $X[]$. Ví dụ $X[] = "010101"$ thì xâu nhị phân tiếp theo của $X[]$ là "010110".

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu nhị phân X .

- T, X[] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(X) \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2 010101 111111	010110 000000

2. Cho hai số N, K và một tập con K phần tử $X[] = (X_1, X_2, \dots, X_K)$ của $1, 2, \dots, N$. Nhiệm vụ của bạn là hãy đưa ra tập con K phần tử tiếp theo của X[]. Ví dụ $N=5, K=3, X[] = \{2, 3, 4\}$ thì tập con tiếp theo của X[] là $\{2, 3, 5\}$.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là hai số N và K; dòng tiếp theo đưa vào K phần tử của X[] là một tập con K phần tử của $1, 2, \dots, N$.
- T, K, N, X[] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq K \leq N \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2 5 3 1 4 5 5 3 3 4 5	2 3 4 1 2 3

3. Cho số tự nhiên N và một hoán vị X[] của $1, 2, \dots, N$. Nhiệm vụ của bạn là đưa ra hoán vị tiếp theo của X[]. Ví dụ $N=5, X[] = \{1, 2, 3, 4, 5\}$ thì hoán vị tiếp theo của X[] là $\{1, 2, 3, 5, 4\}$.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là số N; dòng tiếp theo đưa vào hoán vị X[] của $1, 2, \dots, N$.
- T, N, X[] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	1 2 3 5 4
5	1 2 3 4 5
1 2 3 4 5	
5	
5 4 3 2 1	

4. Xâu ký tự str được gọi là xâu AB nếu mỗi ký tự trong xâu hoặc là ký tự ‘A’ hoặc là ký tự ‘B’. Ví dụ xâu str=”ABBABB” là xâu AB độ dài 6. Nhiệm vụ của bạn là hãy liệt kê tất cả các xâu AB có độ dài n.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên n.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 10$; $1 \leq n \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	AA AB BA BB
2	AAA AAB ABA ABB BAA BAB BBA BBB
3	

5. Cho hai số nguyên dương N và K. Nhiệm vụ của bạn là hãy liệt kê tất cả các tập con K phần tử của 1, 2, ..., N. Ví dụ với N=5, K=3 ta có 10 tập con của 1, 2, 3, 4, 5 như sau: {1, 2, 3}, {1, 2, 4}, {1, 2, 5}, {1, 3, 4}, {1, 3, 5}, {1, 4, 5}, {2, 3, 4}, {2, 3, 5}, {2, 4, 5}, {3, 4, 5}.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một cặp số tự nhiên N, K được viết trên một dòng.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq k \leq n \leq 15$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	123 124 134 234
4 3	123 124 125 134 135 145 234 235 245 345

5 3	
-----	--

6. Cho số nguyên dương N . Nhiệm vụ của bạn là hãy liệt kê tất cả các hoán vị của $1, 2, \dots, N$. Ví dụ với $N = 3$ ta có kết quả: 123, 132, 213, 231, 312, 321.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên N được viết trên một dòng.
- T, n thỏa mãn ràng buộc: $1 \leq T, N \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	12 21
2	123 132 213 231 312 321
3	

7. Cho số nguyên dương N . Nhiệm vụ của bạn là hãy liệt kê tất cả các cách phân tích số tự nhiên N thành tổng các số tự nhiên nhỏ hơn hoặc bằng N . Phép hoán vị vữa một cách được xem là giống nhau. Ví dụ với $N = 5$ ta có kết quả là: (5), (4, 1), (3, 2), (3, 1, 1), (2, 2, 1), (2, 1, 1, 1), (1, 1, 1, 1, 1).

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên N được viết trên một dòng.
- T, n thỏa mãn ràng buộc: $1 \leq T, N \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	(4) (3, 1) (2, 2) (2, 1, 1) (1, 1, 1, 1)
4	(5) (4, 1) (3, 2) (3, 1, 1) (2, 2, 1) (2, 1, 1, 1) (1, 1, 1, 1, 1)
5	

2.3. Một số bài toán xử lý bằng thuật toán sinh nâng cao

Dưới đây là một số bài toán xử lý bằng thuật toán sinh nâng cao :

8. Cho xâu nhị phân $X[]$, nhiệm vụ của bạn là hãy đưa ra xâu nhị phân trước đó của $X[]$. Ví dụ $X[] = "111111"$ thì xâu nhị phân trước của $X[]$ là "000000". Với xâu $X[] = "000001"$ thì xâu nhị phân trước của $X[]$ là "000000".

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu nhị phân X .
- $T, X[]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(X) \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	010100
010101	000000
111111	

9. Cho hai số N, K và một tập con K phần tử $X[] = (X_1, X_2, \dots, X_K)$ của $1, 2, \dots, N$. Nhiệm vụ của bạn là hãy đưa ra tập con K phần tử trước đó của $X[]$. Ví dụ $N=5, K=3, X[] = \{2, 3, 5\}$ thì tập con tiếp theo của $X[]$ là $\{2, 3, 4\}$.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là hai số N và K ; dòng tiếp theo đưa vào K phần tử của $X[]$ là một tập con K phần tử của $1, 2, \dots, N$.
- $T, K, N, X[]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq K \leq N \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	1 4 5
5 3	3 4 5
2 3 4	
5 3	
1 2 3	

10. Cho số tự nhiên N và một hoán vị $X[]$ của $1, 2, \dots, N$. Nhiệm vụ của bạn là đưa ra hoán vị trước đó của $X[]$. Ví dụ $N=5, X[] = \{1, 2, 3, 4, 5\}$ thì hoán vị trước đó của $X[]$ là $\{5, 4, 3, 2, 1\}$.

Input:

- Dòng đầu tiên đưa vào số lượng test T .

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là số N ; dòng tiếp theo đưa vào hoán vị $X[]$ của $1, 2, \dots, N$.
- $T, N, X[]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	1 2 3 4 5
5	5 4 3 1 2
1 2 3 5 4	
5	
5 4 3 2 1	

11. Cho số tự nhiên N . Nhiệm vụ của bạn là in ra các xâu nhị phân có độ dài N theo thứ tự ngược. Ví dụ với $N = 2$ ta có kết quả $\{11, 10, 01, 00\}$

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên n .
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 10$; $1 \leq n \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	11 10 01 00
2	111 110 101 100 011 010 001 000
3	

12. Cho hai số nguyên dương N và K . Nhiệm vụ của bạn là hãy liệt kê tất cả các tập con K phần tử của $1, 2, \dots, N$ theo thứ tự ngược. Ví dụ với $N=5, K=3$ ta có 10 tập con của $1, 2, 3, 4, 5$ theo thứ tự ngược như sau: $\{3, 4, 5\}, \{2, 4, 5\}, \{2, 3, 5\}, \{2, 3, 4\}, \{1, 4, 5\}, \{1, 3, 5\}, \{1, 3, 4\}, \{1, 2, 5\}, \{1, 2, 4\}, \{1, 2, 3\}$.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một cặp số tự nhiên N, K được viết trên một dòng.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq k \leq n \leq 15$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	234 134 124 123
4 3	345 245 235 234 145 135 134 125 124 123
5 3	

13. Cho số nguyên dương N . Nhiệm vụ của bạn là hãy liệt kê tất cả các hoán vị của 1, 2, ..., N theo thứ tự ngược. Ví dụ với $N = 3$ ta có kết quả: 321, 312, 231, 213, 132, 123.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên N được viết trên một dòng.
- T, n thỏa mãn ràng buộc: $1 \leq T, N \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	21 12
2	321 312 231 213 132 123
3	

2.4. Một số bài toán xử lý bằng thuật toán sinh tiên tiến

Dưới đây là một số bài toán xử lý bằng thuật toán sinh tiên tiến.

14. Số nhị phân được xem là cách mặc định biểu diễn các số. Tuy nhiên, trong nhiều ứng dụng của điện tử và truyền thông lại dùng một biến thể của mã nhị phân đó là mã Gray. Mã Gray độ dài n có mã đầu tiên là n số 0, mã kế tiếp của nó là một xâu nhị phân độ dài n khác biệt với xâu trước đó một bit. Ví dụ với $n=3$ ta có 23 mã Gray như sau: 000, 001, 011, 010, 110, 111, 101, 100. Hãy viết chương trình liệt kê các mã Gray có độ dài n .

Input:

- Dòng đầu tiên là số lượng test T .
- T dòng kế tiếp ghi lại mỗi dòng một test. Mỗi test là một số tự nhiên n .
- T, n thỏa mãn ràng buộc: $1 \leq T, n \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	000 001 011 010 110 111 101 100
3	0000 0001 0011 0010 0110 0111 0101 0100 1100 1101 1111 1110 1010 1011 1001 1000
4	

15. Số nhị phân được xem là cách mặc định biểu diễn các số. Tuy nhiên, trong nhiều ứng dụng của điện tử và truyền thông lại dùng một biến thể của mã nhị phân đó là mã Gray. Mã Gray độ dài n có mã đầu tiên là n số 0, mã kế tiếp của nó là một chuỗi nhị phân độ dài n khác biệt với chuỗi trước đó một bit. Ví dụ với $n=3$ ta có 23 mã Gray như sau: 000, 001, 011, 010, 110, 111, 101, 100. Hãy viết chương trình chuyển đổi một chuỗi mã nhị phân X có độ dài n thành một chuỗi mã Gray.

Input:

- Dòng đầu tiên là số lượng test T .
- T dòng kế tiếp ghi lại mỗi dòng một test. Mỗi test là một chuỗi nhị phân độ dài n .
- T, n thỏa mãn ràng buộc: $1 \leq T, n \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples:	Output examples:
2	01101
01001	01011
01101	

16. Số nhị phân được xem là cách mặc định biểu diễn các số. Tuy nhiên, trong nhiều ứng dụng của điện tử và truyền thông lại dùng một biến thể của mã nhị phân đó là mã Gray. Mã Gray độ dài n có mã đầu tiên là n số 0, mã kế tiếp của nó là một chuỗi nhị phân độ dài n khác biệt với chuỗi trước đó một bit. Ví dụ với $n=3$ ta có 23 mã Gray như sau: 000, 001, 011, 010, 110, 111, 101, 100. Hãy viết chương trình chuyển đổi một chuỗi mã Gray X có độ dài n thành một chuỗi mã nhị phân.

Input::

- Dòng đầu tiên là số lượng test T .
- T dòng kế tiếp ghi lại mỗi dòng một test. Mỗi test là một chuỗi mã Gray độ dài n .
- T, n thỏa mãn ràng buộc: $1 \leq T, n \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	01001
01101	01101
01011	

CHƯƠNG 3. MÔ HÌNH THUẬT TOÁN ĐỆ QUI & QUAY LUI

Đệ quy (Recursion) là sự mở rộng của khái niệm qui nạp toán học khá quen thuộc trong lập trình. Có một số bài toán, buộc phải sử dụng đệ quy mới giải quyết được, chẳng hạn một số bài toán trên cây. Mô hình thuật toán đệ qui được thể hiện qua nhiều hình thái khác nhau: đệ qui tuyến tính, đệ qui nhị phân, đệ qui tương hỗ và đệ qui lồng. Một biến thể của mô hình đệ qui là mô hình quay lui. Trong đó, chiến lược giải bài toán một cách đầy đủ nhất, đảm bảo đủ mọi trường hợp bằng phương pháp Thử và Sai. Nội dung của chương này sẽ trình bày các kỹ thuật cơ bản và nâng cao bằng kỹ thuật đệ qui.

3.1. Mô hình thuật toán đệ qui

Một đối tượng được định nghĩa trực tiếp hoặc gián tiếp thông qua chính nó được gọi là phép định nghĩa bằng đệ qui. Thuật toán giải bài toán P một cách trực tiếp hoặc gián tiếp thông qua bài toán P' giống như P được gọi là thuật toán đệ qui giải bài toán P . Một hàm được gọi là đệ qui nếu nó được gọi trực tiếp hoặc gián tiếp đến chính nó.

Tổng quát, một bài toán có thể giải được bằng đệ qui nếu nó thỏa mãn hai điều kiện:

- **Phân tích được:** Có thể giải được bài toán P bằng bài toán P' giống như P . Bài toán P' và chỉ khác P ở dữ liệu đầu vào. Việc giải bài toán P' cũng được thực hiện theo cách phân tích giống như P .
- **Điều kiện dừng:** Dãy các bài toán P' giống như P là hữu hạn và sẽ dừng tại một bài toán xác định nào đó.

Thuật toán đệ qui tổng quát có thể được mô tả như sau:

```
Thuật toán Recursion ( P ) {
    1. Nếu P thỏa mãn điều kiện dừng:
        <Giải P với điều kiện dừng>;
    2. Nếu P không thỏa mãn điều kiện dừng:
        <Giải P' giống như P: Recursion(P')>;
}
```

Ví dụ. Tìm tổng của n số tự nhiên đầu tiên bằng phương pháp đệ qui.

Lời giải. Gọi S_n là tổng của n số tự nhiên. Khi đó:

- **Bước phân tích:** dễ dàng nhận thấy tổng n số tự nhiên $S_n = n + S_{n-1}$, với $n \geq 1$.
- **Điều kiện dừng:** $S_0 = 0$ nếu $n = 0$;

Từ đó ta có lời giải của bài toán như sau:

```
int    Tong (int n) {
        if (n == 0) return(0); //Điều kiện dừng
        else return(n + Tong(n-1)); //Điều kiện phân tích được
    }
```

Chẳng hạn ta cần tìm tổng của 5 số tự nhiên đầu tiên, khi đó:

$$\begin{aligned}
 S &= \text{Tong}(5) \\
 &= 5 + \text{Tong}(4) \\
 &= 5 + 4 + \text{Tong}(3) \\
 &= 5 + 4 + 3 + \text{Tong}(2) \\
 &= 5 + 4 + 3 + 2 + \text{Tong}(1) \\
 &= 5 + 4 + 3 + 2 + 1 + \text{Tong}(0) \\
 &= 5 + 4 + 3 + 2 + 1 + 0 \\
 &= 15
 \end{aligned}$$

3.2. Mô hình thuật toán quay lui (Back-track Algorithm)

Giả sử ta cần xác định bộ $X = (x_1, x_2, \dots, x_n)$ thỏa mãn một số ràng buộc nào đó. Ứng với mỗi thành phần x_i ta có n_i khả năng cần lựa chọn. Ứng với mỗi khả năng $j \in n_i$ dành cho thành phần x_i ta cần thực hiện:

- Kiểm tra xem khả năng j có được chấp thuận cho thành phần x_i hay không? Nếu khả năng j được chấp thuận thì ta xác định thành phần x_i theo khả năng j . Nếu i là thành phần cuối cùng ($i=n$) ta ghi nhận nghiệm của bài toán. Nếu i chưa phải cuối cùng ta xác định thành phần thứ $i+1$.
- Nếu không có khả năng j nào được chấp thuận cho thành phần x_i thì ta quay lại bước trước đó ($i-1$) để thử lại các khả năng còn lại.

Thuật toán quay lui được mô tả như sau:

```

Thuật toán Back-Track ( int i ) {
    for ( j = <Khả năng 1>; j <= ni; j++ ) {
        if ( <chấp thuận khả năng j> ) {
            X[i] = <khả năng j>;
            if ( i == n ) Result();
            else Back-Track(i+1);
        }
    }
}

```

Ví dụ. Duyệt các xâu nhị phân có độ dài n .

Lời giải. Xâu nhị phân $X = (x_1, x_2, \dots, x_n) | x_i = 0, 1$. Mỗi $x_i \in X$ có hai lựa chọn $x_i = 0, 1$. Cả hai giá trị này đều được chấp thuận mà không cần có thêm bất kỳ điều kiện gì. Thuật toán được mô tả như sau:

```

void Try ( int i ) {
    for ( int j = 0; j <= 1; j++ ) {
        X[i] = j;
    }
}

```

```

        if ( i == n) Result();
        else Try (i+1);
    }
}

```

3.3. Một số bài toán xử lý bằng thuật toán đệ qui & quay lui cơ bản

Dưới đây là một số bài toán xử lý bằng thuật toán quay lui cơ bản :

1. Cho một chuỗi ký tự S không có ký tự lặp lại. Hãy đưa ra tất cả các tập con của chuỗi ký tự S theo thứ tự tăng dần của các chuỗi ký tự.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một chuỗi ký tự.
- T, S thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(S) \leq 16$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1	a ab abc ac b bc c
3	
abc	

2. **Back-Tracking 1.** Cho dãy số A[] gồm n số nguyên dương. Tam giác đặc biệt của dãy số A[] là tam giác được tạo ra bởi n hàng, trong đó hàng thứ n là dãy số A[], hàng i là tổng hai phần tử liên tiếp của hàng i+1 ($1 \leq i \leq n-1$). Ví dụ A[] = {1, 2, 3, 4, 5}, khi đó tam giác được tạo nên như dưới đây:

```

[48]
[20, 28]
[8, 12, 16]
[3, 5, 7, 9 ]
[1, 2, 3, 4, 5 ]

```

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào N là số lượng phần tử của dãy số A[]; dòng tiếp theo đưa vào N số của mảng A[].
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $A[i] \leq 10$;

Output:

- Đưa ra tam giác tổng của mỗi test theo từng dòng. Mỗi dòng của tam giác tổng được bao bởi ký tự [,].

Input Examples	Output Examples
1 5 1 2 3 4 5	[48] [20 28] [8 12 16] [3 5 7 9] [1 2 3 4 5]

3. Cho chuỗi ký tự S bao gồm các ký tự in hoa khác nhau. Hãy đưa ra tất cả các hoán vị của chuỗi ký tự S. Ví dụ S="ABC" ta có kết quả {ABC ACB BAC BCA CAB CBA }.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test là một chuỗi ký tự S được viết trên 1 dòng.
- T, S thỏa mãn ràng buộc: $1 \leq T \leq 10$; $1 \leq \text{length}(S) \leq 10$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2 AB ABC	AB BA ABC ACB BAC BCA CAB CBA

4. **Back-Tracking 2.** Cho bàn cờ kích thước $N \times N$. Xuất phát tại một ô nào đó, hãy đưa ra một hành trình của quân mã đến tất cả các ô trên bàn cờ mỗi ô đúng một lần.

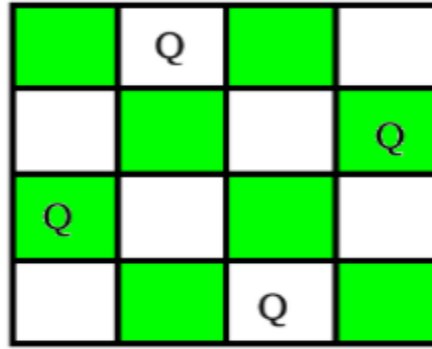
Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một dòng bao gồm bộ ba số N, x, y, trong đó N là kích thước bàn cờ, x và y tương ứng với tọa độ hàng, cột quân mã xuất phát.
- T, N, x, y thỏa mãn ràng buộc: $1 \leq T \leq 100$; $5 \leq N \leq 20$; $0 \leq x, y \leq N$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

5. **Back-Tracking 3.** Cho bàn cờ kích cỡ $N \times N$. Hãy tìm cách đặt N quân hậu trên N hàng khác nhau sao cho tất cả các quân hậu đều không ăn được lẫn nhau. Ví dụ dưới đây là một giải pháp với $N=4$:



Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên N được viết trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T, N \leq 10$.

Output:

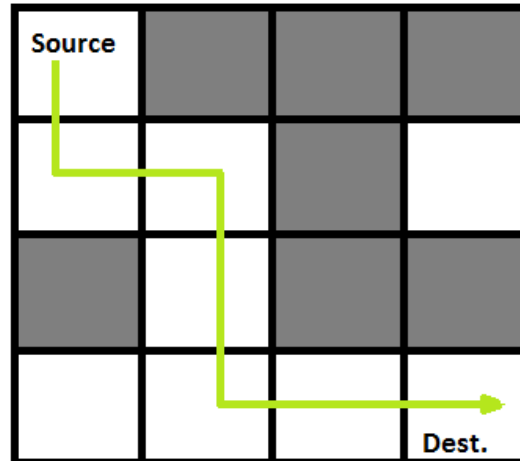
- Đưa ra kết quả mỗi test theo từng dòng, trong đó mỗi giải pháp được đưa ra được đóng bởi ký tự [,]. Nếu không có giải pháp hãy đưa ra -1.

Input Examples	Output Examples
2	-1
2	[2 4 1 3] [3 1 4 2]
4	

6. **Back-Tracking 4.** Cho một mê cung bao gồm các khối được biểu diễn như một ma trận nhị phân $A[N][N]$. Một con chuột đi từ ô đầu tiên góc trái ($A[0][0]$) đến ô cuối cùng góc phải ($A[N-1][N-1]$) theo nguyên tắc:

- Down (D): Chuột được phép xuống dưới nếu ô dưới nó có giá trị 1.
- Right (R): Chuột được phép sang phải dưới nếu ô bên phải nó có giá trị 1.

Hãy đưa ra một hành trình của con chuột trên mê cung. Đưa ra -1 nếu chuột không thể đi đến đích.



Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N là kích cỡ của mê cung; dòng tiếp theo đưa vào ma trận nhị phân $A[N][N]$.
- $T, N, A[i][j]$ thỏa mãn ràng buộc: $1 \leq T \leq 10$; $2 \leq N \leq 10$; $0 \leq A[i][j] \leq 1$.

Output:

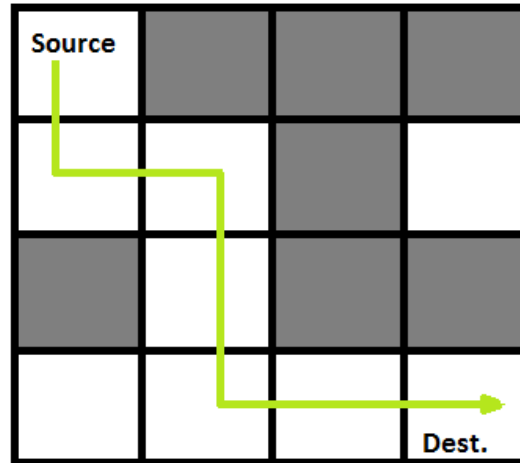
- Đưa ra một đường đi của con chuột trong mê cung. Đưa ra -1 nếu chuột không đi được đến đích.

Input Examples	Output Examples
1 4 1 0 0 0 1 1 0 1 0 1 0 0 1 1 1 1	DRDDRR

7. **Back-Tracking 5.** Cho một mê cung bao gồm các khối được biểu diễn như một ma trận nhị phân $A[N][N]$. Một con chuột đi từ ô đầu tiên góc trái ($A[0][0]$) đến ô cuối cùng góc phải ($A[N-1][N-1]$) theo nguyên tắc:

- Down (D): Chuột được phép xuống dưới nếu ô dưới nó có giá trị 1.
- Right (R): Chuột được phép sang phải dưới nếu ô bên phải nó có giá trị 1.
- Left (L): Chuột được phép sang trái dưới nếu ô bên trái nó có giá trị 1.
- Up (U): Chuột được phép lên trên nếu ô trên nó có giá trị 1.

Hãy đưa ra tất cả các hành trình của con chuột trên mê cung. Đưa ra -1 nếu chuột không thể đi đến đích.



Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N là kích cỡ của mê cung; dòng tiếp theo đưa vào ma trận nhị phân $A[N][N]$.
- $T, N, A[i][j]$ thỏa mãn ràng buộc: $1 \leq T \leq 10$; $2 \leq N \leq 10$; $0 \leq A[i][j] \leq 1$.

Output:

- Đưa ra các chuỗi ký tự được sắp xếp, trong đó mỗi chuỗi là một đường đi của con chuột trong mê cung. Đưa ra -1 nếu chuột không đi được đến đích.

Input Examples	Output Examples
2	DRDDRR
4	DDRRDR DRDDRR
1 0 0 0 1 1 0 1 0 1 0 0 0 1 1 1	
4	
1 0 0 0 1 1 0 1 1 1 0 0 0 1 1 1	

8. **Back-Tracking 6.** Cho dãy số $A[] = (a_1, a_2, \dots, a_n)$ và tự nhiên K . Hãy đưa ra tất cả các dãy con của dãy số $A[]$ sao cho tổng các phần tử của dãy con đó đúng bằng K . Các phần tử của dãy số $A[]$ được giả thuyết là nguyên dương và không có các phần tử giống nhau. Ví dụ với dãy con $A[] = \{5, 10, 15, 20, 25\}$, $K = 50$ ta có 3 dãy con $\{5, 10, 15, 20\}$, $\{5, 20, 25\}$, $\{10, 15, 25\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N là số lượng phần tử của dãy số $A[]$ và số K ; dòng tiếp theo đưa vào N phần tử của dãy số $A[]$.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10$; $1 \leq K, A[i] \leq 100$.

Output:

- Đưa ra tất cả các dãy con của dãy số $A[]$ thỏa mãn yêu cầu bài toán, trong đó mỗi dãy con được bao bởi các ký tự $[,]$. Nếu không có dãy con nào thỏa mãn yêu cầu bài toán, hãy đưa ra -1.

Input Examples	Output Examples
2 5 50 5 10 15 20 25 8 53 15 22 14 26 32 9 16 8	[5 10 15 20] [5 20 25] [10 15 25] [8 9 14 22] [8 14 15 16] [15 16 22]

9. **Back-Tracking 7.** Cho tập các số $A[] = (a_1, a_2, \dots, a_n)$. Hãy kiểm tra xem ta có thể chia tập $A[]$ thành hai tập con sao cho tổng các phần tử của hai tập con bằng nhau hay không. Đưa ra YES nếu có thể thực hiện được, ngược lại đưa ra NO.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N là số lượng phần tử của dãy số $A[]$; dòng tiếp theo đưa vào N phần tử của dãy số $A[]$.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 100$; $1 \leq A[i] \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2 4 1 5 11 5 3 1 3 5	YES NO

3.4. Một số bài toán xử lý bằng thuật toán đệ quy và quay lui nâng cao

Dưới đây là một số bài toán xử lý bằng thuật toán quay lui nâng cao :

10. Cho số nguyên dương N . Nhiệm vụ của bạn là liệt kê tất cả dãy số $2n$ sao cho các số từ 1 đến n xuất hiện 2 lần và khoảng cách giữa các số đúng bằng số đó. Ví dụ với $n = 3$ ta có hai dãy số: $[3 \ 1 \ 2 \ 1 \ 3 \ 2]$ và $2=[2 \ 3 \ 1 \ 2 \ 1 \ 3]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là số nguyên dương n được viết trên 1 dòng.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 10; 1 \leq n \leq 10$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	3 1 2 1 3 2 2 3 1 2 1 3
3	4 1 3 1 2 4 3 2 2 3 4 2 1 3 1 4
4	

11. Cho số tự nhiên K và xâu ký tự các chữ số S . Nhiệm vụ của bạn là đưa ra số lớn nhất bằng cách thực hiện nhiều nhất K lần đổi chỗ các ký tự trong S . Ví dụ $K=3$ và $S="1234567"$ ta được $"7654321"$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là số K ; dòng tiếp theo là xâu ký tự S .
- T, K, S thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq K \leq 10; 1 \leq \text{length}(S) \leq 30$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
3	7654321
4	5543333
1234567	4301
3	
3435335	
2	
1034	

12. Cho mảng các số nguyên $A[]$ gồm N phần tử. Hãy chia mảng số nguyên $A[]$ thành K tập con khác rỗng sao cho tổng các phần tử của mỗi tập con đều bằng nhau. Mỗi phần tử thuộc tập con xuất hiện duy nhất một lần trong tất cả các tập con. Ví dụ với $A[] = \{2, 1, 4, 5, 6\}$, $K=3$ ta có kết quả $\{2, 4\}, \{1, 5\}, \{6\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là hai số N và K ; dòng tiếp theo đưa vào N số của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.

- $T, N, K, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq N, K, A[i] \leq 100$.

Output:

- Đưa ra 1 nếu có thể chia tập con thành K tập thỏa mãn yêu cầu bài toán, ngược lại đưa ra 0.

Input Examples	Output Examples
2	1
5 3	0
2 1 4 5 6	
5 3	
2 1 5 5 6	

13. Cho mảng $A[]$ gồm N số nguyên dương và số X . Nhiệm vụ của bạn là tìm phép tổ hợp các số trong mảng $A[]$ có tổng bằng X . Các số trong mảng $A[]$ có thể được sử dụng nhiều lần. Mỗi tổ hợp các số của mảng $A[]$ được in ra theo thứ tự không giảm các số. Ví dụ với $A[] = \{2, 4, 6, 8\}$, $X = 8$ ta có các tổ hợp các số như sau:

$[2, 2, 2, 2], [2, 2, 4], [2, 6], [4, 4], [8]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là hai số N và X ; dòng tiếp theo đưa vào N số của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, X, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 10; 1 \leq X, N \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng. Mỗi đường tổ hợp được bao bởi cặp ký tự $[,]$. Đưa ra -1 nếu không có tổ hợp nào thỏa mãn yêu cầu bài toán.

Input Examples	Output Examples
1	$[2\ 2\ 2\ 2]\ [2\ 2\ 4]\ [2\ 6]\ [4\ 4]\ [8]$
4 8	
2 4 6 8	

14. **Back-Tracking 13.** Cho ma trận $A[M][N]$. Nhiệm vụ của bạn là đưa ra tất cả các đường đi từ phần tử $A[0][0]$ đến phần tử $A[M-1][N-1]$. Bạn chỉ được phép dịch chuyển xuống dưới hoặc sang phải phần tử liền kề với vị trí hiện tại.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là hai số M, N tương ứng với số hàng và số cột của ma trận; dòng tiếp theo đưa vào các phần tử của ma trận $A[i][j]$; các số được viết cách nhau một vài khoảng trống.
- $T, M, N, A[i][j]$ thỏa mãn ràng buộc: $1 \leq T \leq 10; 1 \leq M, N, A[i][j] \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng. Mỗi đường đi được bao bởi cặp ký tự $[,]$.

Input Examples	Output Examples
2	[1 4 5 6] [1 2 5 6] [1 2 3 6]
2 3	[1 2 4] [1 3 4]
1 2 3 4 5 6	
2 2	
1 2 3 4	

15. **Back-Tracking 13.** Cho ba số S, P, N . Trong đó, P là một số nguyên tố. Nhiệm vụ của bạn là đưa ra tất cả N số nguyên tố sau P có tổng bằng S . Ví dụ với $S = 28, P=7, N=2$ ta có kết quả $11 + 17 = 28$. Với $N = 3, P = 2, S = 23$ ta có kết quả : $\{3, 7, 13\}, \{5, 7, 11\}$

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ ba số S, P, N được viết trên một dòng.
- S, P, N thỏa mãn ràng buộc: $1 \leq T \leq 100; 2 \leq S, N, P \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

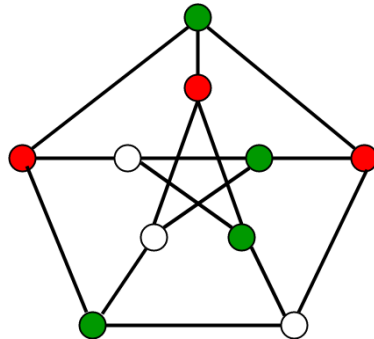
Input Examples	Output Examples
2	11 17
28 7 2	3 7 13 5 7 11
3 2 23	

3.5. Một số bài toán xử lý bằng thuật toán đệ qui và quay lui tiên tiến

Dưới đây là một số bài toán xử lý bằng thuật toán quay lui tiên tiến:

16. **Back-Tracking 8.** Một trong những bài toán kinh điển của lý thuyết đồ thị là bài toán Tô màu đồ thị. Bài toán được phát biểu như sau: Cho đồ thị vô hướng $G = \langle V, E \rangle$,

E> được biểu diễn dưới dạng danh sách cạnh và số M. Nhiệm vụ của bạn là kiểm tra xem đồ thị có thể tô màu các đỉnh bằng nhiều nhất M màu sao cho hai đỉnh kề nhau đều có màu khác nhau hay không?



Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào ba số V, E, M tương ứng với số đỉnh, số cạnh và số màu; phần thứ hai đưa vào các cạnh của đồ thị.
- T, V, E, M thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq V \leq 50$; $1 \leq E \leq N(N-1)$, $1 \leq V \leq N$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	YES
4 5 3	NO
1 2 2 3 3 4 4 1 1 3	
3 3 2	
1 2 2 3 1 3	

17. **Back-Tracking 9.** Đường đi đơn trên đồ thị có hướng hoặc vô hướng đi qua tất cả các đỉnh của đồ thị mỗi đỉnh đúng một lần được gọi là đường đi Hamilton. Cho đồ thị vô hướng $G = \langle V, E \rangle$, hãy kiểm tra xem đồ thị có đường đi Hamilton hay không?

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số V, E tương ứng với số đỉnh, số cạnh của đồ thị; phần thứ hai đưa vào các cạnh của đồ thị.
- T, V, E thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq V \leq 10$; $1 \leq E \leq 15$.

Output:

- Đưa ra 1 hoặc 0 tương ứng với test có hoặc không có đường đi Hamilton theo từng dòng.

Input Examples	Output Examples
2	1
4 4	0
1 2 2 3 3 4 2 4	
4 3	
1 2 2 3 2 4	

18. **Back-Tracking 10.** Đường đi đơn trên đồ thị có hướng hoặc vô hướng đi qua tất cả các đỉnh của đồ thị mỗi đỉnh đúng một lần được gọi là đường đi Hamilton. Cho đồ thị vô hướng $G = \langle V, E \rangle$, hãy kiểm tra xem đồ thị có đường đi Hamilton hay không?

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số V, E tương ứng với số đỉnh, số cạnh của đồ thị; phần thứ hai đưa vào các cạnh của đồ thị.
- T, V, E thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq V \leq 10; 1 \leq E \leq 15$.

Output:

- Đưa ra 1 hoặc 0 tương ứng với test có hoặc không có đường đi Hamilton theo từng dòng.

Input Examples	Output Examples
2	1
4 4	0
1 2 2 3 3 4 2 4	
4 3	
1 2 2 3 2 4	

19. **Back-Tracking 11.** Cho tập từ ghi trong từ điển $dic[]$ và một bảng hai chiều $A[M][N]$ các ký tự. Hãy tạo nên tất cả các từ có mặt trong từ điển $dic[]$ bằng cách nối các ký tự kề nhau trong mảng $A[][]$. Chú ý, phép nối các ký tự kề nhau trong mảng $A[][]$ được thực hiện theo 8 hướng nhưng không có phần tử $A[i][j]$ nào được lặp lại. Ví dụ với từ điển $dic[] = \{ \text{"GEEKS", "FOR", "QUIZ", "GO"} \}$ và mảng $A[][]$ dưới đây sẽ cho ta kết quả: "GEEKS", "QUIZ"

G	I	Z
U	E	K
Q	S	E

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào ba số K, M, N tương ứng với số từ của từ điển dic[], số hàng và số cột của ma trận ký tự A[M][N]; dòng tiếp theo đưa vào K từ của từ điển dic[]; dòng cuối cùng đưa vào các phần tử A[i][j].
- T, K, M, N thỏa mãn ràng buộc: $1 \leq T \leq 10$; $1 \leq K \leq 10$; $1 \leq M, N \leq 7$.

Output:

- Đưa ra theo thứ tự tăng dần các từ có mặt trong từ điển dic[] được tạo ra từ ma trận A[][]. Đưa ra -1 nếu không thể tạo ra từ nào thuộc dic[] từ A[][].

Input Examples	Output Examples
1 4 3 3 GEEKS FOR QUIZ GO G I Z U E K Q S E	GEEKS QUIZ

20. Back-Tracking 12. Cho biểu thức P chỉ chứa các ký tự '(', ')' và các ký tự. Không có phép toán nào trong biểu thức P. Nhiệm vụ của bạn là thực hiện ít nhất các phép loại bỏ các ký tự '(', ')' để P trở thành biểu thức đúng. Nếu có nhiều hơn một biểu thức đúng với cùng số phép loại bỏ ít nhất hãy đưa ra tất cả các biểu thức đúng.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một biểu thức P được viết trên một dòng.
- T, P thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(P) \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2 (())() (u)())()	000 (())() (u)()() u(())()

21. Sudoku là bài toán điền hình cho kỹ thuật quay lui được phát biểu như sau: Cho một lưới hình vuông gồm 9×9 được tạo bởi 9 hình vuông kích cỡ 3×3 hình vuông đơn vị. Trên các hàng, cột của lưới đã điền sẵn một số ô có giá trị từ 1 đến 9 sao cho không có hàng nào có hai giá trị trùng nhau, không có cột nào có hai trị trùng nhau, không có hình vuông kích cỡ 3×3 nào có giá trị các phần tử trùng nhau. Nhiệm vụ của bạn là hãy đưa ra một cách điền vào các ô còn trống của lưới các số từ 1 đến 9

sao cho: đọc theo hàng, cột và 9 hình vuông kích cỡ 3×3 ta đều nhận được các số từ 1 đến 9.

3		6	5		8	4		
5	2							
	8	7					3	1
		3		1			8	
9			8	6	3			5
	5			9		6		
1	3					2	5	
							7	4
		5	2		6	3		

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một ma trận kích cỡ 9×9 .
- T, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 10$; $1 \leq A[i] \leq 9$.

Output:

- Đưa ra kết quả mỗi test theo là một ma trận Sudocu theo từng dòng.

Input Examples	Output Examples
1	3 1 6 5 7 8 4 9 2
3 0 6 5 0 8 4 0 0	5 2 9 1 3 4 7 6 8
5 2 0 0 0 0 0 0 0	4 8 7 6 2 9 5 3 1
0 8 7 0 0 0 0 3 1	2 6 3 4 1 5 9 8 7
0 0 3 0 1 0 0 8 0	9 7 4 8 6 3 1 2 5
9 0 0 8 6 3 0 0 5	8 5 1 7 9 2 6 4 3
0 5 0 0 9 0 6 0 0	1 3 8 9 4 7 2 5 6
1 3 0 0 0 0 2 5 0	6 9 2 3 5 1 8 7 4
0 0 0 0 0 0 0 7 4	7 4 5 2 8 6 3 1 9
0 0 5 2 0 6 3 0 0	

CHƯƠNG 4. MÔ HÌNH THUẬT TOÁN THAM LAM

Đối với nhiều bài toán tối ưu, khi ta không tìm ra được một giải pháp tối ưu toàn cục nhưng lại rất dễ dàng phát hiện ra giải pháp tối ưu cục bộ đều có thể giải quyết bằng mô hình thuật toán tham lam. Khác với mô hình thuật toán quy hoạch động, tại mỗi bước của thuật toán, thuật toán đưa ra quyết định dựa trên các quyết định của bước trước, và có thể xét lại đường đi của bước trước hướng tới lời giải. Giải thuật tham lam quyết định sớm và thay đổi đường đi thuật toán theo quyết định đó, và không bao giờ xét lại các quyết định cũ. Nội dung chính của chương này trình bày các kỹ thuật tham lam cơ bản và nâng cao.

4.1. Giới thiệu mô hình thuật toán tham lam (Greedy Algorithm)

Thuật toán tham lam (Greedy Algorithm) xây dựng một chiến lược “*tham từng miếng*”, miếng để tham nhất thường là một giải pháp tối ưu cục bộ nhưng lại luôn có mặt trong cấu trúc tối ưu toàn cục. Tiếp tục phát triển chiến lược “*tham từng miếng*” cho các bước tiếp theo để đạt được kết quả tối ưu toàn cục. Tóm lại, thuật toán tham lam dùng để giải lớp các bài toán tối ưu thỏa mãn hai điều kiện:

- Ở mỗi bước ta luôn tạo ra một lựa chọn tốt nhất tại thời điểm đó.
- Việc liên kết lại kết quả mỗi bước sẽ cho ta kết quả tối ưu toàn cục.

Tổng quát, thuật toán Greedy bao gồm 5 thành phần chính:

- 1) Một tập các ứng viên (*candidatemembers*) mà giải pháp có thể tham lam.
- 2) Một hàm lựa chọn (*selection fuction*) để chọn ứng viên tốt nhất cho giải pháp tham lam cục bộ.
- 3) Một hàm thực thi (*feasibility function*) được sử dụng để quyết định xem một ứng viên có được dùng để xây dựng lời giải hay không?
- 4) Một hàm mục tiêu (*objective function*) dùng để xác định giá trị của lời giải hoặc một phần của lời giải.
- 5) Một hàm giải pháp (*solution function*) dùng để xác định khi nào giải pháp hoàn chỉnh.

Khi sử dụng giải pháp tham lam, hai thành phần quyết định nhất tới quyết định tham lam đó là:

Lựa chọn tính chất để tham. Khi chưa tìm được lời giải tối ưu toàn cục nhưng ta biết chắc chắn một giải pháp tối ưu cục bộ dựa vào tính chất cụ thể của mỗi bài toán. Bài toán con tiếp theo cũng thực hiện với tính chất như vậy cho đến khi đạt được giải pháp tối ưu toàn cục.

Lựa chọn cấu trúc con tối ưu. Một bài toán được gọi là "có cấu trúc tối ưu" nếu một lời giải tối ưu của bài toán con nằm trong lời giải tối ưu của bài toán lớn hơn. Điều này cho phép ta xác định từng cấu trúc con tối ưu cho mỗi bài toán con, sau đó phát triển các bài toán con cho đến bài toán con cuối cùng.

Ví dụ. Bài toán lựa chọn hành động (*Activity Selection Problem*). Lựa chọn hành động là bài toán tối ưu tổ hợp điển hình quan tâm đến việc lựa chọn các hành động không mâu thuẫn nhau trong các khung thời gian cho trước. Bài toán được phát biểu như sau:

Cho tập gồm n hành động, mỗi hành động được biểu diễn như bộ đôi thời gian bắt đầu s_i và thời gian kết thúc f_i ($i=1, 2, \dots, n$). Bài toán đặt ra là hãy lựa chọn nhiều nhất các hành động có thể thực hiện bởi một máy hoặc một người mà không xảy ra mâu thuẫn. Giả sử mỗi hành động chỉ thực hiện đơn lẻ tại một thời điểm.

Input:

- Số lượng hành động: 6
- Thời gian bắt đầu $Start[] = \{1, 3, 0, 5, 8, 5\}$
- Thời gian kết thúc $Finish[] = \{2, 4, 6, 7, 9, 9\}$

Output: Số lượng lớn nhất các hành động có thể thực hiện bởi một người.

$OPT[] = \{1, 2, 4, 5\}$

Lời giải. Rõ ràng, hệ sẽ xảy ra mâu thuẫn khi tồn tại hai hành động kế tiếp nhau i và $i+1$ có thời gian kết thúc của hành động i lớn hơn thời gian bắt đầu của hành động $i+1$. Nói cách khác hệ mâu thuẫn khi $Finish[i] > Start[i+1]$ ($1 \leq i < n$). Vấn đề còn lại là làm thế nào ta có thể lựa chọn được tập lớn nhất các hành động không tồn tại mâu thuẫn. Để giải quyết điều này, ta có thể sử dụng thuật toán sinh hoặc thuật toán quay lui để duyệt trên tập tất cả các tập con các hành động, sau đó lựa chọn tập con có số lượng hành động lớn nhất và không xảy ra mâu thuẫn. Tuy nhiên lời giải này cho ta độ phức tạp hàm mũ (2^n) nên lời giải này thực tế không giải được bằng máy tính.

Để khắc phục điều này ta có thể xây dựng thuật toán tham lam một cách hiệu quả dựa vào những nhận xét trực quan như sau:

- Trường hợp xấu nhất xảy ra khi hệ chỉ có thể đáp ứng được nhiều nhất là một hành động, hễ cứ thêm một hành động hệ sẽ xảy ra mâu thuẫn. Rõ ràng, lựa chọn tối ưu nhất trong trường hợp này là chọn hành động kết thúc với thời gian sớm nhất.
- Nếu hệ có thể đáp ứng nhiều hơn một hành động, thì lựa chọn tốt tiếp theo chính là kết nạp hành động có thời gian bắt đầu lớn hơn thời gian kết thúc của hành động trước đó và có thời gian kết thúc nhỏ nhất trong số các hành động còn lại. Cứ tiếp tục làm như vậy ta sẽ có được giải pháp tối ưu toàn cục.

Thuật toán tham lam dùng để giải bài toán Activity Selection được thể hiện như sau:

Algorithm Greedy-Activities- Selection(N, S[], F[]):

Input:

- N là số lượng hành động (công việc).
- S[] thời gian bắt đầu mỗi hành động.
- F[] thời gian kết thúc mỗi hành động.

Output:

- Danh sách thực thi nhiều nhất các hành động.

Actions:

Bước 1 (sắp xếp). Sắp xếp các hành động theo thứ tự tăng dần của thời gian kết thúc.

Bước 2 (Khởi tạo) Lựa chọn hành động đầu tiên làm phương án tối ưu (OPT=1). $N = N \setminus \{i\}$;

Bước 3 (Lặp).

```

for each activities  $j \in N$  do {
    if ( S[j] >= F[i] ) {
        OPT = OPT  $\cup$  j;  $i = j$ ;  $N = N \setminus \{i\}$ 
    }
}

```

Bước 4 (Trả lại kết quả).
Return (OPT)

EndActions.

Hình 4.1. Thuật toán tham lam giải bài toán Activity Selection

Kiểm nghiệm thuật toán:

Ví dụ. Thuật toán Greedy-ActivitiesN-Selection(int N, int S[], int F[]):

Input:

- Số lượng hành động $n = 8$.
- Thời gian bắt đầu $S[] = \{1, 3, 0, 5, 8, 5, 9, 14\}$.
- Thời gian kết thúc $F[] = \{2, 4, 6, 7, 9, 9, 12, 18\}$.

Output:

- OPT = { Tập nhiều nhất các hành động có thể thực hiện bởi một máy}.

Phương pháp tiến hành::

Bước	$i = ? j = ?$	$(S[j] \geq F[i]) ? i = ?$	OPT = ?
			OPT = OPT $\cup$ {1}.
1	$i=1; j=2$	$(3 \geq 2)$: Yes; $i=2$.	OPT = OPT $\cup$ {2}.
2	$i=2; j=3$;	$(0 \geq 4)$: No; $i=2$.	OPT = OPT $\cup$ ϕ .
3	$i=2; j=4$;	$(5 \geq 4)$: Yes; $i=4$.	OPT = OPT $\cup$ {4}.
4	$i=4; j=5$;	$(8 \geq 7)$: Yes; $i=5$.	OPT = OPT $\cup$ {5}.
5	$i=5; j=6$;	$(5 \geq 9)$: No; $i=5$.	OPT = OPT $\cup$ ϕ .
6	$i=5; j=7$;	$(9 \geq 9)$: Yes; $i=7$.	OPT = OPT $\cup$ {7}.
7	$i=7; j=8$;	$(14 \geq 12)$: Yes; $i=8$.	OPT = OPT $\cup$ {8}.
OPT = { 1, 2, 4, 5, 7, 8}			

Cài đặt thuật toán: thuật toán được cài đặt với khuôn dạng dữ liệu vào trong file data.in và kết quả ra trong file ketqua.out như sau:

File data.in:

- Dòng đầu tiên ghi lại số tự nhiên N là số lượng hành động.
- N dòng kế tiếp, mỗi dòng ghi lại bộ đôi Start[i], Finish[i] tương ứng với thời gian bắt đầu và thời gian kết thúc mỗi hành động. Hai số được viết cách nhau một vài khoảng trống.

File ketqua.out:

- Dòng đầu ghi lại số lượng lớn nhất các hành động.
- Dòng kế tiếp ghi lại các hành động được lựa chọn. Các hành động được viết cách nhau một vài khoảng trống.

Ví dụ về file data.in và ketqua.out:

data.in		ketqua.out			
6		4			
1	2	1	2	4	5
3	4				
0	6				
5	7				
8	9				
5	9				

Chương trình giải bài toán Activity Selection như dưới đây:

```
#include <iostream>
#include <fstream>
#include <iomanip>
#define MAX 1000
using namespace std;
int Start[MAX], Finish[MAX], n, XOPT[MAX], dem=0;
//đọc dữ liệu từ file data.in
void Read_Data(){
    ifstream fp("data.in"); fp>>n;
    for (int i=1; i<=n; i++){
        fp>>Start[i];
        fp>>Finish[i];
        XOPT[i]=false;
    }
    fp.close();
}
```

```

//Sắp xếp tăng dần theo thời gian kết thúc các hành động
void Sapxep(void) {
    for(int i=1; i<=n-1; i++){
        for(int j=i+1; j<=n; j++){
            if(Finish[i]> Finish[j]){
                int t = Finish[i]; Finish[i]=Finish[j];Finish[i]=t;
                t = Start[i];Start[i]=Start[j];Start[i]=t;
            }
        }
    }
}

//đưa ra kết quả tối ưu
void Result(void){
    ofstream fp("ketqua.out");
    fp<<dem<<endl;
    for(int i=1; i<=n; i++){
        if(XOPT[i])
            fp<<i<<setw(3);
    }
}

void Greedy_Solution(void){ //Thuật toán tham lam
    Read_Data();//đọc dữ liệu từ file data.in
    Sapxep();//Bước 1: Sắp xếp
    int i =1; XOPT[i]=true; dem=1; //Bước 2. Khởi tạo
    for(int j=2; j<=n; j++){//Bước 3: lặp
        if(Finish[i]<=Start[j]){
            dem++; i = j; XOPT[i]=true;
        }
    }
    Result();//Bước 3. Trả lại kết quả
}

int main(void){
    Greedy_Solution();
}

```

4.2. Một số bài toán xử lý bằng thuật toán tham lam cơ bản

Dưới đây là một số bài toán xử lý bằng thuật toán tham lam cơ bản :

1. Cho mảng $A[]$ gồm N phần tử. Nhiệm vụ của bạn là tìm $max = \sum_{i=0}^{n-1} A_i * i$ bằng cách sắp đặt lại các phần tử trong mảng. Chú ý, kết quả của bài toán có thể rất lớn vì vậy bạn hãy đưa ra kết quả lấy modulo với 10^9+7 .

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số phần tử của mảng N ; dòng tiếp theo đưa vào N số $A[i]$ tương ứng với các phần tử của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	40
5	8
5 3 2 4 1	
3	
1 2 3	

2. Cho mảng $A[]$ gồm các số từ 0 đến 9. Nhiệm vụ của bạn là tìm tổng nhỏ nhất của hai số được tạo bởi các số trong mảng $A[]$. Chú ý, tất cả các số trong mảng $A[]$ đều được sử dụng để tạo nên hai số.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số phần tử của mảng N ; dòng tiếp theo đưa vào N số $A[i]$ tương ứng với các phần tử của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 50$; $0 \leq A[i] \leq 9$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	604
6	82
6 8 4 5 2 3	
5	
5 3 0 7 4	

3. Cho mảng $A[]$ gồm N số nguyên không âm và số K . Nhiệm vụ của bạn là hãy chia mảng $A[]$ thành hai mảng con có kích cỡ K và $N-K$ sao cho hiệu giữa tổng hai mảng con là lớn nhất. Ví dụ với mảng $A[] = \{8, 4, 5, 2, 10\}$, $K=2$ ta có kết quả là 17 vì mảng $A[]$ được chia thành hai mảng $\{4, 2\}$ và $\{8, 5, 10\}$ có hiệu của hai mảng con là $23-6=17$ là lớn nhất.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số phần tử của mảng N và số K ; dòng tiếp theo đưa vào N số $A[i]$ tương ứng với các phần tử của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, K, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq K < N \leq 50$; $0 \leq A[i] \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	17
5 2	2
8 4 5 2 10	
8 3	
1 1 1 1 1 1 1 1	

4. Cho mảng $A[]$ gồm N số và thực hiện các thao tác theo nguyên tắc dưới đây:
- Ta chọn một mảng con sao cho phần tử ở giữa của mảng con cũng là phần tử ở giữa của mảng $A[]$ (trong trường hợp N lẻ).
 - Đảo ngược mảng con đã chọn trong mảng $A[]$. Ta được phép chọn mảng con và phép đảo ngược mảng con bao nhiêu lần tùy ý.

Ví dụ với mảng $A[] = \{1, 6, 3, 4, 5, 2, 7\}$ ta có câu trả lời là Yes vì: ta chọn mảng con $\{3, 4, 5\}$ và đảo ngược để nhận được mảng $A[] = \{1, 6, 5, 4, 3, 2, 7\}$, chọn tiếp mảng con $\{6, 5, 4, 3, 2\}$ và đảo ngược ta nhận được mảng $A[] = \{1, 2, 3, 4, 5, 6, 7\}$. Hãy cho biết ta có thể sắp xếp được mảng $A[]$ bằng cách thực hiện các thao tác kể trên hay không?

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số phần tử của mảng N ; dòng tiếp theo đưa vào N số $A[i]$ tương

ứng với các phần tử của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.

- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 50$; $0 \leq A[i] \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	Yes
6	No
1 6 3 4 5 2 7	
6	
1 6 3 4 5 7 2	

5. Cho mảng $A[], B[]$ đều có N phần tử. Nhiệm vụ của bạn là tìm giá trị nhỏ nhất của biểu thức $P = A[0]*B[0] + A[1]*B[1] + \dots + A[N-1]*B[N-1]$ bằng cách trao đổi vị trí các phần tử của cả mảng $A[]$ và $B[]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng thứ nhất đưa vào số phần tử của mảng N ; dòng tiếp theo đưa vào N số $A[i]$; dòng cuối cùng đưa vào N số $B[i]$ các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^7$; $0 \leq A[i], B[i] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	Yes
6	No
1 6 3 4 5 2 7	
6	
1 6 3 4 5 7 2	

6. Cho mảng $A[], B[]$ đều có N phần tử. Nhiệm vụ của bạn là tìm giá trị nhỏ nhất của biểu thức $P = A[0]*B[0] + A[1]*B[1] + \dots + A[N-1]*B[N-1]$ bằng cách trao đổi vị trí các phần tử của cả mảng $A[]$ và $B[]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng thứ nhất đưa vào số phần tử của mảng N ; dòng tiếp theo đưa vào N số $A[i]$; dòng cuối cùng đưa vào N số $B[i]$ các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^7$; $0 \leq A[i], B[i] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	Yes
6	No
1 6 3 4 5 2 7	
6	
1 6 3 4 5 7 2	

4.3. Một số bài toán xử lý bằng thuật toán tham lam nâng cao

Dưới đây là một số bài toán xử lý bằng thuật toán tham lam nâng cao :

- Cho hệ gồm N hành động. Mỗi hành động được biểu diễn như một bộ đôi $\langle S_i, F_i \rangle$ tương ứng với thời gian bắt đầu và thời gian kết thúc của mỗi hành động. Hãy tìm phương án thực hiện nhiều nhất các hành động được thực hiện bởi một máy hoặc một người sao cho hệ không xảy ra mâu thuẫn.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng thứ nhất đưa vào số lượng hành động N ; dòng tiếp theo đưa vào N số S_i tương ứng với thời gian bắt đầu mỗi hành động; dòng cuối cùng đưa vào N số F_i tương ứng với thời gian kết thúc mỗi hành động; các số được viết cách nhau một vài khoảng trống.
- T, N, S_i, F_i thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, F_i, S_i \leq 1000$.

Output:

- Đưa dãy lớn nhất các hành động có thể được thực thi bởi một máy hoặc một người.

Input Examples	Output Examples
1	1 2 4 5
6	
1 3 0 5 8 5	

2 4 6 7 9 9	
-------------	--

8. Một phân số đơn vị nếu tử số của phân số đó là 1. Mọi phân số nguyên dương đều có thể biểu diễn thành tổng các phân số đơn vị. Ví dụ $2/3 = 1/2 + 1/6$. Cho phân số nguyên dương P/Q bất kỳ, hãy biểu diễn phân số nguyên dương thành tổng phân số đơn vị.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ đôi tử số P và mẫu số Q của phân số nguyên dương được viết trên một dòng.
- T, N, S_i, F_i thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, F_i, S_i \leq 1000$.

Output:

- Đưa ra dãy lớn nhất các hành động có thể được thực thi bởi một máy hoặc một người.

Input Examples	Output Examples
2	$1/2 + 1/6$
2 3	$1/3$
1 3	

9. Cho mảng $A[]$ ghi lại N việc. Mỗi việc trong $A[]$ được biểu diễn như một bộ 3 số nguyên dương $\langle \text{JobId}, \text{Deadline}, \text{Profit} \rangle$, trong đó JobId là mã của việc, Deadline là thời gian kết thúc của việc, Profit là lợi nhuận đem lại nếu hoàn thành việc đó đúng thời gian. Hãy cho biết lợi nhuận lớn nhất có thể thực hiện các việc với giả thiết mỗi việc được thực hiện đơn lẻ.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là số lượng $\text{Job } N$; phần thứ hai đưa vào $3 \times N$ số tương ứng với N job.
- $T, N, \text{JobId}, \text{Deadline}, \text{Profit}$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 100$; $1 \leq \text{JobId} \leq 100$; $1 \leq \text{Deadline} \leq 100$; $1 \leq \text{Profit} \leq 500$.

Output:

- Đưa số lượng Job và lợi nhuận lớn nhất có thể đạt được.

Input Examples	Output Examples
2	2 60
2	2 127
4	
1 4 20 2 1 10 3 1 40 4 1 30	

5	
1 2 100 2 1 19 3 2 27 4 1 25 5 1 15	

10. Cho một mảng S gồm $2 \times N$ ký tự, trong đó có N ký tự '[' và N ký tự ']'. Xâu S được gọi là viết đúng nếu S có dạng $S_2[S_1]$ trong đó S, S_2 là các xâu viết đúng. Nhiệm vụ của bạn là tìm số các phép đổi chỗ ít nhất các ký tự kề nhau của xâu S viết sai để S trở thành viết đúng. Ví dụ với xâu $S = "[]][[]"$ ta có số phép đổi chỗ kề nhau ít nhất là 2.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu S viết sai theo nguyên tắc kể trên.
- T, S thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(S) \leq 100000$.

Output:

- Đưa số lượng Job và lợi nhuận lớn nhất có thể đạt được.

Input Examples	Output Examples
2 []][[] []][[]	2 0

11. Cho N sợi dây với độ dài khác nhau được lưu trong mảng $A[]$. Nhiệm vụ của bạn là nối N sợi dây thành một sợi sao cho tổng chi phí nối dây là nhỏ nhất. Biết chi phí nối sợi dây thứ i và sợi dây thứ j là tổng độ dài hai sợi dây $A[i]$ và $A[j]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào số lượng sợi dây N ; dòng tiếp theo đưa vào N số $A[i]$ là độ dài của các sợi dây; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$; $0 \leq A[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2 4 4 3 2 6 5 4 2 7 6 9	29 62

12. Một số X được gọi là số khối lập phương nếu X là lũy thừa bậc 3 của số Y ($X = Y^3$). Cho số nguyên dương N , nhiệm vụ của bạn là tìm số khối lập phương lớn nhất bằng cách loại bỏ đi các chữ số của N . Ví dụ số 4125 ta có kết quả là $125 = 5^3$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên N được viết trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	125
4125	8
976	

13. Giả sử bạn là một người nghèo trong địa phương của bạn. Địa phương của bạn có duy nhất một cửa hàng bán lương thực. Cửa hàng của bạn mở cửa tất cả các ngày trong tuần ngoại trừ chủ nhật. Cho bộ ba số N, S, M thỏa mãn ràng buộc sau:

- N : số đơn vị lương thực nhiều nhất bạn có thể mua trong ngày.
- S : số lượng ngày bạn cần được sử dụng lương thực để tồn tại.
- M : số đơn vị lương thực cần có mỗi ngày để bạn tồn tại.

Giả sử bạn đang ở ngày thứ 2 trong tuần và cần tồn tại trong S ngày tới. Hãy cho biết số lượng ngày ít nhất bạn cần phải mua lương thực từ cửa hàng để tồn tại hoặc bạn sẽ bị chết đói trong S ngày tới.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ 3 số N, S, M được viết trên một dòng.
- T, N, S, M thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, S, M \leq 30$.

Output:

- Đưa ra số ngày ít nhất bạn có thể mua lương thực để tồn tại hoặc đưa ra -1 nếu bạn bị chết đói.

Input Examples	Output Examples
2	2
16 10 2	-1
20 10 30	

14. Cho hai số nguyên dương S và D, trong đó S là tổng các chữ số và D là số các chữ số của một số. Nhiệm vụ của bạn là tìm số nhỏ nhất thỏa mãn S và D? Ví dụ với S = 9, D = 2 ta có số nhỏ nhất thỏa mãn S và D là 18.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ 2 số S và D được viết trên một dòng.
- T, S, D thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq S, D \leq 10000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	18
9 2	299
20 3	

4.4. Một số bài toán xử lý bằng thuật toán tham lam tiên tiến

Dưới đây là một số bài toán xử lý bằng thuật toán tham lam tiên tiến :

15. Cho chuỗi ký tự S. Ta gọi giá trị của chuỗi S là tổng bình phương số lần xuất hiện mỗi ký tự trong S. Hãy tìm giá trị nhỏ nhất của chuỗi S sau khi thực hiện K lần loại bỏ ký tự.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là số K; phần thứ hai là một chuỗi ký tự S được viết trên một dòng.
- T, S, K thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(S) \leq 10000$; $1 \leq K \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	3
2	2
ABCABC	
3	
ABCABC	

16. Cho chuỗi ký tự S bao gồm các ký tự in thường. Nhiệm vụ của bạn là kiểm tra xem ta có thể sắp đặt lại các ký tự trong S để hai ký tự giống nhau đều không kề nhau hay

không? Đưa ra 1 nếu có thể sắp đặt lại các ký tự trong S thỏa mãn yêu cầu bài toán, ngược lại đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu ký tự S được viết trên một dòng.
- T, S thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{lengght}(S) \leq 10000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
3	1
geeksforgeeks	1
bbbabaaacd	-1
bbbbbb	

17. Cho xâu ký tự S bao gồm các ký tự in thường và số D. Nhiệm vụ của bạn là kiểm tra xem ta có thể sắp đặt lại các ký tự trong S để tất cả các ký tự giống nhau đều có khoảng cách là D hay không? Đưa ra 1 nếu có thể sắp đặt lại các ký tự trong S thỏa mãn yêu cầu bài toán, ngược lại đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là số D; dòng tiếp theo là xâu S.
- T, S, D thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{lengght}(S) \leq 10000$; $1 \leq D \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	1
2	-1
abb	
2	
aaa	

18. Cho đồ thị vô hướng có trọng số $G = \langle V, E, W \rangle$. Nhiệm vụ của bạn là hãy xây dựng một cây khung nhỏ nhất của đồ thị bằng thuật toán Kruskal.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số V, E tương ứng với số đỉnh và số cạnh của đồ thị; phần thứ 2 đưa vào E cạnh của đồ thị, mỗi cạnh là một bộ 3: đỉnh đầu, đỉnh cuối và trọng số của cạnh.
- T, S, D thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq V \leq 100; 1 \leq E, W \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	4
3 3	5
1 2 5 2 3 3 1 3 1	
2 1	
1 2 5	

19. Cho đồ thị vô hướng có trọng số $G = \langle V, E, W \rangle$. Nhiệm vụ của bạn là hãy xây dựng một cây khung nhỏ nhất của đồ thị bằng thuật toán PRIM.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số V, E tương ứng với số đỉnh và số cạnh của đồ thị; phần thứ 2 đưa vào E cạnh của đồ thị, mỗi cạnh là một bộ 3: đỉnh đầu, đỉnh cuối và trọng số của cạnh.
- T, S, D thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq V \leq 100; 1 \leq E, W \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	4
3 3	5
1 2 5 2 3 3 1 3 1	
2 1	
1 2 5	

20. Cho đồ thị vô hướng có trọng số $G = \langle V, E, W \rangle$. Nhiệm vụ của bạn là hãy xây dựng một cây khung nhỏ nhất của đồ thị bằng thuật toán Bruvka.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số V, E tương ứng với số đỉnh và số cạnh của đồ thị; phần thứ 2 đưa vào E cạnh của đồ thị, mỗi cạnh là một bộ 3: đỉnh đầu, đỉnh cuối và trọng số của cạnh.
- T, S, D thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq V \leq 100$; $1 \leq E, W \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	4
3 3	5
1 2 5 2 3 3 1 3 1	
2 1	
1 2 5	

21. Một trong những bài toán kinh điển của lý thuyết tổ hợp là Bài toán cái túi. Bài toán được phát biểu như sau: Một nhà thám hiểm cần đem theo một cái túi trọng lượng không quá W. Có N đồ vật cần đem theo. Đồ vật thứ i có trọng lượng $A[i]$, có giá trị sử dụng $C[i]$. Nhiệm vụ của bạn là hãy tìm cách đưa đồ vật vào túi cho nhà thám hiểm sao cho tổng giá trị sử dụng các đồ vật trong túi là lớn nhất. Giả thiết ta không hạn chế số lượng mỗi đồ vật đưa vào túi (Fraction Knapsack).

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số N, W tương ứng với số lượng đồ vật và trọng lượng túi; phần thứ 2 đưa vào $2 \cdot N$ số tương ứng với trọng lượng đồ vật $A[i]$ và giá trị sử dụng $C[i]$ của mỗi đồ vật.
- T, N, W, $A[i]$, $C[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, W \leq 100$; $1 \leq A[i], C[i] \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	240.00
3 50	160.00
60 10 100 20 120 30	
2 50	
60 10 100 20	

CHƯƠNG 5. MÔ HÌNH THUẬT TOÁN CHIA VÀ TRỊ

Trong khoa học máy tính, chia để trị là một mô hình thiết kế thuật toán quan trọng dựa trên đệ quy với nhiều phân nhánh. Thuật toán chia để trị hoạt động bằng cách chia bài toán thành nhiều bài toán nhỏ hơn thuộc cùng thể loại, cứ như vậy lặp lại nhiều lần, cho đến khi bài toán thu được đủ đơn giản để có thể giải quyết trực tiếp. Sau đó lời giải của các bài toán nhỏ được tổng hợp lại thành lời giải cho bài toán ban đầu. Kỹ thuật này là cơ sở cho nhiều thuật toán hiệu quả, chẳng hạn như thuật toán sắp xếp (sắp xếp nhanh, sắp xếp trộn), thuật toán nhân (thuật toán Karatsuba), thuật toán phân tích cú pháp, thuật toán biến đổi Fourier rời rạc.

5.1. Giới thiệu mô hình thuật toán chia và trị (Divide and Conquer Algorithm)

Thuật toán chia để trị (Divide and Conquer) dùng để giải lớp các bài toán có thể thực hiện được thông qua ba bước:

- **Bước chia (Divide).** Chia bài toán lớn thành những bài toán con có cùng kiểu với bài toán lớn.
- **Bước trị (Conquer).** Giải các bài toán con đã chia ở bước trước đó. Thông thường các bài toán con chỉ khác nhau về dữ liệu vào nên ta có thể thực hiện bằng một thủ tục đệ quy.
- **Bước tổng hợp (Combine).** Tổng hợp lại kết quả của các bài toán con để nhận được kết quả của bài toán lớn.

Ví dụ. Nâng x lên lũy thừa n .

Lời giải.

Bước chia. Ta có $x^n = \begin{cases} x^{n/2} \cdot x^{n/2} & \text{nếu } n \text{ chẵn} \\ x \cdot x^{n/2} \cdot x^{n/2} & \text{nếu } x \text{ lẻ} \end{cases}$.

Bước trị. Tính toán $x^{n/2}$ trong trường hợp x chẵn, $x \cdot x^{n/2}$ trong trường hợp x lẻ.

Bước tổng hợp. Kết quả chính là $x^{n/2} \cdot x^{n/2}$ trong trường hợp x chẵn và $x \cdot x^{n/2} \cdot x^{n/2}$ trong trường hợp x lẻ.

Chương trình nâng x lên lũy thừa n bằng kỹ thuật chia và trị được thể hiện như sau:

```
#include <iostream>
#include <iomanip>
using namespace std;
int power(int x, unsigned int n){
    if( n == 0)
        return 1;
    else if (n%2 == 0)
        return power(x, n/2)*power(x, n/2);
    else
        return x*power(x, n/2)*power(x, n/2);
}
```

```

    }
    int main(){
        int x = 2;
        unsigned int n = 5;
        cout<<power(x, n)<<setw(3);
    }

```

Thử nghiệm thuật toán: tính $S = \text{power}(2, 5)$

```

S    = power(2, 5)
      = 2*power(2, 2)*power(2, 2)
      = 2*power(2, 1)*power(2, 1)*power(2, 1)*power(2, 1)
      = 2*2*power(0, 1)*2*power(2, 0)*2*power(2, 0)*2*power(2, 0)
      = 2*2*2*2*2
      = 32

```

Thuật toán trên có độ phức tạp là $O(n)$. Thuật toán trình bày dưới đây có độ phức tạp $O(\log(n))$.

```

#include <iostream>
#include <iomanip>
using namespace std;
int power(int x, unsigned int n){
    int temp;
    if( n == 0)
        return 1;
    temp = power(x, n/2);
    if (n%2 == 0)
        return temp*temp;
    else
        return x*temp*temp;
}
int main(){
    int x = 2;
    unsigned int n = 5;
    cout<<power(x, n)<<setw(3);
}

```

5.2. Một số bài toán xử lý bằng thuật toán chia và trị cơ bản

Dưới đây là một số bài toán xử lý bằng thuật toán chia và trị cơ bản :

1. Cho mảng $A[]$ gồm N phần tử đã được sắp xếp. Nhiệm vụ của bạn là tìm vị trí của phần tử X trong $A[]$

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số N, X tương ứng với số phần tử của mảng A[] và số X; phần thứ 2 là N số của mảng A[]; các số được viết cách nhau một vài khoảng trống.
- T, N, X, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, X, A[i] \leq 10^6$.

Output:

- Đưa ra kết quả 1 nếu X có mặt trong A[], ngược lại đưa ra -1 của mỗi test theo từng dòng.

Input Examples	Output Examples
2 5 6 1 2 3 4 6 5 2 1 3 4 5 6	1 -1

2. Cho mảng A[] gồm N phần tử chưa được sắp xếp. Nhiệm vụ của bạn là sắp xếp các phần tử của mảng A[] theo thứ tự tăng dần bằng thuật toán Merge Sort.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N tương ứng với số phần tử của mảng A[]; phần thứ 2 là N số của mảng A[]; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, A[i] \leq 10^6$.

Output:

- Đưa ra kết quả 1 nếu X có mặt trong A[], ngược lại đưa ra -1 của mỗi test theo từng dòng.

Input Examples	Output Examples
2 5 4 1 3 9 7 10 10 9 8 7 6 5 4 3 2 1	1 3 4 7 9 1 2 3 4 5 6 7 8 9 10

3. Cho mảng A[] gồm N phần tử chưa được sắp xếp. Nhiệm vụ của bạn là sắp xếp các phần tử của mảng A[] theo thứ tự tăng dần bằng thuật toán Quick Sort.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N tương ứng với số phần tử của mảng A[]; phần thứ 2 là N số của mảng A[]; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $A[i] \leq 10^6$.

Output:

- Đưa ra kết quả 1 nếu X có mặt trong A[], ngược lại đưa ra -1 của mỗi test theo từng dòng.

Input Examples	Output Examples
2	1 3 4 7 9
5	1 2 3 4 5 6 7 8 9 10
4 1 3 9 7	
10	
10 9 8 7 6 5 4 3 2 1	

4. Cho mảng A[] gồm N phần. Ta gọi số đảo ngược của mảng A[] là số các cặp i, j sao cho $i < j$ và $A[i] > A[j]$. Đối với mảng đã được sắp xếp thì số đảo ngược là 0. Mảng đã sắp theo thứ tự giảm dần có số đảo ngược cực đại. Nhiệm vụ của bạn là hãy đưa ra số đảo ngược của mảng A[] gồm N phần tử.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số N tương ứng với số phần tử của mảng A[]; phần thứ 2 là N số của mảng A[]; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^7$; $1 \leq A[i] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1	3
5	
2 4 1 3 5	

5. Cho mảng số N. Ta gọi số đảo của N là R. Hãy tìm lũy thừa R của N. Đưa ra kết quả của bài toán dưới dạng modulo với $10^9 + 7$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm là số N được ghi trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^{10}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	4
2	864354781
12	

6. Cho hai đa thức P và Q được biểu diễn như một mảng bao gồm các hệ số của đa thức. Ví dụ với $P(x) = 5 + 0x^1 + 10x^2 + 6x^3$ được biểu diễn như mảng $P[] = \{5, 0, 10, 6\}$. Hãy đưa ra đa thức $R = P \times Q$ theo các hệ số của R với cách biểu diễn như trên.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng thứ nhất đưa vào hai số M, N tương ứng với lũy thừa lớn nhất của đa thức P và Q; dòng tiếp theo đưa vào M số là hệ số của đa thức P; dòng cuối cùng đưa vào M số là hệ số của đa thức Q.
- T, M, N, $P[i]$, $Q[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq M, N \leq 100$; $1 \leq P[i], Q[i] \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	2 0 10 4 12 8
4 3	4 36 14 39 79 23 34 35
1 0 3 2	
2 0 4	
5 4	
1 9 3 4 7	
4 0 2 5	

7. Cho mảng $A[]$ gồm N số có cả các số âm và số dương. Nhiệm vụ của bạn là tìm mảng con liên tục có tổng lớn nhất của mảng. Ví dụ với mảng $A[] = \{-2, -5, 6, -2, -3, 1, 5, -6\}$ ta có kết quả là 7 tương ứng với dãy con $\{6, -2, -3, 1, 5\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào hai số N tương ứng với số phần tử của mảng; dòng tiếp theo đưa vào N số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 100$; $-100 \leq A[i] \leq 100$.

Output:

- Đưa ra tổng con liên tục lớn nhất của mỗi test theo từng dòng.

Input Examples	Output Examples
1 8 -2 -5 6 -2 -3 1 5 -6	7

8. Cho hai xâu nhị phân biểu diễn hai số. Nhiệm vụ của bạn là đưa ra tích của hai số. Ví dụ với xâu S1="1100" và S2="1010" ta sẽ có kết quả là 120.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 hai xâu nhị phân S1, S2 được viết trên một dòng.
- T, S1, S2 thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(S1), \text{length}(S2) \leq 100$.

Output:

- Đưa ra tích của mỗi test theo từng dòng.

Input Examples	Output Examples
2 1100 01 01 01	12 1

5.3. Một số bài toán xử lý bằng thuật toán chia và trị nâng cao

Dưới đây là một số bài toán xử lý bằng thuật toán chia và trị nâng cao :

9. Cho mảng đã được sắp xếp A[] gồm N phần tử không có hai phần tử giống nhau và số X. Nhiệm vụ của bạn là tìm floor(X). Trong đó, $K = \text{floor}(X)$ là phần tử lớn nhất trong mảng A[] lớn hơn hoặc bằng X.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số N là số phần tử của mảng A[] và số X; dòng tiếp theo đưa vào N số của mảng A[]; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^7$; $1 \leq A[i] \leq 10^{18}$.

Output:

- Đưa ra vị trí của $\text{floor}(X)$ trong mảng $A[]$ hoặc -1 nếu không tồn tại $\text{floor}(X)$ của mỗi test theo từng dòng.

Input Examples	Output Examples
3	-1
7 0	1
1 2 8 10 11 12 19	3
7 5	
1 2 8 10 11 12 19	
7 10	
1 2 8 10 11 12 19	

10. Cho hai mảng đã được sắp xếp $A[]$, $B[]$ gồm M , N phần tử theo thứ tự và số K . Nhiệm vụ của bạn là tìm phần tử ở vị trí số K sau khi trộn hai mảng để nhận được một mảng được sắp.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng thứ nhất đưa vào số M , N , K ; dòng tiếp theo đưa vào M số của mảng $A[]$; dòng tiếp theo đưa vào N số của mảng $B[]$; các số được viết cách nhau một vài khoảng trống.
- $T, M, N, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, A[i], B[i] \leq 10^6$; $1 \leq K \leq N+M$.

Output:

- Đưa ra giá trị phần tử thứ K của mỗi test theo từng dòng.

Input Examples	Output Examples
1	6
5 4 5	
2 3 6 7 9	
1 4 8 10	

11. Cho mảng đã được sắp xếp $A[]$ gồm N phần tử. Các phần tử của mảng $A[]$ có thể giống nhau. Mảng $A[]$ đã được quay vòng K lần. Hãy tìm số lần quay vòng K .

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số M , N , K ; dòng tiếp theo đưa vào N số của mảng $A[]$ các số được viết cách nhau một vài khoảng trống.

- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^7$; $0 \leq A[i] \leq 10^{18}$.

Output:

- Đưa ra số lần quay vòng K của mỗi test theo từng dòng.

Input Examples	Output Examples
2	1
5	0
5 1 2 3 4	
5	
1 2 3 4 5	

12. Cho hai mảng đã được sắp xếp $A[]$ gồm N phần tử. Các phần tử của mảng $A[]$ đều khác nhau. Mảng $A[]$ đã được quay vòng với số lần không biết trước. Hãy tìm phần tử nhỏ nhất của mảng $A[]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số M, N, K ; dòng tiếp theo đưa vào N số của mảng $A[]$ các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^7$; $0 \leq A[i] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	1
5	5
4 5 1 2 3	
6	
10 20 30 40 50 5 7	

13. Cho hai mảng đã được sắp xếp $A[]$ và $B[]$ gồm N và $N-1$ phần tử. Các phần tử của mảng $A[]$ chỉ khác mảng $B[]$ một phần tử duy nhất. Hãy tìm vị trí của phần tử khác nhau giữa $A[]$ và $B[]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng thứ nhất đưa vào số N ; dòng tiếp theo đưa vào N số của mảng $A[]$; dòng tiếp theo đưa vào $N-1$ số của mảng $B[]$; các số được viết cách nhau một vài khoảng trống.

- $T, N, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^7$; $0 \leq A[i] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	4
7	3
2 4 6 8 9 10 12	
2 4 6 8 10 12	
6	
3 5 7 9 11 13	
3 5 7 11 13	

14. Cho hai mảng $A[]$ và $B[]$ gồm N phần tử chỉ bao gồm các số 0 và 1. Các số 1 được đặt trước các số 0 trong mảng $A[]$. Hãy đếm các số 0 của mảng $A[]$ với thời gian $\log(N)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất đưa vào số N ; dòng tiếp theo đưa vào N số của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 100$; $0 \leq A[i] \leq 1$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
3	3
12	5
1 1 1 1 1 1 1 1 0 0 0	0
5	
0 0 0 0 0	
6	
1 1 1 1 1 1	

5.4. Một số bài toán xử lý bằng thuật toán chia và trị tiên tiến

Dưới đây là một số bài toán xử lý bằng thuật toán chia và trị tiên tiến :

15. Cho mảng $A[]$ gồm N phần tử. Nhiệm vụ của bạn là tìm tổng của tất cả các phần tử lớn nhất của mỗi dãy con liên tục. Ví dụ với mảng $A[] = \{1, 3, 1, 7\}$ ta có kết quả là 42 bằng cách thực hiện như sau:

- Phần tử lớn nhất của dãy con $\{1\}$: 1
- Phần tử lớn nhất của dãy con $\{1, 3\}$: 3
- Phần tử lớn nhất của dãy con $\{1, 3, 1\}$: 3
- Phần tử lớn nhất của dãy con $\{1, 3, 1, 7\}$: 7
- Phần tử lớn nhất của dãy con $\{3\}$: 3
- Phần tử lớn nhất của dãy con $\{3, 1\}$: 3
- Phần tử lớn nhất của dãy con $\{3, 1, 7\}$: 7
- Phần tử lớn nhất của dãy con $\{1\}$: 1
- Phần tử lớn nhất của dãy con $\{1, 7\}$: 7
- Phần tử lớn nhất của dãy con $\{7\}$: 7

Từ đây ta có kết quả: $1+3+3+7+3+3+7+1+7+7=42$

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào số phần tử của mảng N ; phần thứ hai đưa vào N phần tử của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 100$; $1 \leq A[i] \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1	42
4	
1 3 1 7	

16. Cho số tự nhiên N và thực hiện trừ N lần lượt N đi chữ số đầu tiên của N để N trở về 0. Ví dụ với $N=13$ ta thực hiện các phép dịch chuyển N về 0 như sau:

- Bước 1: $N=N-1 = 13-1=12$
- Bước 2: $N=N-1 = 12-1=11$
- Bước 3: $N=N-1 = 11-1=10$
- Bước 4: $N=N-1 = 10-1=9$
- Bước 5: $N=N-1 = 9-9=0$

Cho K là số các phép dịch chuyển N về 0 theo nguyên tắc kể trên. Nhiệm vụ của bạn là tìm số N lớn nhất từ xuất phát điểm ban đầu.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là số tự nhiên K được viết trên một dòng.
- T, K thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1	9
2	10
3	

17. Cho hai số nguyên dương A và B, trong đó A là tổng các chữ số của N, B là tổng bình phương các chữ số của N. Nhiệm vụ của bạn là tìm số N nhỏ nhất thỏa mãn A và B. Nếu không tồn tại N thỏa mãn A và B hãy đưa ra -1. Giả thiết N có không quá 100 chữ số. Ví dụ với A = 18, B = 162 ta tìm được số nhỏ nhất N=99 vì $9+9=18$ và $9^2 + 9^2 = 162$. Với A = 12, B = 9 ta có kết quả là -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là cặp số A, B được viết trên một dòng.
- T, A, B thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq A \leq 1000$; $1 \leq B \leq 10000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	99
18 162	-1
12 9	

18. Giả sử tra có N cuốn sách. Cuốn sách thứ i có P_i trang sách. Nhiệm vụ của bạn là giao sách đọc đến M sinh viên. Mỗi cuốn sách được giao cho duy nhất một sinh viên. Mỗi sinh viên đọc ít nhất một cuốn sách. Các cuốn sách được giao cho một sinh viên liên tiếp nhau. Nhiệm vụ của bạn là tìm giải pháp giao sách đọc cho sinh viên sao cho số lượng trang sách nhiều nhất giao cho một sinh viên là ít nhất. Ví dụ với số sách N là 4, số sinh viên M = 2 và số trang trong mỗi cuốn sách là {12, 34, 67, 90} ta có các cách phân công 2 sinh viên đọc sách như sau:

- [12] [34, 67, 90]: sinh viên 2 đọc nhiều nhất $34+67+90=191$.

- [12, 34] [67,90]: sinh viên 2 đọc nhiều nhất $67+90 = 157$.
- [12, 34, 67] [90]: sinh viên 1 đọc nhiều nhất $12+34+67 = 113$

Như vậy, phương án 3 là 113 giao nhiều trang sách nhất cho một sinh viên là ít nhất.

Input:

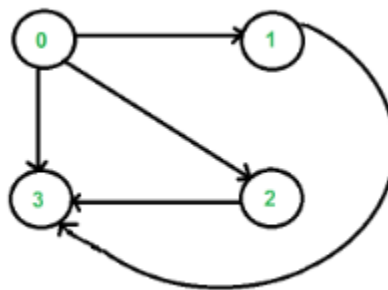
- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào số lượng sách N và số lượng sinh viên M; dòng tiếp theo đưa vào số trang của mỗi sách; các số được viết cách nhau một vài khoảng trống.
- T, N, M, P[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, M \leq 10^6$; $1 \leq P[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng. Đưa ra -1 nếu không có giải pháp giao sách cho M sinh viên.

Input Examples	Output Examples
2	113
4 2	32
12 34 67 90	
3 2	
15 17 20	

19. Cho đồ thị có hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng ma trận kề. Nhiệm vụ của bạn là đếm tất cả các đường đi từ đỉnh u đến đỉnh v qua chính xác K cạnh của đồ thị. Ví dụ với đồ thị dưới đây ta có hai đường đi qua chính xác 2 cạnh: 0-2-3; 0-1-3



Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào số đỉnh của đồ thị; dòng tiếp theo đưa vào ma trận kề biểu diễn đồ thị; dòng cuối cùng đưa vào bộ ba số u, v, K; các số được viết cách nhau một vài khoảng trống.

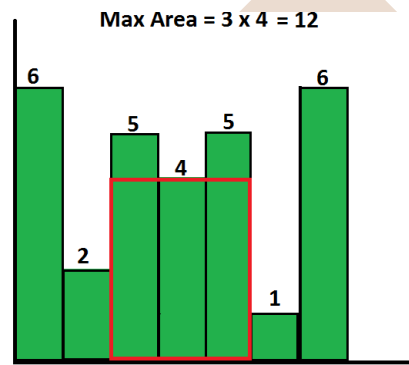
- T, V, u, v, K thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, u, v, K \leq 50$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1 4 0 1 1 1 0 0 0 1 0 0 0 1 0 0 0 0 0 3 2	2

20. Cho một biểu đồ gồm N hình chữ nhật liên tục với độ cao khác nhau và có cùng chiều rộng là 1. Nhiệm vụ của bạn là tìm diện tích hình chữ nhật lớn nhất có thể tạo ra bởi biểu đồ.



Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào N là số lượng hình chữ nhật; dòng tiếp theo đưa vào $A[i]$ là độ cao của các hình chữ nhật; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$; $1 \leq A[i] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2 7 6 2 5 4 5 1 6 4 6 3 4 2	12 9

CHƯƠNG 6. MÔ HÌNH THUẬT TOÁN QUI HOẠCH ĐỘNG

Trong ngành khoa học máy tính, quy hoạch động là một phương pháp giảm thời gian chạy của các thuật toán thể hiện các tính chất của các bài toán con gối nhau (overlapping subproblem) và cấu trúc con tối ưu (optimal substructure). Nhà toán học Richard Bellman đã phát minh phương pháp quy hoạch động vào năm 1953. Ngành này đã được thành lập như là một chủ đề về kỹ nghệ và phân tích hệ thống đã được tổ chức IEEE thừa nhận. Các phương pháp được áp dụng trong quy hoạch động bao gồm: Các bài toán con gối nhau; Cấu trúc con tối ưu; Memoization. Quy hoạch động thường dùng một trong hai cách tiếp cận: top-down (Từ trên xuống) và bottom up. Đối với phương pháp top-down, bài toán được chia thành các bài toán con, các bài toán con này được giải và lời giải được ghi nhớ để phòng trường hợp cần dùng lại chúng. Đây là đệ quy và lưu trữ được kết hợp với nhau. Đối với phương pháp bottom-up, tất cả các bài toán con có thể cần đến đều được giải trước, sau đó được dùng để xây dựng lời giải cho các bài toán lớn hơn. Cách tiếp cận này hơi tốt hơn về không gian bộ nhớ dùng cho ngăn xếp và số lời gọi hàm. Nội dung chính của chương này sẽ cung cấp cho người đọc những kỹ thuật cơ bản và nâng cao về qui hoạch động trong việc giải quyết các bài toán cụ thể.

6.1. Giới thiệu mô hình thuật toán qui hoạch động

Phương pháp qui hoạch động dùng để giải lớp các bài toán thỏa mãn những điều kiện sau:

- Bài toán lớn cần giải có thể phân rã được thành nhiều bài toán con. Trong đó, sự phối hợp lời giải của các bài toán con cho ta lời giải của bài toán lớn. Bài toán con có lời giải đơn giản được gọi là cơ sở của qui hoạch động. Công thức phối hợp nghiệm của các bài toán con để có nghiệm của bài toán lớn được gọi là công thức truy hồi của qui hoạch động.
- Phải có đủ không gian vật lý lưu trữ lời giải các bài toán con (Bảng phương án của qui hoạch động). Vì qui hoạch động đi giải quyết tất cả các bài toán con, do vậy nếu ta không lưu trữ được lời giải các bài toán con thì không thể phối hợp được lời giải giữa các bài toán con.
- Quá trình giải quyết từ bài toán cơ sở (bài toán con) để tìm ra lời giải bài toán lớn phải được thực hiện sau hữu hạn bước dựa trên bảng phương án của qui hoạch động.

Ví dụ. Tìm số các cách chia số tự nhiên n thành tổng các số tự nhiên nhỏ hơn n . Các cách chia là hoán vị của nhau chỉ được tính là một cách.

Lời giải. Gọi $F[m, v]$ là số cách phân tích số v thành tổng các số nguyên dương nhỏ hơn hoặc bằng m . Khi đó, số các cách chia số v thành tổng các số nhỏ hơn hoặc bằng m được chia thành hai loại:

- **Loại 1:** Số các cách phân tích số v thành tổng các số nguyên dương không chứa số m . Điều này có nghĩa số các cách phân tích Loại 1 là số các cách chia số v thành tổng các số nguyên dương nhỏ hơn hoặc bằng $m-1$. Như vậy số Loại 1 chính là $F[m-1, v]$.
- **Loại 2:** Số các cách phân tích số v thành tổng các số nguyên dương chứa ít nhất một số m . Nếu ta xem sự xuất hiện của một hay nhiều số m trong phép phân tích v chỉ là 1, thì theo nguyên lý nhân số lượng các cách phân tích loại này chính là số cách phân tích số v thành tổng các số nhỏ hơn $m-v$ hay $F[m, v-m]$.

Trong trường hợp $m > v$ thì $F[m, v-m] = 0$ nên ta chỉ có các cách phân tích loại 1. Trong trường hợp $m \leq v$ thì ta có cả hai cách phân tích loại 1 và loại 2. Vì vậy:

- $F[m, v] = F[m-1, v]$ nếu $m > v$.
- $F[m, v] = F[m-1, v] + F[m, v-m]$ nếu $m \leq v$.

Tổng quát.

- **Bước cơ sở:** $F[0, 0] = 1$; $F[0, v] = 0$ với $v > 0$.
- **Tính toán giá trị bằng phương án dựa vào công thức truy hồi:**
 - $F[m, v] = F[m-1, v]$ nếu $v > m$;
 - $F[m, v] = F[m-1, v] + F[m, v-m]$ nếu $m \leq v$.
- **Lưu vết.** tìm $F[5, 5] = F[4, 5] + F[5, 0] = 6 + 1 = 7$.

	0	1	2	3	4	5
0	1	0	0	0	0	0
1	1	1	1	1	1	1
2	1	1	2	2	3	3
3	1	1	2	3	4	5
4	1	1	2	3	5	6
5	1	1	2	3	5	7

Chương trình giải quyết bài toán bằng qui hoạch động như dưới đây.

```
#include <iostream>
using namespace std;
int main (void ) {
    int F[100][100], n, m, v;
    cout<<"Nhập n="; cin>>n; //Nhập n=5
    for ( int j=0; j <=n; j++) F[0][j]=0; //thiết lập dòng 0 là 0.
    F[0][0] = 1; //duy có F[0][0] thiết lập là 1
    for (m =1; m<=n; m++) { //xây dựng bảng phương án
        for (v = 0; v<=n; v++){
```



```

        if ( m > v ) F[m][v] = F[m-1][v];
        else F[m][v] = F[m-1][v] + F [m][v-m];
    }
}
cout<<"Kết quả:"<<F[n][n]<<endl;
}

```

6.2. Một số bài toán xử lý bằng thuật toán qui hoạch động cơ bản

Dưới đây là một số bài toán xử lý bằng thuật toán qui hoạch động cơ bản :

1. Catalan Number là dãy số thỏa mãn biểu thức:

$$C_n = \begin{cases} 0 & \text{nếu } n = 0 \\ \sum_{i=0}^{n-1} C_i C_{n-i-1} & \text{nếu } n > 0 \end{cases}$$

Dưới đây là một số số Catalan với $n=0, 1, 2, \dots$: 1, 1, 2, 5, 14, 42, 132, 429,... Cho số tự nhiên N. Nhiệm vụ của bạn là đưa ra số Catalan thứ N.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số nguyên n.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
3	42
5	14
4	16796
10	

2. Cho ba số n, r, p. Nhiệm vụ của bạn là tìm $C(n, r) \% p$. Trong đó, $p = 10^9 + 7$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một cặp n, r được viết trên một dòng.
- T, n, r thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 1000$; $1 \leq r \leq 800$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	3
3 2	6
4 2	

3. $P(n, k)$ là số phép biểu diễn các tập con có thứ tự gồm k phần tử của tập gồm n phần tử. Số $P(n, k)$ được định nghĩa theo công thức sau:

$$P(n, k) = \begin{cases} 0 & \text{nếu } k > n \\ \frac{n!}{(n-k)!} = n \cdot (n-1) \dots (n-k+1) & \text{nếu } k \leq n \end{cases}$$

Cho số hai số n, k . Hãy tìm $P(n, k)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một cặp số n, k được viết trên một dòng.
- T, n, k thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, k \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	20
5 3	12
4 2	

4. Cho mảng $A[]$ gồm N phần tử và số K . Hãy đếm tất cả các dãy con của $A[]$ sao cho tổng các phần tử của dãy con đúng bằng K .

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào hai số N và K ; dòng tiếp theo đưa vào N số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, K, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, K, A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	3
6 10	3
2 3 5 6 8 10	
5 10	
1 2 3 4 5	

5. Số Ugly là các số chỉ có ước số là 2, 3, 5. Theo qui ước số 1 cũng là 1 số Ugly. Dưới đây là 11 số Ugly đầu tiên: 1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15. Cho số tự nhiên N, nhiệm vụ của bạn là tìm số Ugly thứ N.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số tự nhiên N được viết trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^4$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	12
10	4
4	

6. Cho hai chuỗi ký tự str1, str2 chỉ chứa các ký tự in hoa. Nhiệm vụ của bạn là tìm độ dài chuỗi con dài nhất có mặt trong cả hai chuỗi.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào độ dài chuỗi str1 và str2; dòng tiếp theo đưa vào chuỗi str1; dòng cuối cùng đưa vào chuỗi str2.
- T, str1, str2 thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{size}(\text{str1}), \text{size}(\text{str2}) \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
----------------	-----------------

2	3
6 6	2
ABCDGH	
AEDFHR	
3 2	
ABC	
AC	

7. Cho chuỗi ký tự str. Nhiệm vụ của bạn là tìm độ dài chuỗi con lặp lại dài nhất trong str.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào độ dài chuỗi str; dòng tiếp theo đưa vào chuỗi str.
- T, str thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{size}(\text{str}) \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	0
3	2
abc	
5	
axxxy	

8. Cho dãy số A[] gồm N số. Nhiệm vụ của bạn là tìm độ dài dãy con dài nhất được sắp theo thứ tự tăng dần của dãy A[]. Dãy con của dãy số A[] không nhất thiết phải liên tục hoặc duy nhất. Các số giống nhau không được đếm nhiều lần trong dãy tăng dần nhất.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào N là số phần tử của dãy A[]; dòng tiếp theo đưa vào N số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 1000$; $0 \leq A[i] \leq 300$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	6
16	3
0 8 4 12 2 10 6 14 1 9 5 13 3 11 7 15	
6	
5 8 3 7 9 1	

9. Cho ba chuỗi ký tự X, Y, Z. Nhiệm vụ của bạn là tìm độ dài chuỗi chung dài nhất có mặt trong cả ba chuỗi.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào độ dài chuỗi X, Y, X; dòng tiếp theo đưa vào ba chuỗi X, Y, Z.
- T, X, Y, Z thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{size}(X), \text{size}(Y), \text{size}(Z) \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1	5
5 8 13	
geeks geeksfor geeksforgeeks	

10. Cho dãy số A[] gồm N số. Nhiệm vụ của bạn là tìm tổng lớn nhất của dãy con được sắp theo thứ tự tăng dần của dãy A[]. Ví dụ với dãy A[] = {1, 101, 2, 3, 100, 4, 5} ta có kết quả là $106 = 1 + 2 + 3 + 100$. Với dãy A[] = {10, 7, 5} ta có kết quả là 10. Với dãy A[] = {1, 2, 3, 5} ta có kết quả là 11.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào N là số phần tử của dãy A[]; dòng tiếp theo đưa vào N số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$; $0 \leq A[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
----------------	-----------------

2	6
16	3
0 8 4 12 2 10 6 14 1 9 5 13 3 11 7 15	
6	
5 8 3 7 9 1	

11. Một dãy số được gọi là Bi-tonic nếu nó được chia thành hai dãy đầu tăng dần và dãy tiếp theo giảm dần. Nhiệm vụ của bạn là tìm tổng lớn nhất dãy con Bi-tonic của dãy số $A[]$. Ví dụ với dãy $A[] = \{1, 15, 51, 45, 33, 100, 12, 18, 9\}$ ta có kết quả là 194 tương ứng với dãy Bi-tonic $\{1, 15, 51, 100, 18, 9\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào N là số phần tử của dãy $A[]$; dòng tiếp theo đưa vào N số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 100$; $0 \leq A[i] \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	6
6	3 210
80 60 30 40 20 10	194
9	
1 15 51 45 33 100 12 18 9	

12. Cho N cặp số, trong đó số thứ nhất bao giờ cũng nhỏ hơn số thứ 2. Ta nói, cặp số $\langle c, d \rangle$ được gọi là theo sau cặp số $\langle a, b \rangle$ nếu $b < c$. Nhiệm vụ của bạn là tìm số lớn nhất chuỗi các cặp thỏa mãn ràng buộc trên. Ví dụ với các cặp $\{\langle 5, 24 \rangle, \langle 39, 60 \rangle, \langle 15, 28 \rangle, \langle 27, 40 \rangle, \langle 50, 90 \rangle\}$ ta có kết quả là 3 tương ứng với chuỗi các cặp $\{\langle 5, 24 \rangle, \langle 27, 40 \rangle, \langle 50, 90 \rangle\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào N là số cặp $\langle a, b \rangle$; dòng tiếp theo đưa vào $2 \cdot N$ số là N cặp số $\langle a, b \rangle$; các số được viết cách nhau một vài khoảng trống.
- T, N, a, b thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, a, b \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	3
5	1
5 24 39 60 15 28 27 40 50 90	
2	
5 10 1 11	

13. Giả sử bạn cần viết N ký tự giống nhau lên màn hình. Bạn chỉ được phép thực hiện ba thao tác dưới đây với chi phí thời gian khác nhau:

- Thao tác insert: chèn một ký tự với thời gian là X.
- Thao tác delete: loại bỏ ký tự cuối cùng với thời gian là Y.
- Thao tác copying: copy và paste tất cả các ký tự đã viết để số ký tự được nhân đôi với thời gian là Z.

Hãy tìm thời gian ít nhất để có thể đưa ra màn hình N ký tự giống nhau. Ví dụ với $N = 9$, $X = 1$, $Y = 2$, $Z = 1$ ta có kết quả là 5 bằng cách thực hiện: insert, insert, copying, copying, insert.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào N là số các ký tự giống nhau cần viết lên màn hình; dòng tiếp theo đưa vào bộ ba số X, Y, Z tương ứng với thời gian thực hiện ba thao tác; các số được viết cách nhau một vài khoảng trống.
- T, N, X, Y, Z thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 100$; $1 \leq X, Y, Z \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	5
9	14
1 2 1	
10	
2 5 4	

14. Cho số nguyên dương N được biểu diễn như một chuỗi ký tự số. Nhiệm vụ của bạn là tìm tổng của tất cả các số tạo bởi các chuỗi con của N . Ví dụ $N="1234"$ ta có kết quả là $1670 = 1 + 2 + 3 + 4 + 12 + 23 + 34 + 123 + 234 + 1234$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số N .
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^{12}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	1670
1234	491
421	

6.3. Một số bài toán xử lý bằng thuật toán qui hoạch động nâng cao

Dưới đây là một số bài toán xử lý bằng thuật toán qui hoạch động nâng cao :

15. Cho một mảng $A[]$ gồm N số nguyên và số K . Nhiệm vụ của bạn là tìm số cách lấy tổng các phần tử của $A[]$ để bằng K . Phép lấy lặp các phần tử hoặc sắp đặt lại các phần tử được chấp thuận. Ví dụ với mảng $A[] = \{1, 5, 6\}$, $K = 7$ ta có 6 cách sau:

$$7 = 1 + 1 + 1 + 1 + 1 + 1 + 1 \text{ (lặp số 1 7 lần)}$$

$$7 = 1 + 1 + 5 \text{ (lặp số 1)}$$

$$7 = 1 + 5 + 1 \text{ (lặp và sắp đặt lại số 1)}$$

$$7 = 1 + 6$$

$$7 = 6 + 1$$

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào N và K ; dòng tiếp theo đưa vào N số của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, K, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 1000$; $1 \leq A[i] \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng. Khi kết quả quá lớn đưa ra kết quả dưới dạng modulo với 10^9+7 .

Input Examples	Output Examples
----------------	-----------------

2	6
3 7	150
1 5 6	
4	
12 3 1 9	

16. Cho dãy số $A[]$ gồm N phần tử. Hãy tìm tổng lớn nhất của dãy con của dãy số $A[]$ sao cho dãy con không có hai số cạnh nhau trong $A[]$. Ví dụ với $A[] = \{6, 7, 1, 3, 8, 2, 4\}$ ta có kết quả là 19 tương ứng với tổng của dãy con $\{6, 1, 6, 4\}$ không có hai phần tử nào kề nhau trong $A[]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là một số N ; dòng tiếp theo đưa vào N số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$; $1 \leq A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	110
6	13
5 5 10 100 10 5	
4	
3 2 7 10	

17. Có N ngôi nhà trên một dãy phố, mỗi ngôi nhà chứa đựng một số lượng tài sản khác nhau. Một tên chộm muốn ắp cắp được nhiều nhất tài sản của dãy phố nhưng không muốn lấy tài sản của hai nhà kề nhau. Hãy cho biết, bằng cách đó tên chộm có thể đánh cắp được nhiều nhất bao nhiêu tài sản.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là một số N là số lượng ngôi nhà; dòng tiếp theo đưa vào N số là tài sản tương ứng trong mỗi ngôi nhà; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$; $1 \leq A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	110
6	13
5 5 10 100 10 5	
4	
3 2 7 10	

18. Cho mảng $A[]$ gồm N số nguyên. Nhiệm vụ của bạn là sắp xếp lại mảng số với số lượng bước là ít nhất. Tại mỗi bước, bạn chỉ được phép chèn phần tử bất kỳ của mảng vào vị trí bất kỳ trong mảng. Ví dụ $A[] = \{2, 3, 5, 1, 4, 7, 6\}$ sẽ cho ta số phép chèn ít nhất là 3 bằng cách lấy số 1 chèn trước số 2, lấy số 4 chèn trước số 5, lấy số 6 chèn trước số 7 ta nhận được mảng được sắp.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là một số N ; dòng tiếp theo đưa vào N số của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 1000$; $1 \leq A[i] \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1	3
7	
2 3 5 1 4 7 6	

19. Cho số nguyên dương N . Hãy đếm số bước ít nhất để đưa N về 1 bằng cách thực hiện ba thao tác dưới đây:

- Nếu N chia hết cho 2 bạn có thể giảm $N = N/2$.
- Nếu N chia hết cho 3 bạn có thể giảm $N = N/3$.
- Giảm N đi 1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một số N được viết trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	3
10	2
6	

20. Cho hai chuỗi ký tự str1, str2 bao gồm các ký tự in thường và các thao tác dưới đây:

- Insert: chèn một ký tự bất kỳ vào str1.
- Delete: loại bỏ một ký tự bất kỳ trong str1.
- Replace: thay một ký tự bất kỳ trong str1.

Nhiệm vụ của bạn là đếm số các phép Insert, Delete, Replace ít nhất thực hiện trên str1 để trở thành str2.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ đôi hai chuỗi str1 và str2.
- T, str1, str2 thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(\text{str1}), \text{length}(\text{str2}) \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1	1
geek gesek	

21. Giả sử bạn đang ở điểm có tọa độ nguyên dương (n,m) và cần dịch chuyển về tọa độ (0,0). Mỗi bước dịch chuyển bạn chỉ được phép dịch chuyển đến tọa độ (n-1, m-1) hoặc (n, m-1), hoặc (0,m), hoặc (n, 0). Hãy đếm số cách bạn có thể dịch chuyển về tọa độ (0,0).

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là bộ n, m được viết cách nhau một vài khoảng trống.
- T, n, m thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m \leq 25$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
3	10
3 2	84
3 6	1
3 0	

22. Một con ếch có thể nhảy 1, 2, 3 bước để có thể lên đến một đỉnh cần đến. Hãy đếm số các cách con ếch có thể nhảy đến đỉnh.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là số n là số bước con ếch có thể lên được đỉnh.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 50$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	1
1	13
5	

6.4. Một số bài toán xử lý bằng thuật toán qui hoạch động tiên tiến

Dưới đây là một số bài toán xử lý bằng thuật toán qui hoạch động tiên tiến :

23. Cho một mảng $A[]$ gồm N số nguyên. Nhiệm vụ của bạn là tìm số phép thay đổi ít nhất trên mảng để mảng trở thành mảng tăng nghiêm ngặt. Một mảng được gọi là tăng nghiêm ngặt nếu với mọi $0 \leq i < N$ thì $A[i] < A[i+1]$. Ví dụ với mảng $A[] = \{1, 2, 6, 5, 4\}$ ta có kết quả là 2 với phép thay đổi $A[3], A[4]$ thành số có giá trị lớn hơn 6. Với mảng $A[] = \{1, 2, 3, 4, 5\}$ ta có kết quả là 0. Chú ý mảng phải duy trì tính tăng của các số nguyên. Với mảng $A[] = \{6, 9, 7\}$ bạn không được phép chèn giá trị vào giữa 6 và 7.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào N là số phần tử của mảng; dòng tiếp theo đưa vào N số của mảng A[]; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$; $1 \leq A[i] \leq 10^8$.

Output:

Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	2
6	0
1 2 3 6 5 4	
7	
1 2 3 4 5 6 7	

24. Cho đồ thị có hướng có trọng số $G = \langle V, E \rangle$. Nhiệm vụ của bạn là tìm đường đi ngắn nhất giữa tất cả các cặp đỉnh của đồ thị. Đồ thị được biểu diễn dưới dạng ma trận trọng số, nếu u, v là một cạnh của đồ thị thì $G[u][v] = \text{weight}(u, v)$, trường hợp khác $G[u][v] = \text{INF} = 1e7$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào V là số đỉnh của đồ thị; phần thứ hai đưa vào ma trận kề của đồ thị các số được viết cách nhau một vài khoảng trống.
- T, V, G[i][j] thỏa mãn ràng buộc: $1 \leq T \leq 20$; $1 \leq V \leq 100$; $1 \leq G[i][j] \leq 500$.

Output:

- Đưa ra kết quả mỗi test theo thuật toán Floyd Warshall.

Input Examples	Output Examples
2	0 25
2	INF 0
0 25	0 1 7
INF 0	1 0 6
3	INF INF 0
0 1 43	
1 0 6	
INF INF 0	

25. Một trong những bài toán kinh điển của lý thuyết tổ hợp là Bài toán cái túi. Bài toán được phát biểu như sau: Một nhà thám hiểm cần đem theo một cái túi trọng lượng không quá W . Có N đồ vật cần đem theo. Đồ vật thứ i có trọng lượng $A[i]$, có giá trị sử dụng $C[i]$. Nhiệm vụ của bạn là hãy tìm cách đưa đồ vật vào túi cho nhà thám hiểm sao cho tổng giá trị sử dụng các đồ vật trong túi là lớn nhất. Giả thiết mỗi đồ vật được đưa vào túi nhiều nhất là 1 (0-1 Knapsack).

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất đưa vào hai số N, W tương ứng với số lượng đồ vật và trọng lượng túi; phần thứ 2 đưa vào $2*N$ số tương ứng với trọng lượng đồ vật $A[i]$ và giá trị sử dụng $C[i]$ của mỗi đồ vật.
- $T, N, W, A[i], C[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, W \leq 100$; $1 \leq A[i], C[i] \leq 100$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	3
3 4	1
1 2 3	
4 5 1	
3 2	
1 2 3	
4 5 6	

26. Một bản tin M đã mã hóa bí mật thành các con số theo ánh xạ như sau: 'A' $\rightarrow$ 1, 'B' $\rightarrow$ 2, ..., 'Z' $\rightarrow$ 26. Hãy cho biết có bao nhiêu cách khác nhau để giải mã bản tin M . Ví dụ với bản mã $M = "123"$ nó có thể được giải mã thành ABC (1 2 3), LC (12 3), AW (1 23).

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một xâu ký tự số M .
- T, M thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(M) \leq 40$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
----------------	-----------------

2	3
123	2
2563	

27. Cho chuỗi ký tự str. Nhiệm vụ của bạn là tìm số phép chèn tối thiểu các ký tự vào str để str trở thành chuỗi đối xứng. Ví dụ: str = "ab" ta có số phép chèn tối thiểu là 1 để trở thành chuỗi đối xứng "aba" hoặc "bab". Với chuỗi str = "aa" thì số phép chèn tối thiểu là 0. Với chuỗi str = "abcd" có số phép chèn tối thiểu là 3 để trở thành chuỗi "dcbabcd"

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một chuỗi ký tự được viết trên một dòng
- T, str thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(\text{str}) \leq 40$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
3	3
abcd	0
aba	3
geeks	

28. Cho chuỗi ký tự str. Nhiệm vụ của bạn là tìm số phép loại bỏ ít nhất các ký tự trong str để str trở thành chuỗi đối xứng. Chú ý, phép loại bỏ phải bảo toàn trình tự trước sau của các ký tự trong str.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một chuỗi ký tự được viết trên một dòng
- T, str thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(\text{str}) \leq 10000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
2	2
aebcbda	8
geeksforgeeks	

29. Cho mảng $A[]$ gồm N số nguyên. Nhiệm vụ của bạn là tìm số phép loại bỏ ít nhất các phần tử trong $A[]$ để các phần tử còn lại tạo thành một dãy được sắp. Chú ý, các phần tử còn lại phải bảo toàn tính trước sau các phần tử trong $A[]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là số N ; dòng tiếp theo đưa vào N số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, A[i] \leq 1000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
1 5 5 6 1 7 4	2

30. Mọi số nguyên dương N đều có thể phân tích thành tổng các bình phương của các số nhỏ hơn N . Ví dụ số $100 = 10^2$ hoặc $100 = 5^2 + 5^2 + 5^2 + 5^2$. Cho số nguyên dương N . Nhiệm vụ của bạn là tìm số bình phương ít nhất có tổng bằng N .

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ là một số tự nhiên N được viết trên 1 dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10000$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input Examples	Output Examples
3 100 6 25	1 3 1

CHƯƠNG 7. MÔ HÌNH THUẬT TOÁN SẮP XẾP VÀ TÌM KIẾM

Một trong những vấn đề quan trọng bậc nhất của khoa học máy tính là sắp xếp và tìm kiếm thông tin. Có thể nói, hầu hết các hoạt động của người dùng hoặc các ứng dụng tin học đều liên quan đến tìm kiếm. Muốn tìm kiếm thông tin nhanh, hiệu quả, chính xác ta cần có phương pháp tổ chức và sắp xếp dữ liệu tốt. Chính vì vậy, sắp xếp được xem như giai đoạn đầu chuẩn bị cho quá trình tìm kiếm. Nội dung chương này trình bày các thuật toán sắp xếp và tìm kiếm, bao gồm: các thuật toán sắp xếp đơn giản, các thuật toán sắp xếp nhanh, các thuật toán tìm kiếm tuyến tính, tìm kiếm nhị phân, tìm kiếm nội suy & tìm kiếm Jumping.

7.1. Giới thiệu vấn đề

Bài toán tìm kiếm có thể được phát biểu như sau: Cho dãy gồm n đối tượng $r_1, r_2, \dots, r_n$. Mỗi đối tượng r_i được tương ứng với một khóa k_i ($1 \leq i \leq n$). Nhiệm vụ của tìm kiếm là xây dựng thuật toán tìm đối tượng có giá trị khóa là X cho trước. X còn được gọi là khóa tìm kiếm hay tham biến tìm kiếm (argument). Bài toán tìm kiếm bao giờ cũng hoàn thành bởi một trong hai tình huống:

- Nếu tìm thấy đối tượng có khóa X trong tập các đối tượng thì ta nói phép tìm kiếm thành công (successful).
- Nếu không tìm thấy đối tượng có khóa X trong tập các đối tượng thì ta nói phép tìm kiếm không thành công (unsuccessful).

Sắp xếp là phương pháp bố trí lại các đối tượng theo một trật tự nào đó. Ví dụ bố trí theo thứ tự tăng dần hoặc giảm dần đối với dãy số, bố trí theo thứ tự từ điển đối với các chữ ký tự. Mục tiêu của sắp xếp là để lưu trữ và tìm kiếm đối tượng (thông tin) để đạt hiệu quả cao trong tìm kiếm. Có thể nói, sắp xếp là sản phẩm của quá trình tìm kiếm. Muốn tìm kiếm và cung cấp thông tin nhanh thì ta cần phải sắp xếp thông tin sao cho hợp lý. Bài toán sắp xếp có thể được phát biểu như sau: Cho dãy gồm n đối tượng $r_1, r_2, \dots, r_n$. Mỗi đối tượng r_i được tương ứng với một khóa k_i ($1 \leq i \leq n$). Nhiệm vụ của sắp xếp là xây dựng thuật toán bố trí các đối tượng theo một trật tự nào đó của các giá trị khóa. Trật tự của các giá trị khóa có thể là tăng dần hoặc giảm dần tùy thuộc vào mỗi thuật toán tìm kiếm cụ thể.

7.2. Một số bài toán xử lý bằng thuật toán sắp xếp và tìm kiếm cơ bản

Dưới đây là một số bài toán xử lý bằng thuật toán sắp xếp và tìm kiếm cơ bản :

- 1. Sorting 1.** Cho mảng $A[]$ gồm n số nguyên khác nhau. Hãy đưa ra các phần tử của mảng theo khuôn dạng lớn nhất, nhỏ nhất, lớn thứ hai, nhỏ thứ 2, ... Ví dụ với $A[] = \{9, 7, 12, 8, 6, 5\}$ ta đưa ra : 12, 5, 9, 6, 8, 7.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên là số phần tử của mảng n ; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.

- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	7 1 6 2 5 4 3
7	9 1 8 2 7 3 6 4
7 1 2 3 4 5 6	
8	
1 6 9 4 3 7 8 2	

- 2. Sorting 2.** Cho mảng $A[]$ gồm n phần tử và số X. Hãy đưa sắp xếp các phần tử của mảng theo trị tuyệt đối của $|X - A[i]|$. Ví dụ với $A[] = \{10, 5, 3, 9, 2\}$ và $X = 7$ ta đưa ra mảng được sắp xếp theo nguyên tắc kể trên: $A[] = \{5, 9, 10, 3, 2\}$ vì $|7-10|=3$, $|7-5|=2$, $|7-3|=4$, $|7-9|=2$, $|7-2|=5$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên là số phần tử của mảng n và X; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- T, n, X thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, X, A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	5 9 10 3 2
5 7	5 4 3 2 1
10 5 3 9 2	
5 6	
1 2 3 4 5	

- 3. Sorting 3.** Cho mảng $A[]$ gồm n phần tử. Hãy tìm số phép đổi chỗ ít nhất giữa các phần tử của mảng để mảng $A[]$ được sắp xếp. Ví dụ với $A[] = \{4, 3, 2, 1\}$ ta cần thực hiện ít nhất 2 phép đổi chỗ: $\text{Swap}(A[0], A[3])$, $\text{Swap}(A[1], A[2])$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên là số phần tử của mảng n và X; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; các số được viết cách nhau một vài khoảng trống.
- T, n thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
--------	---------

2	2
4	2
4 3 2 1	
5	
1 5 4 3 2	

- 4. Sorting 4.** Cho mảng $A[]$ gồm n phần tử, mảng $B[]$ gồm m phần tử khác nhau. Các phần tử của mảng $A[]$ và $B[]$ đã được sắp xếp. Hãy tìm mảng hợp và giao được sắp giữa $A[]$ và $B[]$. Ví dụ với $A[] = \{1, 3, 4, 5, 7\}$, $B[] = \{2, 3, 5, 6\}$ ta có mảng hợp $\text{Union} = \{1, 2, 3, 4, 5, 6, 7\}$, mảng giao $\text{Intersection} = \{3, 5\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m là số phần tử của mảng $A[]$ và $B[]$; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; dòng tiếp theo là m số $B[i]$ của mảng $B[]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, m, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m, A[i], B[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	1 2 3 4 5
5 3	1 2 3
1 2 3 4 5	
1 2 3	

- 5. Sorting 5.** Cho mảng $A[]$ gồm n phần tử, mảng $B[]$ gồm m phần tử khác nhau. Các phần tử của mảng $A[]$ và $B[]$ chưa được sắp xếp. Hãy tìm mảng hợp và giao được sắp giữa $A[]$ và $B[]$. Ví dụ với $A[] = \{7, 1, 5, 2, 3, 6\}$, $B[] = \{3, 8, 6, 20, 7\}$ ta có mảng hợp $\text{Union} = \{1, 2, 3, 5, 6, 7, 8, 20\}$, mảng giao $\text{Intersection} = \{3, 6\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m là số phần tử của mảng $A[]$ và $B[]$; dòng tiếp theo là n số $A[i]$ của mảng $A[]$; dòng tiếp theo là m số $B[i]$ của mảng $B[]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, m, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m, A[i], B[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	1 2 3 5 6 7 8 20
6 5	3 6
7 1 5 2 3 6	

3 8 6 20 7	
------------	--

- 6. Sorting 6.** Cho mảng $A[]$ gồm n phần tử. Các phần tử của mảng $A[]$ chỉ bao gồm các số 0, 1, 2. Hãy sắp xếp mảng $A[]$ theo thứ tự tăng dần. Ví dụ với $A[] = \{0, 2, 1, 2, 0\}$ ta kết quả $A[] = \{0, 0, 1, 2, 2\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n là số phần tử của mảng $A[]$; dòng tiếp theo là n số $A[i]$ của mảng $A[]$ các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $0 \leq A[i] \leq 2$; $1 \leq n \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	0 0 1 2 2
5	0 0 1
0 2 1 2 0	
3	
0 1 0	

- 7. Sorting 7.** Cho mảng $A[]$ gồm n phần tử. Hãy tìm dãy con liên tục của mảng $A[R], \dots, A[L]$ sao cho khi sắp xếp lại dãy con ta nhận được một mảng được sắp xếp. Ví dụ với $A[] = \{10, 12, 20, 30, 25, 40, 32, 31, 35, 50, 60\}$ ta chỉ cần sắp xếp lại dãy con từ $A[3], \dots, A[8]: \{30, 25, 40, 32, 31, 35\}$ để có mảng được sắp.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n là số phần tử của mảng $A[]$; dòng tiếp theo là n số $A[i]$ của mảng $A[]$ các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^6$; $0 \leq A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	3 8
11	2 5
10 12 20 30 25 40 32 31 35 50 60	
9	
0 1 15 25 6 7 30 40 50	

- 8. Sorting 8.** Cho mảng $X[]$ gồm n phần tử và mảng $Y[]$ gồm m phần tử. Hãy đếm số các cặp $x^y > y^x$, trong đó $x \in X[]$ và $y \in Y[]$. Ví dụ $X[] = \{2, 1, 6\}$, $Y[] = \{1, 5\}$ ta có kết quả là 3 cặp $(2, 1), (2, 5), (6, 1)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m tương ứng với số phần tử của mảng X[] và Y[]; dòng tiếp theo là n số X[i] của mảng X[]; dòng cuối cùng là m số của mảng Y[]; các số được viết cách nhau một vài khoảng trống.
- T, n, m, X[i], Y[j] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m \leq 10^5$; $1 \leq X[i], Y[j] \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	3
3 2	
2 1 6	
1 5	

9. Sorting 9. Cho mảng A[] gồm n phần tử và số k. Đếm tất cả các cặp phần tử của mảng có tổng bằng k. Ví dụ A[] = {1, 5, 3, 4, 2}, k = 3 ta có kết quả là 2 cặp (1, 4), (5, 2).

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n là số phần tử của mảng A[] và k; dòng tiếp theo là n số A[i] của mảng A[] các số được viết cách nhau một vài khoảng trống.
- T, n, k, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 100$; $0 \leq k \leq 100$; $0 \leq A[i] \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1
5 0	1
1 5 4 1 2	
3 0	
1 1 1	

10. Sorting 10. Cho mảng A[] gồm n phần tử. Nhiệm vụ của bạn là đưa ra mảng đã được sắp xếp bao gồm các chữ số của mỗi phần tử trong A[]. Ví dụ A[] = {110, 111, 112, 113, 114} ta có kết quả là {0, 1, 2, 3, 4}.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n là số phần tử của mảng A[]; dòng tiếp theo là n số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, n, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^7$; $0 \leq A[i] \leq 10^{16}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
--------	---------

2	1 3 4 8
3	1 2 3 4 6
131 11 48	
4	
111 222 333 446	

11. Sum Closest Zezo. Cho mảng $A[]$ gồm n phần tử, hãy tìm cặp phần tử có tổng gần nhất so với 0.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n là số phần tử của mảng $A[]$; dòng tiếp theo đưa vào n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq N \leq 10^3$, $-10^6 \leq A[i] \leq 10^6$.

Output:

- Đưa ra tổng gần nhất với 0 của cặp phần tử.

Input:	Output:
2	-68
3	-14
-8 -66 -60	
6	
-21 -67 -37 -18 4 -65	

12. K Largest Element. Cho mảng $A[]$ gồm n phần tử, hãy tìm k phần tử lớn nhất của mảng. Các phần tử được đưa ra theo thứ tự giảm dần.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào N và K ; dòng tiếp theo đưa vào n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, K, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq K < N \leq 10^3$, $1 \leq A[i] \leq 10^6$.

Output:

- Đưa ra tổng gần nhất với 0 của cặp phần tử.

Input:	Output:
2	12 10 8
5 3	12 9
10 7 9 12 6	
6 2	
9 7 12 8 6 5	

13. K Largest Element. Cho mảng $A[]$ gồm n phần tử đã được sắp xếp. Hãy tìm số lần xuất hiện số X trong mảng. Nếu số lần xuất hiện số x trong mảng là 0 hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .

- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào N và X; dòng tiếp theo đưa vào n số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, N, X, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$, $1 \leq A[i]$, $X \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	4
7 2	-1
1 1 2 2 2 2 3	
7 4	
1 1 2 2 2 2 3	

14. K Largest Element. Cho mảng A[] gồm n phần tử đã được sắp xếp. Hãy tìm số lần xuất hiện số X trong mảng. Nếu số lần xuất hiện số x trong mảng là 0 hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào N và X; dòng tiếp theo đưa vào n số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, N, X, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$, $1 \leq A[i]$, $X \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	4
7 2	-1
1 1 2 2 2 2 3	
7 4	
1 1 2 2 2 2 3	

15. Sum of Primes. Số tự nhiên N. Hãy tìm cặp số nguyên tố đầu tiên có tổng là N. Nếu không tồn tại cặp số nguyên tố có tổng bằng N, hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm là một số N được ghi trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	2 2
4	3 5
8	

7.3. Một số bài toán xử lý bằng thuật toán sắp xếp và tìm kiếm nâng cao

Dưới đây là một số bài toán xử lý bằng thuật toán sắp xếp và tìm kiếm nâng cao :

16. Sorting 11. Cho mảng $A[]$ gồm n phần tử và mảng $B[]$ gồm m phần tử. Nhiệm vụ của bạn là tìm tích giữa phần tử lớn nhất của mảng $A[]$ và phần tử nhỏ nhất của mảng $B[]$.

Ví dụ $A[] = \{5, 7, 112, 9, 3, 6, 2\}$, $B[] = \{1, 2, 6, -1, 0, 9\}$ ta có kết quả là $-9 = 9 * (-1)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m tương ứng với số phần tử của mảng $A[]$ và $B[]$; dòng tiếp theo là n số $A[i]$; dòng cuối cùng là m số $B[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, m, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m \leq 10^6$; $-10^8 \leq A[i] \leq 10^8$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1 3 4 8
3	1 2 3 4 6
131 11 48	
4	
111 222 333 446	

17. Sorting 12. Cho mảng $A[]$ gồm n phần tử và mảng $B[]$ gồm m phần tử. Nhiệm vụ của bạn là tìm tích giữa phần tử lớn nhất của mảng $A[]$ và phần tử nhỏ nhất của mảng $B[]$.

Ví dụ $A[] = \{5, 7, 112, 9, 3, 6, 2\}$, $B[] = \{1, 2, 6, -1, 0, 9\}$ ta có kết quả là $-9 = 9 * (-1)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m tương ứng với số phần tử của mảng $A[]$ và $B[]$; dòng tiếp theo là n số $A[i]$; dòng cuối cùng là m số $B[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, m, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m \leq 10^6$; $-10^8 \leq A[i] \leq 10^8$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	-9
6 6	20
5 7 9 3 6 2	
1 2 6 -1 0 9	
6 6	
1 4 2 3 10 2	
4 2 6 5 2 9	

18. Sorting 13. Cho mảng $A[]$ gồm n phần tử và mảng $B[]$ gồm m phần tử. Nhiệm vụ của bạn là hợp nhất hai mảng $A[]$ và $B[]$ để được một mảng mới đã được sắp xếp. Ví dụ $A[] = \{5, 7, 112, 9, 3, 6, 2\}$, $B[] = \{1, 2, 6, -1, 0, 9\}$ ta có kết quả là $C[] = \{-1, 1, 0, 2, 3, 5, 6, 6, 7, .\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm ba dòng: dòng đầu tiên đưa vào n, m tương ứng với số phần tử của mảng $A[]$ và $B[]$; dòng tiếp theo là n số $A[i]$; dòng cuối cùng là m số $B[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, m, A[i], B[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, m \leq 10^6$; $-10^8 \leq A[i] \leq 10^8$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	2 3 5 10 15 20
3 3	
10 5 15	
20 3 2	

19. Sorting 14. Cho mảng $A[]$ gồm n số nguyên dương. Gọi L, R là max và min các phần tử của $A[]$. Nhiệm vụ của bạn là tìm số phần tử cần thiết cần thêm vào mảng để mảng có đầy đủ các số trong khoảng $[L, R]$. Ví dụ $A[] = \{5, 7, 9, 3, 6, 2\}$ ta nhận được kết quả là 2 tương ứng với các số còn thiếu là 4, 8.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n , tương ứng với số phần tử của mảng $A[]$; dòng tiếp theo là n số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n, A[i] \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1
5	0
4 5 3 8 6	
3	
2 1 3	

20. Sorting 15. Cho mảng $A[]$ gồm n số nguyên dương và số k . Nhiệm vụ của bạn là đếm số các cặp phần tử có hiệu nhỏ hơn k . Ví dụ $A[] = \{1, 10, 4, 2\}$, $k=3$ ta nhận được kết quả là 2 tương ứng với hiệu các cặp $(1, 2), (4, 2)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n, tương ứng với số phần tử của mảng A[] và số k; dòng tiếp theo là n số A[i] ; các số được viết cách nhau một vài khoảng trống.
- T, n, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^4$; $1 \leq k \leq 10^3$; $1 \leq A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	2
4 3	3
1 10 4 2	
3 5	
2 3 4	

21. Sorting 16. Cho mảng A[] gồm n số nguyên. Nhiệm vụ của bạn là sắp xếp mảng theo số lần xuất hiện các phần tử của mảng. Số xuất hiện nhiều lần nhất đứng trước. Nếu hai phần tử có số lần xuất hiện như nhau, số nhỏ hơn đứng trước. Ví dụ A[] = {5, 5, 4, 6, 4}, ta nhận được kết quả là A[] = {4, 4, 5, 5, 6}.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n, tương ứng với số phần tử của mảng A[] và số k; dòng tiếp theo là n số A[i] ; các số được viết cách nhau một vài khoảng trống.
- T, n, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^4$; $1 \leq k \leq 10^3$; $1 \leq A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	4 4 5 5 6
5	9 9 9 2 5
5 5 4 6 4	
5	
9 9 9 2 5	

22. Sorting 17. Cho mảng A[] gồm n số nguyên. Nhiệm vụ của bạn là sắp xếp mảng theo số lần xuất hiện các phần tử của mảng. Số xuất hiện nhiều lần nhất đứng trước. Nếu hai phần tử có số lần xuất hiện như nhau, số nhỏ hơn đứng trước. Ví dụ A[] = {5, 5, 4, 6, 4}, ta nhận được kết quả là A[] = {4, 4, 5, 5, 6}.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm hai dòng: dòng đầu tiên đưa vào n, tương ứng với số phần tử của mảng A[] và số k; dòng tiếp theo là n số A[i] ; các số được viết cách nhau một vài khoảng trống.
- T, n, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^4$; $1 \leq k \leq 10^3$; $1 \leq A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	4 4 5 5 6
5	9 9 9 2 5
5 5 4 6 4	
5	
9 9 9 2 5	

23. Linear Search. Cho mảng $A[]$ gồm n phần tử. Hãy tìm vị trí của phần tử có giá trị X trong mảng $A[]$. Nếu không tìm thấy X hãy đưa ra -1 .

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n , X là số các phần tử của mảng $A[]$ và số X cần tìm; dòng tiếp theo đưa vào n số $A[i]$ ($1 \leq i \leq n$) các số được viết cách nhau một vài khoảng trống.
- T, n, A, X thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, X, A[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	4
5 16	-1
9 7 2 16 4	
7 98	
1 22 57 47 34 18 66	

24. Binary Search. Cho mảng $A[]$ gồm n phần tử đã được sắp xếp. Hãy đưa ra 1 nếu X có mặt trong mảng $A[]$, ngược lại đưa ra -1 .

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n , X là số các phần tử của mảng $A[]$ và số X cần tìm; dòng tiếp theo đưa vào n số $A[i]$ ($1 \leq i \leq n$) các số được viết cách nhau một vài khoảng trống.
- T, n, A, X thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, X, A[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1
5 16	-1
9 7 2 16 4	
7 98	
1 22 57 47 34 18 66	

25. Missing Number. Cho mảng $A[]$ gồm $n-1$ phần tử bao gồm các khác nhau từ 1, 2, ..., n . Hãy tìm số không có mặt trong mảng $A[]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n l; dòng tiếp theo đưa vào n-1 số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, n, A thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	4
5	9
1 2 3 5	
10	
1 2 3 4 5 6 7 8 10	

26. Search in Sorted & Rotated Array. Một mảng được sắp được chia thành hai đoạn tăng dần được gọi là mảng sắp xếp vòng. Ví dụ mảng $A[] = \{ 5, 6, 7, 8, 9, 10, 1, 2, 3, 4 \}$ là mảng sắp xếp vòng. Cho mảng A[] gồm n phần tử, hãy tìm vị trí của phần tử x trong mảng A[] với thời gian $\log(n)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n và x; dòng tiếp theo đưa vào n số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, n, A[i], x thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $x, A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	8
10 3	2
5 6 7 8 9 10 1 2 3 4	
10 3	
1 2 3 4 5 6 7 8 9 10	

27. First & Second Smallest. Cho mảng A[] gồm n phần tử, hãy đưa ra số nhỏ nhất và số nhỏ thứ hai của mảng. Nếu không có số nhỏ thứ hai, hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n là số phần tử của mảng A[]; dòng tiếp theo đưa vào n số A[i]; các số được viết cách nhau một vài khoảng trống.
- T, n, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N$, $A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1
10	-1
5 6 7 8 9 10 1 2 3 4	
5	
1 1 1 1 1	

7.4. Một số bài toán xử lý bằng thuật toán sắp xếp tiên tiến

Dưới đây là một số bài toán xử lý bằng thuật toán sắp xếp và tìm kiếm nâng cao :

28. Tìm X trong A[]. Cho mảng A[] gồm N phần tử được chia thành hai phần: phần thứ nhất bao gồm các số được sắp theo thứ tự tăng dần; phần còn lại được sắp theo thứ tự giảm dần. Nhiệm vụ của bạn là tìm số X có thuộc mảng A[] hay không với thời gian $O(\log(n))$. Nếu tìm thấy X trong mảng A[] hãy đưa ra vị trí của X trong mảng A, ngược lại đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là cặp số N, X; dòng tiếp theo là N số A[i] là các phần tử của mảng A[].
- T, N, X, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^7$, $-10^7 \leq X$, $A[i] \leq 10^7$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	9
10 4	-1
1 2 3 5 10 9 8 7 6 4	
10 4	
1 2 3 5 10 9 8 7 6 -1	

29. Cặp có hiệu là X. Cho mảng A[] gồm N phần tử và số X. Nhiệm vụ của bạn là tìm cặp phần tử $A[i] - A[j] = X$. Nếu tồn tại $A[i] - A[j] = X$ đưa ra 1, ngược lại đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất là cặp số N, X; dòng tiếp theo là N số A[i] là các phần tử của mảng A[].
- T, N, X, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^5$, $1 \leq X$, $A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1
6 78	-1
5 20 3 2 5 80	

5 45	
90 70 20 80 50	

30. Min Rotated Array. Mảng $A[]$ được gọi là mảng sắp xếp vòng nếu $A[]$ được chia thành hai đoạn đã được sắp xếp. Đọc từ phần tử nhỏ nhất đến phần tử cuối cùng và vòng lại các phần tử ở phần còn lại ta sẽ được một dãy đã được sắp xếp. Ví dụ dãy $A[] = \{5, 6, 1, 2, 3, 4\}$ là mảng sắp xếp vòng. Nhiệm vụ của bạn là tìm phần tử nhỏ nhất của mảng sắp xếp vòng với thời gian $O(\log(n))$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào số phần tử của mảng N ; dòng tiếp theo là N số $A[i]$ là các phần tử của mảng $A[]$.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^5$, $1 \leq A[i] \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1
5	5
4 5 1 2 3	
6	
10 20 30 40 50 5 7	

31. First Element Repeating. Cho mảng $A[]$ gồm N phần tử. Hãy tìm phần tử lặp lại đầu tiên của mảng. Ví dụ với mảng $A[] = \{5, 6, 1, 2, 1, 4\}$ thì ta có 1 là phần tử đầu tiên lặp lại trong mảng.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào số phần tử của mảng N ; dòng tiếp theo là N số $A[i]$ là các phần tử của mảng $A[]$.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^6$, $1 \leq A[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
2	1
5	20
4 5 1 2 1	
6	
10 20 30 30 20 5 7	

32. K Closest to X. Cho mảng $A[]$ gồm N phần tử đã được sắp xếp. Hãy tìm K phần tử gần nhất của X . Ví dụ với mảng $A[] = \{1, 3, 5, 7, 9, 11\}$, $X = 7$, $K=2$ thì ta có 2 phần tử gần nhất của 7 là 5 và 9. Chú ý: X có thể có mặt hoặc không có mặt trong mảng $A[]$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào số phần tử của mảng N, K, X; dòng tiếp theo là N số $A[i]$ là các phần tử của mảng A[].
- T, N, K, X, $A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N, K, X \leq 10^6$, $1 \leq A[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	23 75
5	
11 23 24 75 89	
2 24	

33. Common Elements. Cho mảng A[], B[], C[] gồm N1, N2, N3 phần tử đã được sắp xếp. Hãy đưa ra các phần tử có mặt trong cả ba mảng theo thứ tự tăng dần.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm bốn dòng: dòng thứ nhất đưa vào N1, N2, N3 là số phần tử của mảng A[], B[], C[]; các dòng tiếp theo đưa vào N1 số $A[i]$, N2 số $B[j]$, N3 số $C[k]$.
- T, N1, N2, N3, $A[i]$, $B[j]$, $C[k]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N1, N2, N3 \leq 10^6$, $0 \leq A[i], B[j], C[k] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	20 80
6 5 8	
1 5 10 20 40 80	
6 7 20 80 100	
3 4 15 20 30 70 80 120	

34. Kth Smallest Element. Cho mảng A[] gồm N phần tử. Hãy đưa ra các phần tử nhỏ nhất thứ k của mảng.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào N, k là số phần tử của mảng A[] và số k; các dòng tiếp theo đưa vào N số $A[i]$.
- T, N, $A[i]$, k thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq k < N \leq 10^6$, $0 \leq A[i] \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
--------	---------

1 6 3 10 5 15 50 40 80	15
------------------------------	----

PTT

CHƯƠNG 8. MÔ HÌNH THUẬT TOÁN DỰA VÀO NGĂN XẾP

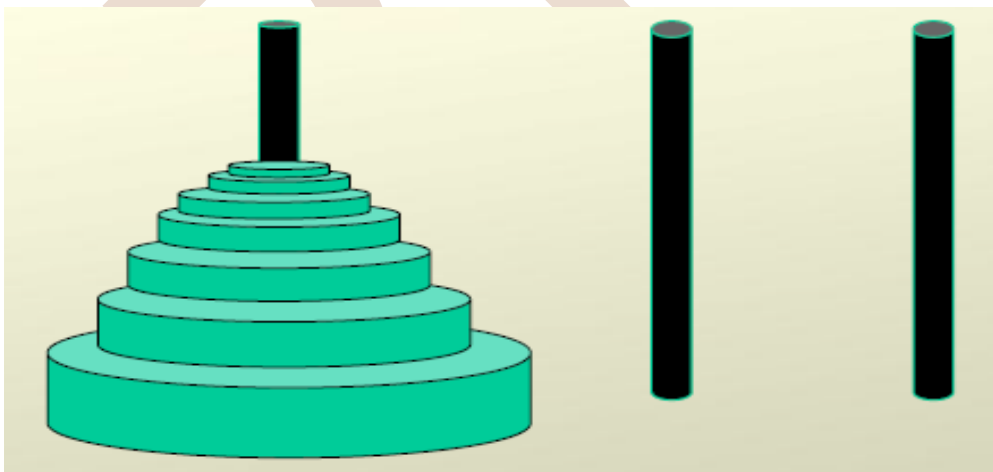
Ngăn xếp (stack) là cấu trúc dữ liệu được định nghĩa theo kiểu xếp chồng hoạt động theo nguyên tắc vào trước ra sau (LIFO: first in last out). Ngăn xếp được sử dụng trong việc xây dựng các giải thuật đệ qui, khử bỏ các giải thuật đệ qui, biểu diễn tính toán, duyệt cây và duyệt đồ thị. Ngoài ra, ứng dụng rất quan trọng đối với ngăn xếp là giảm độ phức tạp thuật toán cho một lớp rất rộng các bài toán thực tế. Trong chương này, chúng tôi sẽ trình bày các kỹ thuật cơ bản và nâng cao để giải quyết các bài toán dựa vào ngăn xếp.

8.1. Giới thiệu ngăn xếp (stack)

Ngăn xếp được hiểu như một cái ngăn để xếp. Ngăn xếp được ứng dụng trong nhiều lĩnh vực quan trọng khác nhau của khoa học máy tính. Trong mục này, ta sẽ xem xét phương pháp xây dựng kiểu dữ liệu ngăn xếp và ứng dụng của ngăn xếp.

Định nghĩa ngăn xếp: Tập hợp các node thông tin được tổ chức liên tục hoặc rời rạc nhau trong bộ nhớ và thực hiện theo cơ chế vào trước ra sau FILO (First – In – Last – Out).

Ta có thể hình dung stack như chồng đĩa trong bài toán “Tháp Hà Nội”. Đĩa có đường kính lớn nhất được xếp trước nhất, đĩa có đường kính nhỏ nhất được xếp sau cùng vào chồng đĩa. Tuy nhiên, khi lấy ra các đĩa có đường kính nhỏ nhất được lấy ra trước nhất, đĩa có đường lớn nhất sẽ được lấy ra sau cùng.



Hình 2.10. Mô tả ngăn xếp

Biểu diễn ngăn xếp:

- **Biểu diễn liên tục:** các phần tử dữ liệu của ngăn xếp được lưu trữ liên tục nhau trong bộ nhớ. Trong trường hợp này ta sử dụng mảng để biểu diễn các node dữ liệu.

- **Biểu diễn rời rạc:** các phần tử dữ liệu của ngăn xếp được lưu trữ rời rạc nhau trong bộ nhớ. Trong trường hợp này, ta sử dụng danh sách liên kết để biểu diễn các node dữ liệu (Danh sách liên kết).

Độ phức tạp của các thao tác trên ngăn xếp:

STT	Thao tác	Ý nghĩa	Độ phức tạp
1	empty()	Kiểm tra tính rỗng của ngăn xếp	O(1)
2	size()	Lấy kích cỡ của ngăn xếp	O(1)
3	top()	Lấy phần tử trên cùng của ngăn xếp	O(1)
4	push(g)	Đưa phần tử g vào ngăn xếp	O(1)
5	pop()	Loại bỏ phần tử khỏi ngăn xếp	O(1)

Ví dụ về các thao tác trên ngăn xếp:

Ví dụ:

```
#include <iostream>
#include <stack>
using namespace std;
int main(void){
    stack <int> s; //khai báo stack thích nghi với tùy biến int
    for(int i=1; i<=5; i++) s.push(i*10); //đưa i*10 vào stack
    cout<<"Kích cỡ stack:"<<s.size()<<endl; // s.size() = 10
    while(!s.empty()){ //lặp đến khi stack rỗng
        int t = s.top();cout<<t<<" "; //lấy phần tử đầu stack
        s.pop(); //đưa phần tử đầu tiên ra khỏi stack
    }
}
```

Hình 1: push(1*10), push(2*10), push(3*10), push(4*10), push(5*10)

The diagram illustrates the sequence of stack operations. It shows a series of five stacks connected by arrows labeled 'pop()'. The first stack contains elements 10, 20, 30, 40, and 50. The second stack contains 10, 20, and 30. The third stack contains 10 and 20. The fourth stack contains 10. The fifth stack is labeled 'Empty()'.

Ứng dụng của ngăn xếp: ngăn xếp có nhiều ứng dụng khác nhau trong khoa học máy tính. Các ứng dụng điển hình của ngăn xếp bao gồm:

- Xây dựng các giải thuật đệ qui: mọi giải thuật đệ qui đều thực hiện theo cơ chế của ngăn xếp.
- Khử bỏ các giải thuật đệ qui: muốn khử bỏ phép đệ qui hầu hết ta cần dùng đến ngăn xếp.

- Biểu diễn tính toán: biểu diễn tính toán trên máy tính được thực hiện dựa vào ngăn xếp.
- Duyệt cây, duyệt đồ thị: các phép duyệt cây, duyệt theo chiều sâu trên đồ thị đều có thể thực hiện dựa vào ngăn xếp.
- Giảm độ phức tạp tính toán: với một lớp rất lớn các bài toán, sử dụng ngăn xếp có thể giảm được độ phức tạp tính toán.

8.2. Một số bài toán cơ bản dựa vào ngăn xếp

Dưới đây là một số bài toán cơ bản dựa vào ngăn xếp:

1. **Stack 1.** Cho biểu thức exp độ dài n chứa đựng một số ký tự ‘(’, ‘)’. Hãy in ra số thứ tự của các cặp ‘(’, ‘)’ trong biểu thức. Ví dụ với biểu thức $\text{exp} = (a + (b * c)) + (d/e)$ ta có thứ tự của các cặp ‘(’, ‘)’ là 1 2 2 1 3 3.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một biểu thức số học được đưa vào trên một dòng.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
2	1 2 2 1 3 3
$(a + (b * c)) + (d/e)$	1 2 3 3 2 4 5 5 4 1
$((()))((()))$	

2. **Stack 2.** Cho biểu thức số học bất kỳ. Nhiệm vụ của bạn là xác định độ dài lớn nhất của các cặp đóng mở ngoặc đúng lồng nhau. Ví dụ với biểu thức $P = “(((X)) (((Y))))”$ ta có độ dài các cặp đóng mở ngoặc lồng nhau đúng là 4. Nếu biểu thức không đúng hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test là một biểu thức số học được đưa vào trên một dòng.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
2	4

$((X))((Y)))$ (b) ((c))	-1
-----------------------------	----

3. **Stack 3.** Có ba dạng biểu diễn cho các biểu thức số học và logic:

Infix (trung tố): Biểu diễn biểu thức dưới dạng trung tố là phép biểu diễn biểu thức trong đó phép toán được đặt giữa hai toán hạng. Ví dụ $(A+B) * (C-D)$.

Prefix (tiền tố): Biểu diễn biểu thức dưới dạng tiền tố là phép biểu diễn biểu thức trong đó phép toán được đặt trước hai toán hạng. Ví dụ $*+AB-CD$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Postfix (hậu tố): Biểu diễn biểu thức dưới dạng hậu tố là phép biểu diễn biểu thức trong đó phép toán được đặt sau hai toán hạng. Ví dụ $AB+CD-*$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng tiền tố về dạng trung tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
2 *+AB-CD *-A/BC-/AKL	$((A+B)*(C-D))$ $((A-(B/C))*((A/K)-L))$

4. **Stack 4.** Có ba dạng biểu diễn cho các biểu thức số học và logic:

Infix (trung tố): Biểu diễn biểu thức dưới dạng trung tố là phép biểu diễn biểu thức trong đó phép toán được đặt giữa hai toán hạng. Ví dụ $(A+B) * (C-D)$.

Prefix (tiền tố): Biểu diễn biểu thức dưới dạng tiền tố là phép biểu diễn biểu thức trong đó phép toán được đặt trước hai toán hạng. Ví dụ $*+AB-CD$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Postfix (hậu tố): Biểu diễn biểu thức dưới dạng hậu tố là phép biểu diễn biểu thức trong đó phép toán được đặt sau hai toán hạng. Ví dụ $AB+CD-*$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng tiền tố về dạng hậu tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;

- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
2 *+AB-CD *-A/BC-/AKL	AB+CD-* ABC/-AK/L-*

5. **Stack 5.** Có ba dạng biểu diễn cho các biểu thức số học và logic:

Infix (trung tố): Biểu diễn biểu thức dưới dạng trung tố là phép biểu diễn biểu thức trong đó phép toán được đặt giữa hai toán hạng. Ví dụ $(A+B) * (C-D)$.

Prefix (tiền tố): Biểu diễn biểu thức dưới dạng tiền tố là phép biểu diễn biểu thức trong đó phép toán được đặt trước hai toán hạng. Ví dụ *+AB-CD (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Postfix (hậu tố): Biểu diễn biểu thức dưới dạng hậu tố là phép biểu diễn biểu thức trong đó phép toán được đặt sau hai toán hạng. Ví dụ AB+CD-* (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng hậu tố về dạng tiền tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
2 AB+CD-* ABC/-AK/L-*	*+AB-CD *-A/BC-/AKL

6. **Stack 6.** Có ba dạng biểu diễn cho các biểu thức số học và logic:

Infix (trung tố): Biểu diễn biểu thức dưới dạng trung tố là phép biểu diễn biểu thức trong đó phép toán được đặt giữa hai toán hạng. Ví dụ $(A+B) * (C-D)$.

Prefix (tiền tố): Biểu diễn biểu thức dưới dạng tiền tố là phép biểu diễn biểu thức trong đó phép toán được đặt trước hai toán hạng. Ví dụ $*+AB-CD$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Postfix (hậu tố): Biểu diễn biểu thức dưới dạng hậu tố là phép biểu diễn biểu thức trong đó phép toán được đặt sau hai toán hạng. Ví dụ $AB+CD-*$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng hậu tố về dạng trung tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp .
- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(exp) \leq 10^6$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(exp) \leq 10^6$.

Input examples	Output examples
2	$(A+(B+C))$
ABC++	$((A*B)+C)$
AB*C+	

8.3. Một số bài toán nâng cao dựa vào ngăn xếp

Dưới đây là một số bài toán nâng cao dựa vào ngăn xếp:

7. **Stack 7.** Có ba dạng biểu diễn cho các biểu thức số học và logic:

Infix (trung tố): Biểu diễn biểu thức dưới dạng trung tố là phép biểu diễn biểu thức trong đó phép toán được đặt giữa hai toán hạng. Ví dụ $(A+B) * (C-D)$.

Prefix (tiền tố): Biểu diễn biểu thức dưới dạng tiền tố là phép biểu diễn biểu thức trong đó phép toán được đặt trước hai toán hạng. Ví dụ $*+AB-CD$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Postfix (hậu tố): Biểu diễn biểu thức dưới dạng hậu tố là phép biểu diễn biểu thức trong đó phép toán được đặt sau hai toán hạng. Ví dụ $AB+CD-*$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Hãy viết chương trình chuyển đổi biểu thức biểu diễn dưới dạng trung tố về dạng hậu tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp .

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
2 (A+(B+C)) ((A*B)+C)	ABC++ AB*C+

8. **Stack 8.** Có ba dạng biểu diễn cho các biểu thức số học và logic:

Infix (trung tố): Biểu diễn biểu thức dưới dạng trung tố là phép biểu diễn biểu thức trong đó phép toán được đặt giữa hai toán hạng. Ví dụ $(A+B) * (C-D)$.

Prefix (tiền tố): Biểu diễn biểu thức dưới dạng tiền tố là phép biểu diễn biểu thức trong đó phép toán được đặt trước hai toán hạng. Ví dụ $*+AB-CD$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Postfix (hậu tố): Biểu diễn biểu thức dưới dạng hậu tố là phép biểu diễn biểu thức trong đó phép toán được đặt sau hai toán hạng. Ví dụ $AB+CD-*$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Hãy viết chương trình chuyển tính toán giá trị của biểu thức tiền tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
2 - + 8 / 6 3 2 - + 7 * 4 5 + 2	8 25

9. **Stack 9.** Có ba dạng biểu diễn cho các biểu thức số học và logic:

Infix (trung tố): Biểu diễn biểu thức dưới dạng trung tố là phép biểu diễn biểu thức trong đó phép toán được đặt giữa hai toán hạng. Ví dụ $(A+B) * (C-D)$.

Prefix (tiền tố): Biểu diễn biểu thức dưới dạng tiền tố là phép biểu diễn biểu thức trong đó phép toán được đặt trước hai toán hạng. Ví dụ $*+AB-CD$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Postfix (hậu tố): Biểu diễn biểu thức dưới dạng hậu tố là phép biểu diễn biểu thức trong đó phép toán được đặt sau hai toán hạng. Ví dụ $AB+CD-*$ (tương ứng với biểu thức trung tố $(A+B)*(C-D)$).

Hãy viết chương trình chuyển tính toán giá trị của biểu thức hậu tố.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
2	-4
2 3 1 * + 9 -	34
8 7 5 * + 9 -	

10. **Stack 10.** Cho biểu thức số học, hãy cho biết biểu thức số học có dư thừa các cặp ký hiệu ‘(,’) ‘hay không?

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức tiền tố exp.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{exp}) \leq 10^6$.

Input examples	Output examples
3	Yes
((a+b))	Yes
(a + (b)/c)	No
(a + b*(c-d))	

11. **Stack 11.** Cho một chuỗi ký tự str bao gồm nhiều từ trong chuỗi. Hãy đảo ngược từng từ trong chuỗi?

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một dòng ghi lại nhiều từ trong chuỗi str.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, str thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(\text{str}) \leq 10^6$.

Input examples	Output examples
2	CBA FED
ABC DEF	321 654

123 456	
---------	--

8.4. Một số bài toán tiên tiến dựa vào ngăn xếp

Dưới đây là một số bài toán khó dựa vào ngăn xếp:

12. **Stack 12.** Cho biểu thức được tạo bởi các cặp “(,)”, “[,]”, “{, }”. Hãy cho biết có hay không các cặp “(,)”, “[,]”, “{, }” viết đúng hay không đúng trong biểu. Ví dụ biểu thức $exp = "[()]\{\{[()]\}\}"$ là biểu thức đúng (true), biểu thức $exp = "[()]"$ là sai (false).

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức exp .

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, exp thỏa mãn ràng buộc: $1 \leq T \leq 100$; $2 \leq \text{lenght}(exp) \leq 10^6$.

Input examples	Output examples
2	CBA FED
ABC DEF	321 654
123 456	

13. **Stack 13.** Ta gọi $NGE(i)$ của một mảng $A[]$ là phần tử lớn hơn $A[i]$ đầu tiên bên phải $A[i]$; $NGE(i) = -1$ nếu i là phần tử cuối cùng của mảng hoặc bên phải $A[i]$ không có phần tử nào lớn hơn $A[i]$. Cho mảng $A[]$ gồm n phần tử, hãy in ra $NGE(i)$ của mỗi phần tử với độ phức tạp thời gian $O(n)$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng, dòng đầu tiên đưa vào số n là số các phần tử của mảng $A[]$; dòng tiếp theo đưa vào n số của mảng $A[]$; các phần tử được viết cách nhau một vài khoảng trống.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $0 \leq n, A[i] \leq 10^6$.

Input examples	Output examples
1	-1 12 12 -1
4	
13 7 6 12	

14. **Stack 14.** Cho mảng $A[]$ gồm n phần tử. Nhiệm vụ của bạn là tìm giá trị gần nhất phía bên phải có số lần xuất hiện lớn hơn phần tử hiện tại. Nếu không có phần tử nào

có số lần xuất hiện lớn hơn phần tử hiện tại hãy đưa ra -1. Ví dụ với dãy $A[] = \{1, 1, 2, 3, 4, 2, 1\}$, ta nhận được kết quả $ans[] = \{-1, -1, 1, 2, 2, 1, -1\}$ vì số lần xuất hiện mỗi phần tử trong mảng là $F = \{3, 3, 2, 1, 1, 2, 3\}$ vì vậy phần tử $A[0] = 1$ có số lần xuất hiện là 3 và không có phần tử nào xuất hiện nhiều hơn 3 nên $ans[0] = -1$, tương tự như vậy với $A[2]=2$ tồn tại $A[6]=1$ có số lần xuất hiện là 3 nên $ans[2] = A[6]=1$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng, dòng đầu tiên đưa vào số n là số các phần tử của mảng $A[]$; dòng tiếp theo đưa vào n số của mảng $A[]$; các phần tử được viết cách nhau một vài khoảng trống.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- $T, n, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $0 \leq n, A[i] \leq 10^6$.

Input examples	Output examples
1	-1 -1 1 2 2 1 -1
7	
1 1 2 3 4 2 1	

15. **Stack 15.** Cho mảng $A[]$ gồm n phần tử tập các câu hỏi Q . Mỗi câu hỏi là một số k dùng để biểu diễn cho câu hỏi “Có nhiều nhất bao nhiêu phần tử lớn hơn $A[k]$ tăng dần về bên phải. Ví dụ với mảng $A[] = \{3, 4, 2, 7, 5, 8, 10, 6\}$ và $Q = \{0, 5\}$ ta có kết quả là $\{4, 1\}$ vì $A[0] = 3$ có 4 phần tử lớn hơn $A[0]$ là $\{4, 7, 8, 10\}$. Với $A[5] = 5$ chỉ có 1 phần tử là $\{10\}$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm ba dòng, dòng đầu tiên đưa vào số n là số các phần tử của mảng $A[]$; dòng tiếp theo đưa vào n số của mảng $A[]$; dòng cuối cùng đưa vào các câu hỏi Q ; các phần tử được viết cách nhau một vài khoảng trống.

Output:

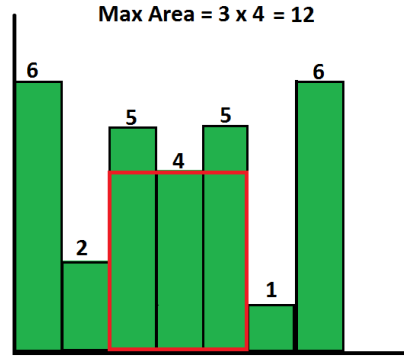
- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- $T, n, A[i], Q_i$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $0 \leq n, A[i] \leq 10^6$; $0 < Q_i < n$.

Input examples	Output examples
1	4 1
8	
3 4 2 7 5 8 10 6	
0 5	

16. **Stack 16.** Cho lược đồ bao gồm các hình chữ nhật có chiều dài khác nhau, có chiều rộng giống nhau là 1 được biểu diễn như các phần tử của mảng $A[]$. Nhiệm vụ của bạn là hãy tìm hình chữ nhật có diện tích lớn nhất được tạo bởi lược đồ. Ví dụ là có lược đồ gồm 7 hình chữ nhật với độ dài $\{6, 2, 5, 4, 5, 1, 6\}$ như dưới đây ta sẽ tìm ra hình chữ nhật lớn nhất là 12.



Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng, dòng đầu tiên đưa vào số n là số các phần tử của mảng $A[]$; dòng tiếp theo đưa vào n số là độ dài tương ứng của các hình chữ nhật trong mảng $A[]$; các phần tử được viết cách nhau một vài khoảng trống.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- $T, n, A[i], Q_i$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^6$; $1 \leq A[i] \leq 10^{18}$.

Input examples	Output examples
1 8 3 4 2 7 5 8 10 6 0 5	4 1

17. **Stack 17.** Cho mảng $A[]$ gồm n phần tử. Nhiệm vụ của bạn là tìm max của min cho tất cả các dãy con con liên tục có độ dài khác nhau từ 1, 2, ..., n . Ví dụ với mảng $A[] = \{10, 20, 30, 50, 10, 70, 30\}$ ta có kết quả là $\text{ans}[] = \{70, 30, 20, 10, 10, 10, 10\}$. Kết quả trên được thực hiện như sau:

- Max của min tất cả các dãy con độ dài 1 là 70.
- Max của min tất cả các dãy con độ dài 2 là $\min(10, 20, 30, 10, 10, 30) = 30$.
- Max của min tất cả các dãy con độ dài 3 là $\min(10, 20, 10, 10, 10) = 20$.
- Max của min tất cả các dãy con độ dài 4 là $\min(10, 20, 10, 10, 10) = 10$.
- Max của min tất cả các dãy con độ dài 5 là $\min(10, 20, 10, 10, 10) = 10$.
- Max của min tất cả các dãy con độ dài 6 là $\min(10, 20, 10, 10, 10) = 10$.
- Max của min tất cả các dãy con độ dài 7 là $\min(10, 20, 10, 10, 10) = 10$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng, dòng đầu tiên đưa vào số n là số các phần tử của mảng A[]; dòng tiếp theo đưa vào n số của mảng A[]; các phần tử được viết cách nhau một vài khoảng trống.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, n, A[i], Q_i thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n \leq 10^6$; $1 \leq A[i] \leq 10^{18}$.

Input examples	Output examples
2	70 30 20 10 10 10 10
7	30 20 10
10 20 30 50 10 70 30	
3	
10 20 30	

18. **Stack 18.** Cho biểu thức P chỉ bao gồm các ký tự mở ngoặc '(' hoặc đóng ngoặc ')'. Biểu thức P có thể viết đúng hoặc không đúng. Nhiệm vụ của bạn là tìm độ dài lớn nhất viết đúng của P. Độ dài của biểu thức đúng ngắn nhất là 2.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức P được viết trên một dòng.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, P thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(P) \leq 100$.

Input examples	Output examples
4	2
((	4
((	10
(((((	6
(((((	

19. **Stack 19.** Cho biểu thức P chỉ bao gồm các ký tự mở ngoặc '{' hoặc đóng ngoặc '}'. Biểu thức P có thể viết đúng hoặc không đúng. Nhiệm vụ của bạn là tìm số phép biến đổi ít nhất các ký tự '{' thành '}' và ngược lại để ta nhận được một biểu thức đúng. Ví dụ với P = "{{" ta cần một phép biến đổi để trở thành biểu thức đúng "}}". Nếu không thể biến đổi hãy đưa ra -1.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;

- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức P được viết trên một dòng.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, P thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(P) \leq 100$.

Input examples	Output examples
4 }{}{}{} {}}} {}}{}{}{} {{{}}}	3 1 -1 0

20. **Stack 20.** Cho P1, P2 là hai biểu thức đúng chỉ bao gồm các ký tự mở ngoặc '(' hoặc đóng ngoặc ')' và các toán hạng in thường. Nhiệm vụ của bạn là định xem P1 và P2 có giống nhau hay không.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào P1, dòng tiếp theo đưa vào P2.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, P thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(P) \leq 100$.

Input examples	Output examples
2 -(a+b+c) -a-b-c a-b-(c-d) a-b-c-d	YES NO

21. **Stack 21.** Cho mảng A[] chỉ bao gồm các ký tự I hoặc D. Ký tự I được hiểu là tăng (Increasing) ký tự D được hiểu là giảm (Decreasing). Sử dụng các số từ 1 đến 9, hãy đưa ra số nhỏ nhất được đoán nhận từ mảng A[]. Chú ý, các số không được phép lặp lại. Dưới đây là một số ví dụ mẫu:

- A[] = "I" : số tăng nhỏ nhất là 12.
- A[] = "D" : số giảm nhỏ nhất là 21
- A[] = "DD" : số giảm nhỏ nhất là 321
- A[] = "DDIDDIID": số thỏa mãn 321654798

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.

- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test là một xâu ID
- T, Length(A) thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{Length}(A) \leq 9$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
4	12
I	21
D	321
DD	321654798
DDIDDIID	

22. **Stack 22.** Cho mảng A[] gồm n phần tử. Hãy đưa ra các phần tử nhỏ hơn tiếp theo của phần tử lớn hơn đầu tiên phần tử hiện tại. Nếu phần tử hiện tại không có phần tử lớn hơn tiếp theo ta xem là -1. Nếu phần tử không có phần tử nhỏ hơn tiếp theo ta cũng xem là -1. Ví dụ với mảng A[] = {5, 1, 9, 2, 5, 1, 7} ta có kết quả là ans = {2, 2, -1, 1, -1, -1, -1} vì:

Next Greater	Right Smaller
5 -> 9	9 -> 2
1 -> 9	9 -> 2
9 -> -1	-1 -> -1
2 -> 5	5 -> 1
5 -> 7	7 -> -1
1 -> 7	7 -> -1
7 -> -1	7 -> -1

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test gồm hai dòng: dòng thứ nhất đưa vào n là số phần tử của mảng A[], dòng tiếp theo đưa vào n số A[i].

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, n, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq n$, $A[i] \leq 10^3$.

Input examples	Output examples
2	2 2 1 1 -1 -1 -1
7	2 5 5 5 -1 3 -1 -1
5 1 9 2 5 1 7	
7	
4 8 2 1 9 5 6 3	

23. **Stack 23.** Cho xâu ký tự mã hóa str. Hãy viết chương trình giải mã xâu ký tự str. Xâu ký tự mã hóa được thực hiện theo số lần lặp các xâu con của str như sau:

Xâu đầu vào: “abbbababbbababbbab ”

Xâu mã hóa : "3[a3[b]1[ab]]"

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một xâu mã hóa str được viết trên một dòng.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, str thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(\text{str}) \leq 10^3$.

Input examples	Output examples
2	b
1[b]	bcacabcacabcaca
3[b2[ca]]	

24. **Stack 24.** Cho biểu thức đúng P chỉ bao gồm các phép toán +, -, các toán hạng cùng với các ký tự ‘(’, ‘)’. Hãy bỏ tất cả các ký tự ‘(’, ‘)’ trong P để nhận được biểu thức tương đương. Ví dụ với $P = a - (b + c)$ ta có kết quả $P = a - b - c$.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T;
- Những dòng tiếp theo mỗi dòng đưa vào một bộ test. Mỗi bộ test là một biểu thức P được viết trên một dòng.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Ràng buộc:

- T, P thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq \text{length}(P) \leq 10^3$.

Input examples	Output examples
2	a- b - c
a - (b + c)	a- b + c
(a - (b - c)	

CHƯƠNG 9. MÔ HÌNH THUẬT TOÁN DỰA VÀO HÀNG ĐỢI

Hàng đợi (queue) là cấu trúc dữ liệu được thực hiện theo nguyên tắc vào trước ra trước (First In First Out). Hàng đợi được chia thành nhiều loại khác nhau: hàng đợi tuyến tính, hàng đợi ưu tiên, hàng đợi vòng, hàng đợi hai điểm cuối. Hàng đợi được ứng dụng rộng rãi trong biểu diễn tính toán, duyệt cây, duyệt đồ thị. Ngoài ra, ứng dụng rất quan trọng đối với ngăn xếp là giảm độ phức tạp thuật toán cho một lớp rất rộng các bài toán thực tế. Trong chương này, chúng tôi sẽ trình bày các kỹ thuật cơ bản và nâng cao để giải quyết các bài toán dựa vào hàng đợi.

9.1. Giới thiệu hàng đợi (queue)

Hàng đợi được hiểu là một hàng để đợi. Hàng đợi trong máy tính cũng giống như hàng đợi trong thực tế: hàng đợi mua vé tàu, vé xe, vé máy bay. Hàng đợi ứng dụng trong nhiều lĩnh vực khác nhau của khoa học máy tính. Trong mục này ta sẽ xem xét phương pháp xây dựng hàng đợi cùng với ứng dụng của hàng đợi.

Định nghĩa hàng đợi: Hàng đợi (queue) là tập các node thông tin được tổ chức liên tục hoặc rời rạc nhau trong bộ nhớ và thực hiện theo cơ chế FIFO (First-In-First-Out).

Hàng đợi hai điểm cuối (double ended queue): hàng đợi được xây dựng theo nguyên tắc phép đưa phần tử vào và lấy phần tử ra khỏi hàng đợi được thực hiện ở hai điểm cuối.

Biểu diễn hàng đợi: Có hai phương pháp biểu diễn hàng đợi: biểu diễn liên tục và biểu diễn rời rạc. Trong trường hợp biểu diễn liên tục ta sử dụng mảng, trong trường hợp biểu diễn rời rạc ta sử dụng danh sách liên kết.

Hàng đợi tuyến tính(sequential queue): hàng đợi được xây dựng theo nguyên tắc FIFO. Các thao tác trên hàng đợi bao gồm:

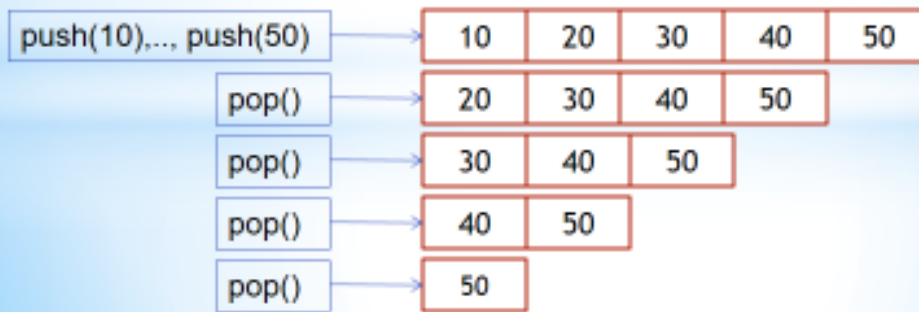
STT	Thao tác	Ý nghĩa	Độ phức tạp
1	empty()	Kiểm tra tính rỗng của hàng đợi	$O(1)$
2	size()	Lấy kích cỡ hàng đợi	$O(1)$
3	swap()	Đổi nội dung hai phần tử của hàng đợi	$O(1)$
4	emplace()	Đặt phần tử vào đầu hàng đợi	$O(1)$
5	front()	Lấy phần tử vào trước trong hàng đợi	$O(1)$
6	back()	Lấy phần tử vào sau trong hàng đợi	$O(1)$
7	push()	Đưa phần tử vào hàng đợi	$O(1)$
8	pop()	Loại bỏ phần tử vào trước của hàng đợi	$O(1)$

Ví dụ về các thao tác trên hàng đợi:**Ví dụ:**

```

#include <iostream>
#include <queue>
using namespace std;
int main(void){ queue <int> q;
    for(int i=1; i<=5; i++) { q.push(i*10); }
    cout<<"Kích cỡ hàng đợi:"<<q.size()<<endl;
    cout<<"Phần tử đầu tiên:"<<q.front()<<endl;
    cout<<"Phần tử cuối cùng:"<<q.back()<<endl;
    cout<<"Trạng thái hàng đợi:"<<q.empty()<<endl;
    cout<<"Loại bỏ phần tử:";
    while(!q.empty()){
        int t = q.front();cout<<t<<" "; q.pop();
    }
}

```



Hàng đợi ưu tiên (priority queue): hàng đợi được xây dựng theo nguyên tắc phép đưa phần tử vào hàng đợi được xếp ứng với thứ tự ưu tiên của nó.

Các thao tác trên hàng đợi ưu tiên:

STT	Thao tác	Ý nghĩa	Độ phức tạp
1	empty()	Kiểm tra tính rỗng của hàng đợi	O(1)
2	size()	Lấy kích cỡ hàng đợi	O(1)
3	swap()	Đổi nội dung hai phần tử của hàng đợi	O(1)
4	emplace()	Đặt phần tử vào đầu hàng đợi	O(1)
5	top()	Lấy phần tử vào trước trong hàng đợi	O(1)
6	push()	Đưa phần tử vào hàng đợi theo thứ tự ưu tiên.	O(1)
7	pop()	Loại bỏ phần tử vào trước của hàng đợi	O(1)

9.2. Một số bài toán cơ bản dựa vào hàng đợi

Dưới đây là một số bài toán cơ bản dựa vào hàng đợi:

1. **Queue 1.** Cho chuỗi ký tự $S[]$ bao gồm các ký tự in hoa $[A, B, \dots, Z]$. Ta định nghĩa giá trị của chuỗi $S[]$ là tổng bình phương số lần xuất hiện mỗi ký tự trong chuỗi. Ví dụ với chuỗi $S[] = \text{"AAABBCD"}$ ta có $F(S) = 3^2 + 2^2 + 1^2 + 1^2 = 15$. Hãy tìm giá trị nhỏ nhất của chuỗi $S[]$ sau khi loại bỏ K ký tự trong chuỗi.

Input:

- Dòng đầu tiên đưa vào số lượng test T ($T \leq 100$).
- Mỗi test được tổ chức thành 2 dòng. Dòng thứ nhất ghi lại số K . Dòng thứ 2 ghi lại chuỗi ký tự $S[]$.

Output:

- Đưa ra giá trị nhỏ nhất của mỗi test theo từng dòng.

Input	Output
2	6
2	3
ABCC	
1	
ABCC	

2. **Queue 2.** Cho số tự nhiên n . Hãy in ra tất cả các số nhị phân từ 1 đến n .

Input:

- Dòng đầu tiên ghi lại số lượng test T ($T \leq 100$).
- Mỗi test là một số tự nhiên n được ghi trên một dòng ($n \leq 500$).

Output:

- Đưa ra kết quả mỗi test trên một dòng.

Input	Output
2	1 10
2	1 10 11 100 101 110
5	

3. **Queue 3.** Cho số tự nhiên N . Hãy tìm số nguyên dương X nhỏ nhất được tạo bởi số 9 và số 0 chia hết cho N . Ví dụ với $N = 5$ ta sẽ tìm ra $X = 90$.

Input:

- Dòng đầu tiên ghi lại số lượng test T ($T \leq 100$).
- Những dòng kế tiếp mỗi dòng ghi lại một test. Mỗi test là một số tự nhiên N được ghi trên một dòng ($N \leq 100$).

Output:

- Đưa ra theo từng dòng số X nhỏ nhất chia hết cho N tìm được.

Input	Output
2	90
5	9009
7	

4. **Queue 4.** Ta gọi số nguyên dương K là một số BDN nếu các chữ số trong K chỉ bao gồm các 0 hoặc 1 có nghĩa. Ví dụ số $K = 1, 10, 101$. Cho số tự nhiên N ($N < 2^{63}$). Hãy cho biết có bao nhiêu số BDN nhỏ hơn N . Ví dụ $N=100$ ta có 4 số BDN bao gồm các số: 1, 10, 11, 100.

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng Test;
- T dòng kế tiếp mỗi dòng ghi lại một bộ Test. Mỗi test là một số tự nhiên N .

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
3	2
10	4
100	7
200	

5. **Queue 5.** Ta gọi số nguyên dương K là một số BDN nếu các chữ số trong K chỉ bao gồm các 0 hoặc 1 có nghĩa. Ví dụ số $K = 101$ là số BDN, $k=102$ không phải là số BDN. Số BDN của N là số $P = M \times N$ sao cho P là số BDN. Cho số tự nhiên N ($N < 1000$), hãy tìm số BDN nhỏ nhất của N .

Ví dụ. Với $N=2$, ta tìm được số BDN của N là $P = 5 \times 2 = 10$. $N = 17$ ta tìm được số BDN của 17 là $P = 653 \times 17 = 11101$.

Dữ liệu vào cho bởi file data.in theo khuôn dạng:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng Test;
- T dòng kế tiếp mỗi dòng ghi lại một bộ Test. Mỗi test là một số tự nhiên N .

Kết quả thực hiện của mỗi test được ghi lại trong file ketqua.out theo từng dòng. Mỗi dòng ghi lại số các số BDN của mỗi test. Ví dụ dưới đây sẽ minh họa cho file data.in và ketqua.out của bài toán.

Input	Output
3	10
2	11100
12	11101
17	

9.3. Một số bài toán nâng cao dựa vào hàng đợi

Dưới đây là một số bài toán nâng cao dựa vào hàng đợi:

6. **Queue 6.** Cho hai số nguyên dương S và T ($S, T < 10000$) và hai thao tác (a), (b) dưới đây:

Thao tác (a): Trừ S đi 1 ($S = S - 1$);

Thao tác (b): Nhân S với 2 ($S = S * 2$);

Hãy dịch chuyển S thành T sao cho số lần thực hiện các thao tác (a), (b) là ít nhất. Ví dụ với $S=2, T=5$ thì số các bước ít nhất để dịch chuyển S thành T thông qua 4 thao tác sau:

Thao tác (a): $2*2 = 4$;

Thao tác (b): $4-1 = 3$;

Thao tác (a): $3*2 = 6$;

Thao tác (b): $6-1 = 5$;

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng Test;
- T dòng kế tiếp mỗi dòng ghi lại một bộ Test. Mỗi test là một bộ đôi S và T.

Output: Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
3	4
2 5	4
3 7	3
7 4	

7. **Queue** 7. Cho số tự nhiên N ($N < 2^{63}$) và hai phép biến đổi (a), (b) dưới đây.

Thao tác (a): Trừ N đi 1 ($N=N-1$). Ví dụ $N=17$, thao tác (a) biến đổi $N = N-1 = 16$.

Thao tác (b): $N = \max(u,v)$ nếu $u*v = N$ ($u>1, v>1$). Ví dụ $N=16$, thao tác (b) có thể biến đổi $N = \max(2, 8)=8$ hoặc $N=\max(4, 4)=4$.

Chỉ được phép sử dụng hai thao tác (a) hoặc (b), hãy biến đổi N thành 1 sao cho số các thao tác (a), (b) được thực hiện ít nhất. Ví dụ với $N=17$, số các phép (a), (b) nhỏ nhất biến đổi N thành 1 là 4 bước như sau:

Thao tác (a): $N = N-1 = 17-1 = 16$

Thao tác (b): $16 = \max(4,4) = 4$

Thao tác (b): $4 = \max(2,2) = 2$

Thao tác (a): $2 = 2-1 = 1$

Input:

- Dòng đầu tiên ghi lại số tự nhiên T là số lượng Test;
- T dòng kế tiếp mỗi dòng ghi lại một bộ Test. Mỗi test là một số N.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
3	4
17	5
50	5
100	

8. **Queue 8.** Cho cặp số S và T là các số nguyên tố có 4 chữ số (Ví dụ S = 1033, T = 8197 là các số nguyên tố có 4 chữ số). Hãy viết chương trình tìm cách dịch chuyển S thành T thỏa mãn đồng thời những điều kiện dưới đây:

- Mỗi phép dịch chuyển chỉ được phép thay đổi một chữ số của số ở bước trước đó (ví dụ nếu S=1033 thì phép dịch chuyển S thành 1733 là hợp lệ);
- Số nhận được cũng là một số nguyên tố có 4 chữ số (ví dụ nếu S=1033 thì phép dịch chuyển S thành 1833 là không hợp lệ, và S dịch chuyển thành 1733 là hợp lệ);
- Số các bước dịch chuyển là ít nhất.

Ví dụ số các phép dịch chuyển ít nhất để S = 1033 thành T = 8179 là 6 bao gồm các phép dịch chuyển như sau: 8179<- 8779<- 3779<- 3739<- 3733<- 1733<- 1033.

Input:

- Dòng đầu tiên đưa vào số lượng test T ($T \leq 100$).
- Những dòng kế tiếp mỗi dòng đưa vào một test. Mỗi test là một bộ đôi S, T.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	6
1033 8179	5
1033 8779	

9. **Queue 9.** Cho tập n xâu ký tự S và hai xâu s, t ∈ S. Ta giả thiết các xâu ký tự S[i] ∈ S có độ dài bằng nhau. Hãy tìm khoảng cách đường đi ngắn nhất từ s đến t. Biết từ một xâu ký tự bất kỳ ta chỉ được phép dịch chuyển đến xâu khác với nó duy nhất 1 ký tự. Ví dụ ta có tập các từ S = { POON, TOON, PLEE, SAME, POIE, PLEA, PLIE, POIN }, s = TOON, t = PLEA ta có độ dài đường đi ngắn nhất là 7 tương ứng với các phép dịch chuyển : TOON → POON → POIN → POIE → PLIE → PLEE → PLEA.

Input:

- Dòng đầu tiên đưa vào số lượng test T ($T \leq 100$).
- Mỗi test được tổ chức thành 2 dòng. Dòng thứ nhất ghi lại n là số từ trong S và hai từ s, t. Dòng thứ 2 đưa vào n xâu ký tự của S; các xâu ký tự được viết cách nhau một vài khoảng trống.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
1	7
8 TOON PLEA POON TOON PLEE SAME POIE PLEA PLIE POIN	

10. **Queue 10.** Cho hai số nguyên dương L, R . Nhiệm vụ của bạn là hãy đưa ra số các số K trong khoảng $[L, R]$ thỏa mãn điều kiện:

- Tất cả các chữ số của K đều khác nhau.
- Tất cả các chữ số của K đều nhỏ hơn hoặc bằng 5.

Ví dụ với $L = 4, R = 13$ ta có 5 số thỏa mãn yêu cầu là 4, 5, 10, 12, 13,

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Dòng tiếp theo đưa vào các bộ test. Mỗi bộ test được là một cặp L, R được viết trên một dòng.
- T, L, R thỏa mãn ràng buộc: $1 \leq T \leq 100; 0 \leq L \leq R \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	5
4 13	100
100 1000	

9.4. Một số bài toán tiên tiến dựa vào hàng đợi

Dưới đây là một số bài toán khó dựa vào hàng đợi:

11. **Queue 11.** Cho hai số nguyên dương L, R . Nhiệm vụ của bạn là hãy đưa ra số các số K trong khoảng $[L, R]$ thỏa mãn điều kiện:

- Tất cả các chữ số của K đều khác nhau.
- Tất cả các chữ số của K đều nhỏ hơn hoặc bằng 5.

Ví dụ với $L = 4, R = 13$ ta có 5 số thỏa mãn yêu cầu là 4, 5, 10, 12, 13,

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Dòng tiếp theo đưa vào các bộ test. Mỗi bộ test được là một cặp L, R được viết trên một dòng.
- T, L, R thỏa mãn ràng buộc: $1 \leq T \leq 100; 0 \leq L \leq R \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	5
4 13	100
100 1000	

12. **Queue 12.** Cho số nguyên N . Nhiệm vụ của bạn hãy đưa ra N số nguyên thủy. Số K được gọi là số nguyên thủy nếu số đó thỏa mãn điều kiện:

- Số các chữ số của K là một số chẵn.
- Tất cả các chữ số của K chỉ bao gồm số 4 hoặc 5.
- K là một số đối xứng.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Dòng tiếp theo đưa vào các bộ test. Mỗi bộ test được là một N được viết trên một dòng.
- T, N thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^5$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	44 55 4444 5445
4	44 55 4444 4554 5445 5555 444444 454454 544445 554455
10	

13. **Queue 13.** Cho ma trận $A[M][N]$. Nhiệm vụ của bạn hãy tìm số bước đi ít nhất dịch chuyển từ vị trí $A[0][0]$ đến vị trí $A[M-1][N-1]$. Biết mỗi bước đi ta chỉ được phép dịch chuyển đến vị trí $A[i][j+A[i][j]]$ hoặc vị trí $A[i][j+A[i][j]]$ bên trong ma trận.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm hai phần: phần thứ nhất là hai số M, N; phần thứ hai là các phần tử của ma trận $A[i][j]$; các số được viết cách nhau một vài khoảng trống.
- T, M, N, $A[i][j]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq M, N, A[i][j] \leq 10^3$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
1	2
3 3	
2 1 2	
1 1 1	
1 1 1	

14. Cho cặp số S và T là các số nguyên tố có 4 chữ số (Ví dụ $S = 1033$, $T = 8197$ là các số nguyên tố có 4 chữ số). Hãy viết chương trình tìm cách dịch chuyển S thành T thỏa mãn đồng thời những điều kiện dưới đây:

- Mỗi phép dịch chuyển chỉ được phép thay đổi một chữ số của số ở bước trước đó (ví dụ nếu $S=1033$ thì phép dịch chuyển S thành 1733 là hợp lệ);
- Số nhận được cũng là một số nguyên tố có 4 chữ số (ví dụ nếu $S=1033$ thì phép dịch chuyển S thành 1833 là không hợp lệ, và S dịch chuyển thành 1733 là hợp lệ);

c) Số các bước dịch chuyển là ít nhất.

Ví dụ số các phép dịch chuyển ít nhất để $S = 1033$ thành $T = 8179$ là 6 bao gồm các phép dịch chuyển như sau: $8179 < -8779 < -3779 < -3739 < -3733 < -1733 < -1033$.

Input:

- Dòng đầu tiên đưa vào số lượng test T ($T \leq 100$)
- Những dòng kế tiếp mỗi dòng đưa vào một test. Mỗi test là một bộ đôi S, T .

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input	Output
2	6
1033 8179	5
1033 8779	

CHƯƠNG 10. MÔ HÌNH THUẬT TOÁN DỰA VÀO CÂY NHỊ PHÂN

Như đã được đề cập ở trên, hạn chế lớn nhất của mảng là luôn đòi hỏi một không gian nhớ liên tục. Điều này sẽ gặp phải khó khăn khi xử lý các đối tượng dữ liệu lớn. Danh sách liên kết không đòi hỏi một không gian nhớ liên tục nhưng gặp phải vấn đề trong tìm kiếm. Trong chương này ta sẽ xem xét một cấu trúc dữ liệu rời rạc đó là cây nhị phân. Các node trên cây được tổ chức và truy cập không theo thứ tự. Cây nhị phân cho phép ta tổ chức và xử lý được các ứng dụng có dữ liệu rất lớn. Tìm kiếm trên cây nhị phân không nhanh như tìm kiếm trên mảng nhưng tốt hơn rất nhiều so với danh sách liên kết.

10.1. Giới thiệu cây nhị phân (binary tree)

Định nghĩa: tập hợp hữu hạn các node có cùng kiểu dữ liệu (có thể là tập $\emptyset$) được phân thành 3 tập:

- Tập thứ nhất có thể là $\emptyset$ hoặc chỉ có một node gọi là node gốc (root).
- Hai tập con còn lại tự hình thành hai cây con bên trái (left subtree) và cây con bên phải (right subtree) của node gốc (hai tập con này cũng có thể là tập $\emptyset$).

Một số tính chất của cây nhị phân:

- Số lượng lớn nhất các node ở mức h là 2^{h-1} . Ví dụ, số lượng mức của node gốc là $2^{1-1} = 1$, mức 2 là $2^{2-1} = 2 \dots$
- Số lượng node lớn nhất của cây nhị phân có chiều cao h là $2^h - 1$. Điều này có thể suy luận trực tiếp từ tính chất 1 vì $N \leq 2^0 + 2^1 + \dots + 2^{h-1} = 2^h - 1$.
- Đối với cây nhị phân có N node thì chiều cao tối thiểu hay mức tối thiểu của cây là $\lceil \log_2(N + 1) \rceil$.
- Một cây nhị phân có L node lá có chiều cao tối đa là $\lceil \log_2(L) \rceil + 1$.
- Trong cây nhị phân số lượng các node lá luôn nhiều hơn các node có hai node con 1 đơn vị.

Một số loại cây nhị phân đặc biệt:

- **Cây lệch trái:** cây nhị phân chỉ có node con bên trái.
- **Cây lệch phải:** cây chỉ có node con bên phải.
- **Cây nhị phân đúng (strictly binary tree):** node gốc và tất cả các node trung gian đều có đúng hai node con (Hình 4.2).
- **Cây nhị phân đầy đủ (complete binary tree):** cây nhị phân đúng và tất cả node lá đều có mức là d (Hình 4.3).
- **Cây nhị phân gần đầy (almost complete binary tree):** Cây nhị phân gần đầy có chiều sâu d là cây nhị phân thỏa mãn (Hình 4.4):
 - Tất cả node con có mức không nhỏ hơn $d-1$ đều có hai node con (cây nhị phân đầy đủ mức $d-1$).

- Các node ở mức d đầy từ trái qua phải.
- **Cây nhị phân hoàn toàn cân bằng.** Cây nhị phân có số node thuộc nhánh cây con trái và số node thuộc nhánh cây con phải chênh lệch nhau không quá 1 (Hình 4.5).
- **Cây nhị phân tìm kiếm.** Cây nhị phân thỏa mãn điều kiện (Hình 4.6):
 - Hoặc là rỗng hoặc có một node gốc.
 - Mỗi node gốc có tối đa hai cây con. Nội dung node gốc lớn hơn nội dung node con bên trái và nhỏ hơn nội dung node con bên phải.
 - Hai cây con bên trái và bên phải cũng hình thành nên hai cây tìm kiếm.
- **Cây nhị phân tìm kiếm hoàn toàn cân bằng.** Cây nhị phân tìm kiếm có chiều sâu cây con bên trái và chiều sâu cây con bên phải chênh lệch nhau không quá 1

Biểu diễn cây nhị phân: có hai phương pháp biểu diễn cây nhị phân: biểu diễn liên tục và biểu diễn rời rạc. Trong trường hợp biểu diễn liên tục ta sử dụng mảng. Trong trường hợp biểu diễn rời rạc ta sử dụng danh sách liên kết. Phương pháp biểu diễn được thực hiện như dưới đây.

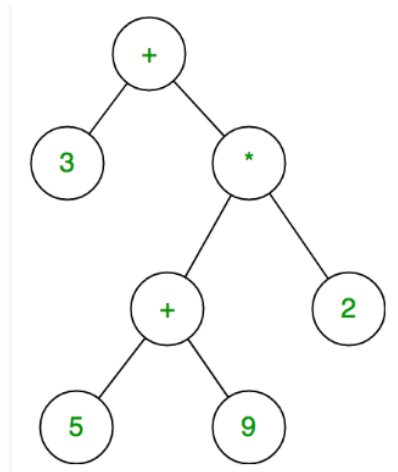
Các thao tác trên cây nhị phân:

- Tạo node cho cây.
- Tạo node gốc cho cây.
- Thêm vào node lá bên trái node p.
- Thêm vào node lá bên phải node p.
- Loại bỏ node lá bên trái node p.
- Loại bỏ node lá bên phải node p.
- Loại bỏ cả cây.
- Duyệt cây theo thứ tự trước.
- Duyệt cây theo thứ tự giữa.
- Duyệt cây theo thứ tự sau.

10.2. Một số bài toán cơ bản dựa vào cây nhị phân

Dưới đây là một số bài toán cơ bản dựa vào cây nhị phân:

1. Cây biểu thức là một cây nhị phân trong đó mỗi node trung gian là một phép toán, mỗi node lá là một toán hạng. Ví dụ với biểu thức $P = 3 + ((5+9)*2)$ sẽ được biểu diễn như cây dưới đây. Đối với cây biểu thức, duyệt theo thứ tự trước ta sẽ được biểu thức trung tố, duyệt theo thứ tự sau ta sẽ được biểu thức hậu tố, duyệt theo thứ tự giữa ta được biểu thức tiền tố. Chú ý, cây biểu thức luôn là cây nhị phân đầy (mỗi node trung gian đều có hai node con).



Cho biểu thức hậu tố P, hãy sử dụng cây biểu thức để đưa ra biểu thức trung tố tương ứng với biểu thức P.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test là một biểu thức hậu tố P.
- T, P thỏa mãn ràng buộc : $1 \leq T \leq 100$; $1 \leq \text{lenght}(P) \leq 100$.

Output:

- Đưa ra biểu thức trung tố tương ứng với P.

Input examples	Output examples
2 ab+ef*g* wlr b+-*	a + b - e * f * g w * l - r + b

- Cho một cây biểu thức là một cây nhị phân đầy đủ bao gồm các phép toán +, -, *, / và một số toán hạng có giá trị nguyên. Nhiệm vụ của bạn là hãy tính toán giá trị biểu thức được biểu diễn trên cây nhị phân đầy đủ. Ví dụ với cây dưới đây là biểu diễn của biểu thức $P = ((5*4) + (100-20))$ sẽ cho ta giá trị là 100.

Input:

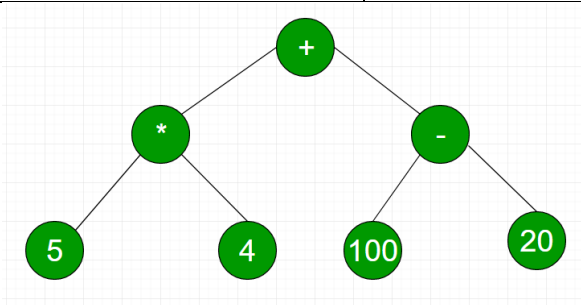
- Dòng đầu tiên đưa vào số lượng bộ test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test là gồm hai dòng: dòng thứ nhất đưa vào N là số lượng node của cây; dòng thứ hai đưa vào nội dung các node của cây; các node được viết cách nhau một vài khoảng trống.
- T, N, P thỏa mãn ràng buộc : $1 \leq T \leq 100$; $1 \leq N$, $\text{lenght}(P) \leq 100$.

Output:

- Đưa ra giá trị của cây biểu thức.

Input examples	Output examples
----------------	-----------------

2	100
7	-3
+ * - 5 4 100 20	
3	
- 4 7	



3. Cho phép duyệt cây nhị phân Inoder và Preoder, hãy đưa ra kết quả phép duyệt Postoder của cây nhị phân. Ví dụ với cây nhị phân dưới đây ta có các phép duyệt cây nhị phân của cây dưới đây:

Inoder : 4 2 5 1 3 6

Preoder: : 1 2 4 5 3 6

Postode : 4 5 2 6 3 1

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng node; dòng tiếp theo đưa vào N số theo phép duyệt Inoder; dòng cuối cùng đưa vào N số là kết quả của phép duyệt Preoder; các số được viết cách nhau một vài khoảng trống.
- T, N, node thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 1000$; $1 \leq \text{node} \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt Postoder theo từng dòng.

Input examples	Output examples
1	4 5 2 6 3 1
4 2 5 1 3 6	
1 2 4 5 3 6	

4. Cho mảng A[] gồm N node là biểu diễn phép duyệt theo thứ tự giữa (Preoder) của cây nhị phân tìm kiếm. Nhiệm vụ của bạn là đưa ra phép duyệt theo thứ tự sau của cây nhị phân tìm kiếm.

Input:

- Dòng đầu tiên đưa vào số lượng test T.

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng node; dòng tiếp theo đưa vào N số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- T, N , node thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq A[i] \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt Postoder theo từng dòng.

Input examples	Output examples
2	35 30 100 80 40
5	35 32 30 120 100 90 80 40
40 30 35 80 100	
8	
40 30 32 35 80 90 100 120	

5. Cho hai mảng là phép duyệt Inoder và Preoder, nhiệm vụ của bạn là xây dựng cây nhị phân và đưa ra kết quả phép duyệt Postoder.

Input:

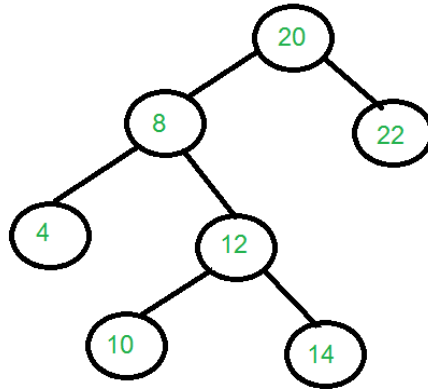
- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng node; dòng tiếp theo đưa vào N số là phép duyệt Inoder; dòng cuối cùng đưa vào N số là phép duyệt Preoder; các số được viết cách nhau một vài khoảng trống.
- T, N , node thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq A[i] \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt Postoder theo từng dòng.

Input examples	Output examples
1	3 4 1 5 6 2 0
7	
3 1 4 0 5 2 6	
0 1 3 4 2 5 6	

6. Cho cây nhị phân, nhiệm vụ của bạn là duyệt cây theo Level-oder. Phép duyệt level-oder trên cây là phép thăm node theo từng mức của cây. Ví dụ với cây dưới đây sẽ cho ta kết quả của phép duyệt level-oder: 20 8 22 4 12 10 14.



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

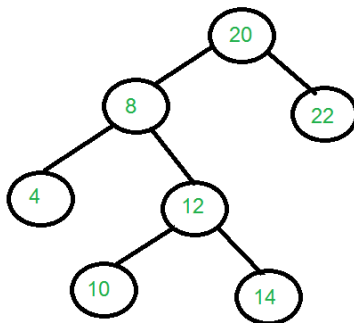
- Đưa ra kết quả phép duyệt level-oder theo từng dòng.

Input examples	Output examples
2	1 3 2
2	10 20 30 40 60
1 2 R 1 3 L	
4	
10 20 L 10 30 R 20 40 L 20 60 R	

7. Cho hai mảng là phép duyệt Inoder và Level-oder, nhiệm vụ của bạn là xây dựng cây nhị phân và đưa ra kết quả phép duyệt Postoder. Level-oder là phép duyệt theo từng mức của cây. Ví dụ như cây dưới đây ta có phép Inoder và Level-oder như dưới đây:

Inoder : 4 8 10 12 14 20 22

Level oder: 20 8 22 4 12 10 14



Input:

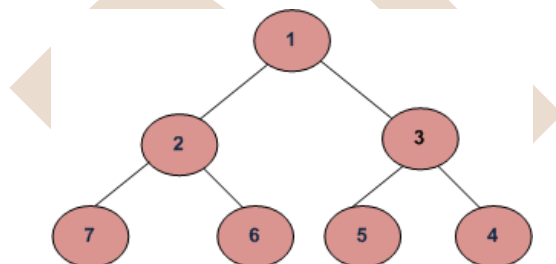
- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng node; dòng tiếp theo đưa vào N số là phép duyệt Inoder; dòng cuối cùng đưa vào N số là phép duyệt Level-oder; các số được viết cách nhau một vài khoảng trống.
- T, N, node thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq A[i] \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt Postoder theo từng dòng.

Input examples	Output examples
2	0 1 2
3	0 1 3 4 2 5 6
1 0 2	
0 1 2	
7	
3 1 4 0 5 2 6	
0 1 2 3 4 5 6	

8. Cho cây nhị phân, nhiệm vụ của bạn là duyệt cây theo xoắn ốc (spiral-oder). Phép. Ví dụ với cây dưới đây sẽ cho ta kết quả của phép duyệt spiral-oder: 1 2 3 4 5 6 7.



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

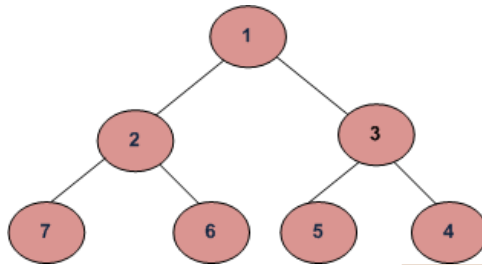
Output:

- Đưa ra kết quả phép duyệt level-oder theo từng dòng.

Input examples	Output examples
2	1 3 2
2	10 20 30 60 40

1 2 R 1 3 L 4 10 20 L 10 30 R 20 40 L 20 60 R	
---	--

9. Cho cây nhị phân, nhiệm vụ của bạn là duyệt cây theo mức đảo ngược. Với cây dưới đây sẽ cho ta kết quả của phép duyệt đảo ngược theo mức là : 7 6 5 4 3 2 1.



Input:

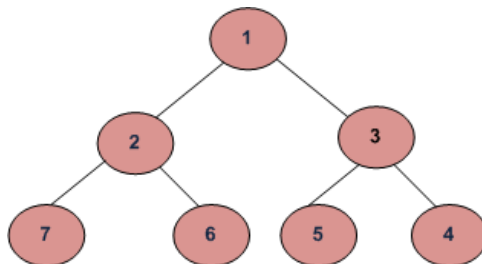
- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả phép duyệt reverse-level-order theo từng dòng.

Input examples	Output examples
2 2 1 2 R 1 3 L 4 10 20 L 10 30 R 20 40 L 20 60 R	3 2 1 40 60 20 30 10

10. Cho cây nhị phân, nhiệm vụ của bạn là kiểm tra xem tất cả các node lá của cây có cùng một mức hay không? Ví dụ với cây dưới đây sẽ cho ta kết quả là Yes.



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

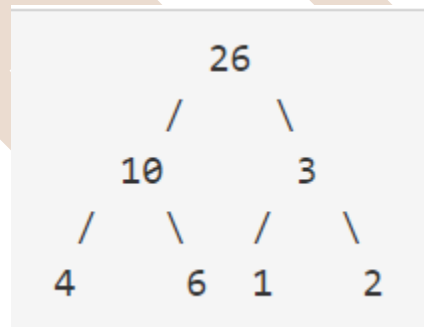
- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	1
1 2 R 1 3 L	0
4	
10 20 L 10 30 R 20 40 L 20 60 R	

10.3. Một số bài toán nâng cao dựa vào vào cây nhị phân

Dưới đây là một số bài toán nâng cao dựa vào cây nhị phân:

11. Cho cây nhị phân, nhiệm vụ của bạn là kiểm tra xem cây nhị phân có phải là một cây tổng hay không? Một cây nhị phân được gọi là cây tổng nếu tổng các node con của node trung gian bằng giá trị node cha. Node không có node con trái hoặc phải được hiểu là có giá trị 0. Ví dụ dưới đây là một cây tổng



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	1
2	0
3 1 L 3 2 R	
4	
10 20 L 10 30 R 20 40 L 20 60 R	

12. Cho cây nhị phân, nhiệm vụ của bạn là kiểm tra xem cây nhị phân có phải là một cây hoàn hảo hay không (perfect tree)? Một cây nhị phân được gọi là cây hoàn hảo nếu tất cả các node trung gian của nó đều có hai node con và tất cả các node lá đều có cùng một mức.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
3	Yes
6	Yes
10 20 L 10 30 R 20 40 L 20 50 R 30 60 L 30 70 R	No
2	
18 15 L 18 30 R	
5	
1 2 L 2 4 R 1 3 R 3 5 L 3 6 R	

13. Cho cây nhị phân, nhiệm vụ của bạn là kiểm tra xem cây nhị phân có phải là một cây đầy đủ hay không (full binary tree)? Một cây nhị phân được gọi là cây đầy đủ nếu tất cả các node trung gian của nó đều có hai node con.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u,

v, x), trong đó u là node cha, v là node con, $x=R$ nếu v là con phải, $x=L$ nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.

- T, N, u, v , thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	1
4	0
1 2 L 1 3 R 2 4 L 2 5 R	
3	
1 2 L 1 3 R 2 4	

14. Cho cây nhị phân, nhiệm vụ của bạn là kiểm tra xem cây nhị phân có phải là một cây đầy đủ hay không (full binary tree)? Một cây nhị phân được gọi là cây đầy đủ nếu tất cả các node trung gian của nó đều có hai node con.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, $x=R$ nếu v là con phải, $x=L$ nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v , thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	1
4	0
1 2 L 1 3 R 2 4 L 2 5 R	
3	
1 2 L 1 3 R 2 4	

15. Cho hai cây nhị phân, nhiệm vụ của bạn là kiểm tra xem cây nhị phân có giống nhau hay không?

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, $x=R$ nếu v là con phải, $x=L$ nếu v là con trái của mỗi cây; u, v, x được viết cách nhau một vài khoảng trống.

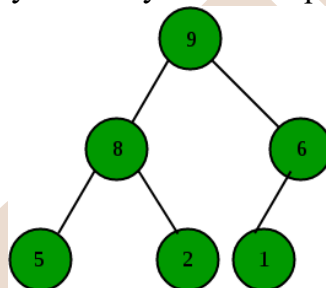
- T, N, u, v , thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	1
2	0
1 2 L 1 3 R	
2	
1 2 L 1 3 R	
2	
1 2 L 1 3 R	
2	
1 3 L 1 2 R	

16. Cho cây nhị phân, nhiệm vụ của bạn là tính tổng của tất cả các node lá bên trái trên cây? Ví dụ với cây dưới đây ta có kết quả là 6.



Input:

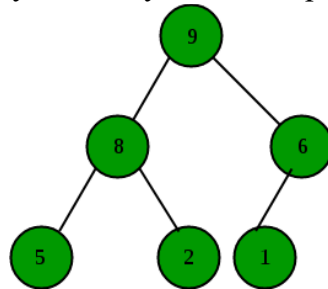
- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x) , trong đó u là node cha, v là node con, $x = R$ nếu v là con phải, $x = L$ nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v , thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	2
2	130
1 2 L 1 3 R	
5	
10 20 L 10 30 R 20 40 L 20 60 R 30 90 L	

17. Cho cây nhị phân, nhiệm vụ của bạn là tính tổng của tất cả các node lá bên phải trên cây? Ví dụ với cây dưới đây ta có kết quả là 2.



Input:

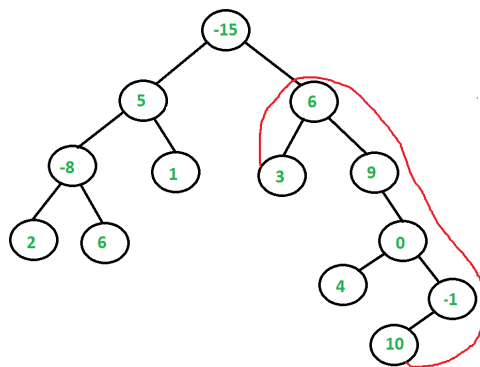
- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
2	3
2	60
1 2 L 1 3 R	
5	
10 20 L 10 30 R 20 40 L 20 60 R 30 90 L	

18. Cho cây nhị phân có giá trị mỗi node là một số, nhiệm vụ của bạn là tìm tổng lớn nhất từ một node lá này sang một node lá khác? Ví dụ với cây dưới đây ta có tổng lớn nhất là 27.



Input:

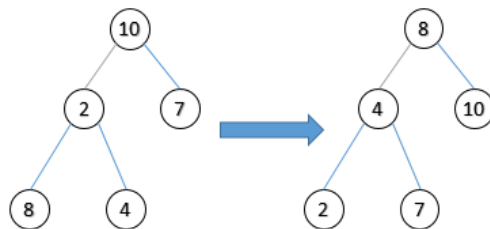
- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
1 12 -15 5 L -15 6 R 5 -8 L 5 1 R -8 2 L -8 -3 R 6 3 L 6 9 R 9 0 R 0 4 L 0 -1 R -1 10 L	27

19. Cho cây nhị phân, nhiệm vụ của bạn là dịch chuyển cây nhị phân thành cây nhị phân tìm kiếm. Phép dịch chuyển phải bảo toàn được cấu trúc cây nhị phân ban đầu. Ví dụ dưới đây sẽ minh họa phép dịch chuyển:



Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 3 dòng: dòng đầu tiên đưa vào số N là số lượng cạnh của cây; dòng tiếp theo đưa vào N bộ ba (u, v, x), trong đó u là node cha, v là node con, x= R nếu v là con phải, x=L nếu v là con trái; u, v, x được viết cách nhau một vài khoảng trống.
- T, N, u, v, thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq u, v \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng là phép duyệt Inoder của cây tìm kiếm.

Input examples	Output examples
2 2 1 2 R 1 3 L 4 10 20 L 10 30 R 20 40 L 20 60 R	1 2 3 10 20 30 40 60

20. Cho một mảng là phép duyệt level-oder của cây nhị phân tìm kiếm. Nhiệm vụ của bạn là xây dựng lại cây nhị phân tìm kiếm. toàn được cấu trúc cây nhị phân ban đầu.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm dòng: dòng đầu tiên đưa vào số N là số lượng node của cây tìm kiếm; dòng tiếp theo đưa vào phép duyệt level-oder của cây tìm kiếm; các số được viết cách nhau một vài khoảng trống.
- T, N, node thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq \text{node} \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng là phép duyệt Inoder của cây tìm kiếm.

Input examples	Output examples
2	7 4 3 1 6 5 12 8 10
9	1 3 4 6 7 8
7 4 12 3 6 8 1 5 10	
6	
1 3 4 6 7 8	

10.4. Một số bài toán tiên tiến dựa vào vào cây nhị phân

Dưới đây là một số bài toán tiên tiến dựa vào cây nhị phân:

21. Cho một mảng A[] gồm N phần tử biểu diễn phép duyệt preoder của cây nhị phân tìm kiếm. Nhiệm vụ của bạn là đưa ra phép duyệt postoder của cây nhị phân tìm kiếm.

Input:

- Dòng đầu tiên đưa vào số lượng test T.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng node của cây tìm kiếm; dòng tiếp theo đưa vào phép duyệt preoder của cây tìm kiếm; các số được viết cách nhau một vài khoảng trống.
- T, N, A[i] thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq A[i] \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng là phép duyệt postoder của cây tìm kiếm.

Input examples	Output examples
3	35 30 100 80 40
5	35 32 30 120 100 90 80 40
40 30 35 80 100	
8	
40 30 32 35 80 90 100 120	

22. Cho một mảng $A[]$ gồm N phần tử. Nhiệm vụ của bạn là đưa ra 1 nếu mảng $A[]$ biểu diễn phép duyệt inorder của cây nhị phân tìm kiếm, trái lại đưa ra 0.

Input:

- Dòng đầu tiên đưa vào số lượng test T .
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm 2 dòng: dòng đầu tiên đưa vào số N là số lượng node của cây tìm kiếm; dòng tiếp theo đưa vào N số $A[i]$; các số được viết cách nhau một vài khoảng trống.
- $T, N, A[i]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N \leq 10^3$; $1 \leq A[i] \leq 10^4$;

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input examples	Output examples
3	1
5	0
10 20 30 40 50	0
6	
90 80 100 70 40 30	
3	
1 1 2	

23. Cho dãy số gồm N số là phép duyệt theo thứ tự trước (Preoder) của một cây nhị phân tìm kiếm. Hãy in ra tất cả các node lá của cây ?

Ví dụ với dãy $A[] = \{30, 20, 15, 25, 23, 28, 40, 35, 33, 38, 45\}$ là phép duyệt cây theo thứ tự trước sẽ cho ta kết quả: 15, 23, 28, 33, 38, 45.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ($T \leq 100$).
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất là số tự nhiên N ($N \leq 106$). Dòng tiếp theo là N số là phép duyệt theo thứ tự trước của cây BST.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output
2	1 7 50
6	15 23 28 33 38 45
10 5 1 7 40 50	
11	
30 20 15 25 23 28 40 35 33 38 45	

24. Cho dãy số gồm N số là phép duyệt theo thứ tự trước (Preoder) của một cây nhị phân tìm kiếm. Hãy đưa ra số các node trung gian của cây ?

Ví dụ với dãy $A[] = \{30, 20, 15, 25, 23, 28, 40, 35, 33, 38, 45\}$ là phép duyệt cây theo thứ tự trước sẽ cho ta kết quả là 5 bao gồm các node: 30, 20, 25, 40, 35.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ($T \leq 100$).
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất là số tự nhiên N ($N \leq 106$). Dòng tiếp theo là N số là phép duyệt theo thứ tự trước của cây BST.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output
2	3
6	5
10 5 1 7 40 50	
11	
30 20 15 25 23 28 40 35 33 38 45	

25. Cho dãy số gồm N số là phép duyệt theo thứ tự trước (Preoder) của một cây nhị phân tìm kiếm. Hãy tìm độ sâu của cây ?

Ví dụ với dãy $A[] = \{30, 20, 15, 25, 23, 28, 40, 35, 33, 38, 45\}$ là phép duyệt cây theo thứ tự trước sẽ cho ta kết quả là 3.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ($T \leq 100$).
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất là số tự nhiên N ($N \leq 106$). Dòng tiếp theo là N số là phép duyệt theo thứ tự trước của cây BST.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output
2	2
6	3
10 5 1 7 40 50	
11	
30 20 15 25 23 28 40 35 33 38 45	

26. Hãy xây dựng một cây nhị phân tìm kiếm cân bằng từ dãy số $A[] = \{a_0, a_1, \dots, a_{n-1}\}$. Đưa ra nội dung node gốc của cây tìm kiếm cân bằng. Ví dụ với dãy $A[] = \{40, 28, 45, 38, 33, 15, 25, 20, 23, 35, 30\}$ ta sẽ có cây nhị phân tìm kiếm cân bằng với node gốc là 33.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ($T \leq 100$).
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất là số tự nhiên N ($N \leq 106$). Dòng tiếp theo là N số của mảng $A[]$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output
2	33
11	4
40 28 45 38 33 15 25 20 23 35 30	
10	
1 2 3 4 5 6 7 8 9 10	

27. Hãy xây dựng một cây nhị phân tìm kiếm cân bằng từ dãy số $A[] = (a_0, a_1, \dots, a_{n-1})$. Đưa ra phép duyệt theo thứ tự trước của cây tìm kiếm cân bằng. Ví dụ với dãy $A[] = \{40, 28, 45, 38, 33, 15, 25, 20, 23, 35, 30\}$ ta sẽ có phép duyệt theo thứ tự trước của cây nhị phân tìm kiếm cân bằng với node gốc là 33 : 33, 25, 20, 15, 23, 28, 30, 40, 38, 35, 45.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ($T \leq 100$).
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất là số tự nhiên N ($N \leq 106$). Dòng tiếp theo là N số của mảng $A[]$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output
2	33 25 20 15 23 28 30 40 38 35 45
11	4 2 1 3 8 6 5 7 9 10
40 28 45 38 33 15 25 20 23 35 30	
10	
1 2 3 4 5 6 7 8 9 10	

28. Hãy xây dựng một cây nhị phân tìm kiếm cân bằng từ dãy số $A[] = (a_0, a_1, \dots, a_{n-1})$. Đưa ra phép duyệt theo thứ tự sau của cây tìm kiếm cân bằng. Ví dụ với dãy $A[] = \{40, 28, 45, 38, 33, 15, 25, 20, 23, 35, 30\}$ ta sẽ có phép duyệt theo thứ tự trước của cây nhị phân tìm kiếm cân bằng với node gốc là 33 : 15, 23, 20, 30, 28, 25, 35, 38, 45, 40, 33.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ($T \leq 100$).
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất là số tự nhiên N ($N \leq 106$). Dòng tiếp theo là N số của mảng $A[]$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output
2	15 23 20 30 28 25 35 38 45 40 33
11	1 3 2 5 7 6 10 9 8 4

40 28 45 38 33 15 25 20 23 35 30	
10	
1 2 3 4 5 6 7 8 9 10	

29. Hãy xây dựng một cây nhị phân tìm kiếm cân bằng từ dãy số $A[] = (a_0, a_1, \dots, a_{n-1})$. Đếm số node lá của cây nhị phân tìm kiếm cân bằng. Ví dụ với dãy $A[] = \{40, 28, 45, 38, 33, 15, 25, 20, 23, 35, 30\}$ ta sẽ có phép duyệt theo thứ tự trước của cây nhị phân tìm kiếm cân bằng với số node lá là 5 : 15, 23, 30, 35 45.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T ($T \leq 100$).
- Những dòng kế tiếp đưa vào T bộ test. Mỗi bộ test gồm 2 dòng: dòng thứ nhất là số tự nhiên N ($N \leq 106$). Dòng tiếp theo là N số của mảng $A[]$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output
2	5
11	5
40 28 45 38 33 15 25 20 23 35 30	
10	
1 2 3 4 5 6 7 8 9 10	

35. Common Elements. Cho mảng $A[], B[], C[]$ gồm N_1, N_2, N_3 phần tử đã được sắp xếp. Hãy đưa ra các phần tử có mặt trong cả ba mảng theo thứ tự tăng dần.

Input:

- Dòng đầu tiên đưa vào số lượng bộ test T .
- Những dòng kế tiếp đưa vào các bộ test. Mỗi bộ test gồm bốn dòng: dòng thứ nhất đưa vào N_1, N_2, N_3 là số phần tử của mảng $A[], B[], C[]$; các dòng tiếp theo đưa vào N_1 số $A[i]$, N_2 số $B[j]$, N_3 số $C[k]$.
- $T, N_1, N_2, N_3, A[i], B[j], C[k]$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq N_1, N_2, N_3 \leq 10^6$, $0 \leq A[i], B[j], C[k] \leq 10^{18}$.

Output:

- Đưa ra kết quả mỗi test theo từng dòng.

Input:	Output:
1	20 80
6 5 8	
1 5 10 20 40 80	
6 7 20 80 100	
3 4 15 20 30 70 80 120	

CHƯƠNG 11. MÔ HÌNH THUẬT TOÁN DỰA VÀO ĐỒ THỊ

Đồ thị là một cấu trúc dữ liệu rời rạc bao gồm các đỉnh và các cạnh nối các cặp đỉnh này. Với quan niệm như trên, một mạng máy tính được xem như một đồ thị có mỗi đỉnh là một máy tính, có các cạnh là một liên kết giữa các máy tính khác nhau trong mạng. Các mạch điện, các hệ thống giao thông, các mạng xã hội đều được xem xét như một đồ thị. Có thể nói đồ thị được ứng dụng rộng rãi trong nhiều lĩnh vực khác nhau của khoa học máy tính. Nội dung chính của chương này đề cập đến phương pháp biểu diễn và các thuật toán trên đồ thị.

11.1. Giới thiệu về đồ thị

Ta có thể phân chia đồ thị thành hai loại: đồ thị vô hướng (undirected graph) và đồ thị có hướng (directed graph). Mỗi loại đồ thị lại được chia thành 3 loại: đơn đồ thị, đa đồ thị và giả đồ thị. Mỗi loại đồ thị có các thuật ngữ chung và những khái niệm riêng. Dưới đây là một số thuật ngữ cơ bản trên các loại đồ thị.

Định nghĩa. Bộ đồ $G = \langle V, E \rangle$, trong đó $V = \{1, 2, \dots, n\}$ là tập hợp hữu hạn được gọi là tập đỉnh, E là tập có thứ tự hoặc không có thứ tự các cặp đỉnh trong V được gọi là tập cạnh.

Đồ thị vô hướng. Đồ thị $G = \langle V, E \rangle$ được gọi là đồ thị vô hướng nếu các cạnh thuộc E là các cặp không tính đến thứ tự các đỉnh trong V .

Đơn đồ thị vô hướng. Đồ thị $G = \langle V, E \rangle$ được gọi là đơn đồ thị vô hướng nếu G là đồ thị vô hướng và giữa hai đỉnh bất kỳ thuộc V có nhiều nhất một cạnh nối.

Đa đồ thị vô hướng. Đồ thị $G = \langle V, E \rangle$ được gọi là đa đồ thị vô hướng nếu là đồ thị vô hướng và tồn tại một cặp đỉnh trong V có nhiều hơn một cạnh nối. Cạnh $e_1 \in E$, $e_2 \in E$ được gọi là cạnh bội nếu chúng cùng chung cặp đỉnh.

Giả đồ thị vô hướng. Đồ thị $G = \langle V, E \rangle$ bao gồm V là tập đỉnh, E là họ các cặp không có thứ tự gồm hai phần tử (hai phần tử không nhất thiết phải khác nhau) trong V được gọi là các cạnh. Cạnh e được gọi là khuyên nếu có dạng $e = (u, u)$, trong đó u là đỉnh nào đó thuộc V .

Đơn đồ thị có hướng. Đồ thị $G = \langle V, E \rangle$ bao gồm V là tập các đỉnh, E là tập các cặp có thứ tự gồm hai phần tử của V gọi là các cung. Giữa hai đỉnh bất kỳ của G tồn tại nhiều nhất một cung.

Đa đồ thị có hướng. Đồ thị $G = \langle V, E \rangle$ bao gồm V là tập đỉnh, E là tập các cặp có thứ tự gồm hai phần tử của V được gọi là các cung. Hai cung e_1 , e_2 tương ứng với cùng một cặp đỉnh được gọi là cung lặp.

Giả đồ thị có hướng. Đa đồ thị $G = \langle V, E \rangle$, trong đó V là tập đỉnh, E là tập các cặp không có thứ tự gồm hai phần tử (hai phần tử không nhất thiết phải khác nhau) trong V

được gọi là các cung. Cung e được gọi là khuyên nếu có dạng $e=(u, u)$, trong đó u là đỉnh nào đó thuộc V .

11.1.1. Một số thuật ngữ trên đồ thị vô hướng

Đỉnh kề. Hai đỉnh u và v của đồ thị vô hướng $G = \langle V, E \rangle$ được gọi là kề nhau nếu (u, v) là cạnh thuộc đồ thị G . Nếu $e = (u, v)$ là cạnh của đồ thị G thì ta nói cạnh này liên thuộc với hai đỉnh u và v , hoặc ta nói cạnh e nối đỉnh u với đỉnh v , đồng thời các đỉnh u và v sẽ được gọi là đỉnh đầu của cạnh (u, v) .

Bậc của đỉnh. Ta gọi bậc của đỉnh v trong đồ thị vô hướng là số cạnh liên thuộc với nó và ký hiệu là $\deg(v)$. Đỉnh có bậc là 0 được gọi là đỉnh cô lập. Đỉnh có bậc 1 được gọi là đỉnh treo.

Đường đi, chu trình. Đường đi độ dài n từ đỉnh u đến đỉnh v trên đồ thị vô hướng $G = \langle V, E \rangle$ là dãy $x_0, x_1, \dots, x_{n-1}, x_n$, trong đó n là số nguyên dương, $x_0 = u, x_n = v, (x_i, x_{i+1}) \in E, i = 0, 1, 2, \dots, n-1$. Đường đi có đỉnh đầu trùng với đỉnh cuối gọi là chu trình.

Tính liên thông. Đồ thị vô hướng được gọi là liên thông nếu luôn tìm được đường đi giữa hai đỉnh bất kỳ của nó.

Thành phần liên thông. Đồ thị vô hướng liên thông thì số thành phần liên thông là 1. Đồ thị vô hướng không liên thông thì số liên thông của đồ thị là số các đồ thị con của nó liên thông.

Đỉnh trụ. Đỉnh $u \in V$ được gọi là đỉnh trụ nếu loại bỏ u cùng với các cạnh nối với u làm tăng thành phần liên thông của đồ thị.

Cạnh cầu. Cạnh $(u, v) \in E$ được gọi là cầu nếu loại bỏ (u, v) làm tăng thành phần liên thông của đồ thị.

Đỉnh rẽ nhánh. Đỉnh s được gọi là đỉnh rẽ nhánh (đỉnh thắt) của cặp đỉnh u, v nếu mọi đường đi từ u đến v đều qua s .

11.1.2. Một số thuật ngữ trên đồ thị có hướng

Đỉnh kề. Nếu $e = (u, v)$ là cung của đồ thị có hướng G thì ta nói hai đỉnh u và v là kề nhau, và nói cung (u, v) nối đỉnh u với đỉnh v , hoặc nói cung này đi ra khỏi đỉnh u và đi vào đỉnh v . Đỉnh u được gọi là đỉnh đầu, đỉnh v được gọi là đỉnh cuối của cung (u, v) .

Bán bậc của đỉnh. Ta gọi bán bậc ra của đỉnh v trên đồ thị có hướng là số cung của đồ thị đi ra khỏi v và ký hiệu là $\deg^+(v)$. Ta gọi bán bậc vào của đỉnh v trên đồ thị có hướng là số cung của đồ thị đi vào v và ký hiệu là $\deg^-(v)$.

Đường đi. Đường đi độ dài n từ đỉnh u đến đỉnh v trong đồ thị có hướng $G = \langle V, A \rangle$ là dãy $x_0, x_1, \dots, x_n$, trong đó, n là số nguyên dương, $u = x_0, v = x_n, (x_i, x_{i+1}) \in A$. Đường đi như trên có thể biểu diễn thành dãy các cung: $(x_0, x_1), (x_1, x_2), \dots, (x_{n-1}, x_n)$. Đỉnh u được gọi là đỉnh đầu, đỉnh v được gọi là đỉnh cuối của đường đi. Đường đi có đỉnh đầu trùng với

đỉnh cuối ($u=v$) được gọi là một chu trình. Đường đi hay chu trình được gọi là đơn nếu như không có hai cạnh nào lặp lại.

Liên thông mạnh. Đồ thị có hướng $G=\langle V,E \rangle$ được gọi là liên thông mạnh nếu giữa hai đỉnh bất kỳ $u \in V, v \in V$ đều có đường đi từ u đến v .

Liên thông yếu. Ta gọi đồ thị vô hướng tương ứng với đồ thị có hướng $G=\langle V,E \rangle$ là đồ thị tạo bởi G và bỏ hướng của các cạnh trong G . Khi đó, đồ thị có hướng $G=\langle V,E \rangle$ được gọi là liên thông yếu nếu đồ thị vô hướng tương ứng với nó là liên thông.

Thành phần liên thông mạnh. Đồ thị con có hướng $H=\langle V_1, E_1 \rangle$ được gọi là một thành phần liên thông mạnh của đồ thị có hướng $G=\langle V,E \rangle$ nếu $V_1 \subseteq V, E_1 \subseteq E$ và H liên thông mạnh.

11.1.3. Một số loại đồ thị đặc biệt

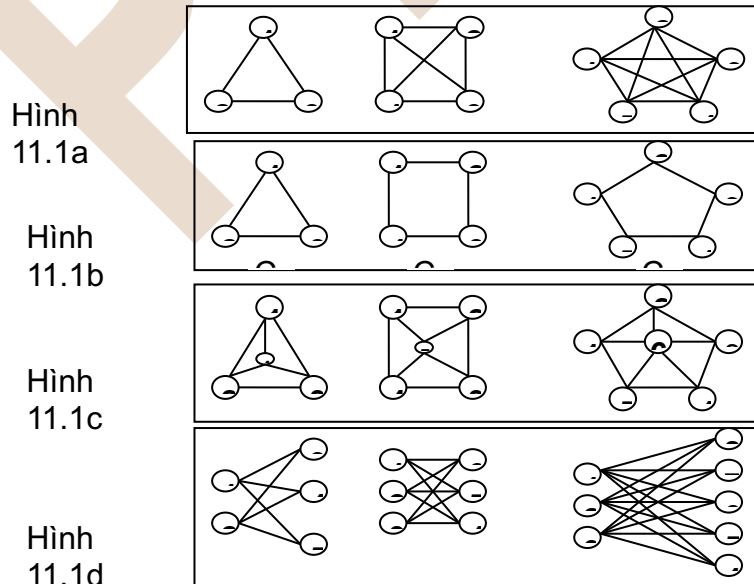
Dưới đây là một số dạng đơn đồ thị vô hướng đặc biệt có nhiều ứng dụng khác nhau của thực tế.

Đồ thị đầy đủ. Đồ thị đầy đủ n đỉnh, ký hiệu là K_n , là đơn đồ thị vô hướng mà giữa hai đỉnh bất kỳ của nó đều có cạnh nối. Ví dụ đồ thị K_3, K_4, K_5 trong Hình 11.1a.

Đồ thị vòng. Đồ thị vòng C_n ($n \geq 3$) có các cạnh $(1,2), (2,3), \dots, (n-1,n), (n,1)$. Ví dụ đồ thị C_3, C_4, C_5 trong Hình 11.1.b.

Đồ thị bánh xe. Đồ thị bánh xe W_n thu được bằng cách bổ sung một đỉnh nối với tất cả các đỉnh của C_n . Ví dụ đồ thị W_3, W_4, W_5 trong Hình 11.1.c.

Đồ thị hai phía. Đồ thị $G=\langle V,E \rangle$ được gọi là đồ thị hai phía nếu tập đỉnh V của nó có thể phân hoạch thành hai tập X và Y sao cho mỗi cạnh của đồ thị chỉ có dạng (x,y) , trong đó $x \in X$ và $y \in Y$. Ví dụ đồ thị $K_{2,3}, K_{3,3}, K_{3,5}$ trong Hình 11.1.d.



Hình 11.1. Một số dạng đồ thị đặc biệt.

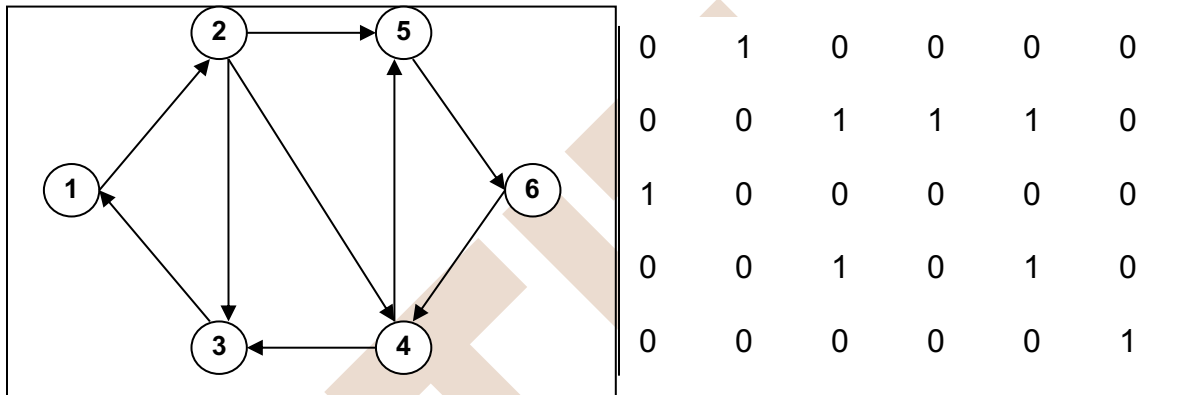
11.1.4. Biểu diễn đồ thị

Để lưu trữ, xử lý hiệu quả ta cần có phương pháp biểu diễn đồ thị trên máy tính. Ba phương pháp biểu diễn thông dụng thường được ứng dụng trên đồ thị đó là: ma trận kề, danh sách cạnh và danh sách kề. Phương pháp biểu diễn cụ thể được thể hiện như dưới đây.

Biểu diễn bằng ma trận kề: Phương pháp biểu diễn đồ thị bằng ma trận kề là phép làm tương ứng đồ thị $G = \langle V, E \rangle$ với một ma trận vuông cấp n . Các phần tử của ma trận kề được xác định như dưới đây.

$$a_{uv} = \begin{cases} 1 & \text{nếu } (u, v) \in E \\ 0 & \text{otherwise} \end{cases}$$

Ví dụ với đồ thị cho bởi Hình 4.2 sẽ cho ta biểu diễn ma trận kề như sau:



Hình 4.2. Biểu diễn ma trận kề của đồ thị.

Tính chất của ma trận kề:

- Ma trận kề biểu diễn đồ thị vô hướng $G = \langle V, E \rangle$ là ma trận đối xứng.
- Ma trận kề biểu diễn đồ thị có hướng $G = \langle V, E \rangle$ thường là không đối xứng.
- Tổng các phần tử của ma trận kề biểu diễn đồ thị vô hướng bằng $2.m$, trong đó m là số cạnh của đồ thị.
- Tổng các phần tử của ma trận kề biểu diễn đồ thị có hướng bằng đúng m , trong đó m là số cạnh của đồ thị.
- Tổng các phần tử của hàng u hoặc cột u của ma trận kề biểu diễn đồ thị vô hướng là bậc của đỉnh u ($\deg(u)$).
- Tổng các phần tử của hàng u của ma trận kề biểu diễn đồ thị có hướng là bán bậc ra của đỉnh u ($\deg^+(u)$).
- Tổng các phần tử của cột u của ma trận kề biểu diễn đồ thị có hướng là bán bậc vào của đỉnh u ($\deg^-(u)$).

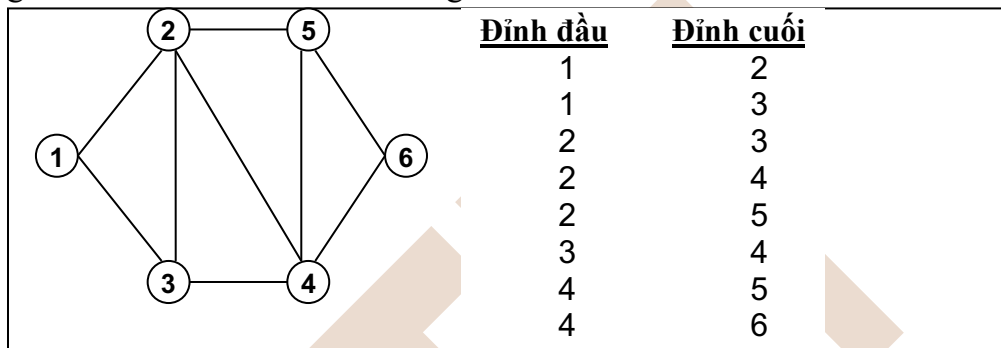
Ưu điểm của ma trận kề:

- Đơn giản dễ cài đặt trên máy tính bằng cách sử dụng một mảng hai chiều.
- Dễ dàng kiểm tra được hai đỉnh u, v có kề với nhau hay không bằng đúng một phép so sánh ($a[u][v] \neq 0$).

Nhược điểm của ma trận kề:

- Lãng phí bộ nhớ: bất kể số cạnh nhiều hay ít ta cần n^2 đơn vị bộ nhớ để biểu diễn.
- Không thể biểu diễn được với các đồ thị có số đỉnh lớn.
- Để xem xét đỉnh u có những đỉnh kề nào cần mất n phép so sánh kể cả đỉnh u là đỉnh cô lập hoặc đỉnh treo.

Biểu diễn đồ thị bằng danh sách cạnh: Phương pháp biểu diễn đồ thị $G = \langle V, E \rangle$ bằng cách liệt kê tất cả các cạnh của nó được gọi là phương pháp biểu diễn bằng danh sách cạnh. Đối với đồ thị có hướng ta liệt kê các cung tương ứng. Đối với đồ thị vô hướng ta chỉ cần liệt kê các cạnh $(u, v) \in E$ mà không cần liệt kê các cạnh $(v, u) \in E$. Ví dụ về biểu diễn đồ thị bằng danh sách cạnh được cho trong Hình 4.3.



Hình 4.3. Biểu diễn đồ thị bằng danh sách cạnh

Tính chất của danh sách cạnh:

- Đỉnh đầu luôn nhỏ hơn đỉnh cuối của mỗi cạnh đối với đồ thị vô hướng.
- Đỉnh đầu không phải lúc nào cũng nhỏ hơn đỉnh cuối của mỗi cạnh đối với đồ thị có hướng.
- Số các số có giá trị u thuộc cả tập đỉnh đầu và tập đỉnh cuối các cạnh là bậc của đỉnh u đối với đồ thị vô hướng.
- Số các số có giá trị u thuộc cả tập đỉnh đầu các cạnh là bán bậc ra của đỉnh u đối với đồ thị có hướng.
- Số các số có giá trị u thuộc cả tập đỉnh cuối các cạnh là bán bậc vào của đỉnh u đối với đồ thị có hướng.

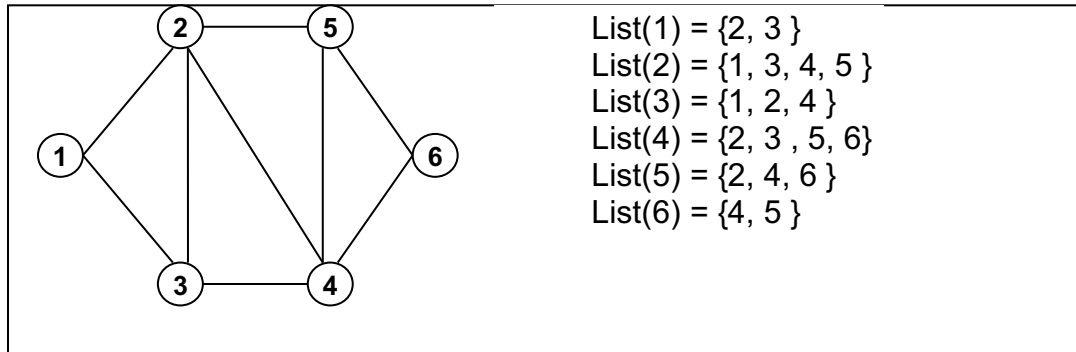
Ưu điểm của danh sách cạnh:

- Trong trường hợp đồ thị thưa ($m \ll n^2$), biểu diễn bằng danh sách cạnh tiết kiệm được không gian nhớ.
- Thuận lợi cho một số thuật toán chỉ quan tâm đến các cạnh của đồ thị.

Nhược điểm của danh sách cạnh:

- Khi cần duyệt các đỉnh kề với đỉnh u bắt buộc phải duyệt tất cả các cạnh của đồ thị. Điều này làm cho thuật toán có chi phí tính toán cao.

Biểu diễn đồ thị bằng danh sách kề: Ta định nghĩa $List(u) = \{v \in V: (u, v) \in E\}$ là danh sách các đỉnh kề với đỉnh u . Biểu diễn đồ thị bằng danh sách kề là phương pháp liệt kê tập đỉnh kề của mỗi đỉnh. Ví dụ về biểu diễn đồ thị bằng danh sách kề được cho trong Hình 4.3.



Hình 4.3. Biểu diễn đồ thị bằng danh sách kề.

Tính chất của danh sách kề:

- Lực lượng tập đỉnh kề của đỉnh u là bậc của đỉnh u đối với đồ thị vô hướng ($\deg(u) = |List(u)|$).
- Lực lượng tập đỉnh kề của đỉnh u là bán bậc ra của đỉnh u đối với đồ thị có hướng ($\deg^+(u) = |List(u)|$).
- Số các số có giá trị u thuộc tất cả các danh sách kề là bán bậc vào của đỉnh u đối với đồ thị có hướng.

Ưu điểm của danh sách kề:

- Dễ dàng duyệt tất cả các đỉnh của một danh sách kề.
- Dễ dàng duyệt các cạnh của đồ thị trong mỗi danh sách kề.
- Tối ưu việc cài đặt một số giải thuật trên đồ thị.

Nhược điểm của danh sách kề:

- Khó khăn cho người học có kỹ năng lập trình yếu vì khi biểu diễn đồ thị ta phải dùng một mảng, mỗi phần tử của nó là một danh sách liên kết.

11.2. Một số bài toán cơ bản dựa vào đồ thị

Dưới đây là một số bài toán cơ bản dựa vào đồ thị:

1. **Biểu diễn đồ thị vô hướng.** Cho đồ thị vô hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy viết chương trình thực hiện chuyển đổi biểu diễn đồ thị dưới dạng danh sách kề.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh của đồ thị; $|E|$

dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.

- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách kề của các đỉnh tương ứng theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1	0-> 1-> 2
6 9	1-> 0-> 2->4
0 1 0 2 1 2 1 4 2 3 2 4 3 4 3 5 4 5	2-> 0-> 1-> 3-> 4
	3-> 2-> 4-> 5
	4-> 1-> 2-> 3-> 5
	5-> 3-> 4

2. **Biểu diễn đồ thị vô hướng.** Cho đồ thị có hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy viết chương trình thực hiện chuyển đổi biểu diễn đồ thị dưới dạng danh sách kề.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách kề của các đỉnh tương ứng theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1	0-> 1
6 9	1-> 4
0 1 1 4 2 0 2 1 2 4 3 2 4 3 4 5 5 3	2-> 0-> 1-> 4
	3-> 2
	4-> 3-> 5
	5-> 3

3. **DFS(u) trên đồ thị vô hướng.** Cho đồ thị vô hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy viết thuật toán duyệt theo chiều sâu bắt đầu tại đỉnh $u \in V$ ($DFS(u)=?$)

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào ba số $|V|$, $|E|$, $u \in V$ tương ứng với số đỉnh, số cạnh và đỉnh bắt đầu duyệt; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh được duyệt theo thuật toán DFS(u) của mỗi test theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1 6 9 4 0 1 0 2 1 2 1 4 2 3 2 4 3 4 3 5 4 5	4 1 0 2 3 5

4. **DFS(u) trên đồ thị có hướng.** Cho đồ thị có hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy viết thuật toán duyệt theo chiều sâu bắt đầu tại đỉnh $u \in V$ (DFS(u))=?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào ba số $|V|$, $|E|$, $u \in V$ tương ứng với số đỉnh, số cạnh và đỉnh bắt đầu duyệt; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh được duyệt theo thuật toán DFS(u) của mỗi test theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1 6 9 4 0 1 1 4 2 0 2 1 2 4 3 2 4 3 4 5 5 3	4 3 2 0 1 5

5. **BFS(u) trên đồ thị có hướng.** Cho đồ thị vô hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy viết thuật toán duyệt theo chiều rộng bắt đầu tại đỉnh $u \in V$ (BFS(u))=?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào ba số $|V|$, $|E|$, $u \in V$ tương ứng với số đỉnh, số cạnh và đỉnh bắt đầu duyệt; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.

- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh được duyệt theo thuật toán BFS(u) của mỗi test theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1 6 9 0 0 1 0 2 1 2 1 4 2 3 2 4 3 4 3 5 4 5	0 1 2 4 3 5

6. **BFS(u) trên đồ thị có hướng.** Cho đồ thị có hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy viết thuật toán duyệt theo chiều rộng bắt đầu tại đỉnh $u \in V$ ($\text{BFS}(u)$)=?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào ba số $|V|, |E|, u \in V$ tương ứng với số đỉnh, số cạnh và đỉnh bắt đầu duyệt; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh được duyệt theo thuật toán BFS(u) của mỗi test theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1 6 9 0 0 1 1 4 2 0 2 1 2 4 3 2 4 3 4 5 5 3	0 1 4 3 5 2

7. **Đường đi từ u đến v bằng DFS(u).** Cho đồ thị vô hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm đường đi từ đỉnh $s \in V$ đến đỉnh $t \in V$ trên đồ thị bằng thuật toán DFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào bốn số $|V|, |E|, s \in V, t \in V$ tương ứng với số đỉnh, số cạnh, đỉnh u, đỉnh v; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra đường đi từ đỉnh s đến đỉnh t của mỗi test theo thuật toán DFS của mỗi test theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1 6 9 1 6 1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	1-> 2-> 3-> 4-> 5-> 6

8. **Đường đi từ u đến v bằng DFS(u).** Cho đồ thị có hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm đường đi từ đỉnh $s \in V$ đến đỉnh $t \in V$ trên đồ thị bằng thuật toán DFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào bốn số $|V|, |E|, s \in V, t \in V$ tương ứng với số đỉnh, số cạnh, đỉnh u , đỉnh v ; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra đường đi từ đỉnh s đến đỉnh t của mỗi test theo thuật toán DFS của mỗi test theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1 6 9 1 6 1 2 2 5 3 1 3 2 3 5 4 3 5 4 5 6 6 4	1-> 2-> 5-> 6

9. **Đường đi từ u đến v bằng BFS(u).** Cho đồ thị vô hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm đường đi từ đỉnh $s \in V$ đến đỉnh $t \in V$ trên đồ thị bằng thuật toán BFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào bốn số $|V|, |E|, s \in V, t \in V$ tương ứng với số đỉnh, số cạnh, đỉnh u , đỉnh v ; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra đường đi từ đỉnh s đến đỉnh t của mỗi test theo thuật toán BFS của mỗi test theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1	1-> 2-> 7-> 8-> 10

6 9 1 6	
1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	

10. **Đường đi từ u đến v bằng BFS(u).** Cho đồ thị có hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm đường đi từ đỉnh $u \in V$ đến đỉnh $v \in V$ trên đồ thị bằng thuật toán BFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào bốn số $|V|, |E|, s \in V, t \in V$ tương ứng với số đỉnh, số cạnh, đỉnh u, đỉnh v; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra đường đi từ đỉnh s đến đỉnh t của mỗi test theo thuật toán BFS của mỗi test theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1	1-> 2-> 5-> 6
6 9 1 6	
1 2 2 5 3 1 3 2 3 5 4 3 5 4 5 6 6 4	

11.3. Một số bài toán nâng cao dựa vào đồ thị

Dưới đây là một số bài toán nâng cao dựa vào đồ thị:

11. **Thuật toán DFS tìm số thành phần liên thông.** Cho đồ thị vô hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm số thành phần liên thông của đồ thị bằng thuật toán DFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra số thành phần liên thông của đồ thị bằng thuật toán DFS.

Input:	Output:
1	2
6 6	
1 2 1 3 2 3 3 4 3 5 4 5	

12. Thuật toán BFS tìm số thành phần liên thông. Cho đồ thị vô hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy tìm số thành phần liên thông của đồ thị bằng thuật toán BFS.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra số thành phần liên thông của đồ thị bằng thuật toán BFS.

Input:	Output:
1 6 6 1 2 1 3 2 3 3 4 3 5 4 5	2

13. Kiểm tra tính liên thông mạnh 1. Cho đồ thị có hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS, hãy kiểm tra xem đồ thị có liên thông mạnh hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra “YES”, hoặc “NO” theo từng dòng tương ứng với test là liên thông mạnh hoặc không liên thông mạnh.

Input:	Output:
1 6 9 1 2 2 4 3 1 3 2 3 5 4 3 5 4 5 6 6 3	YES

14. Kiểm tra tính liên thông mạnh 2. Cho đồ thị có hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy kiểm tra xem đồ thị có liên thông mạnh hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra “YES”, hoặc “NO” theo từng dòng tương ứng với test là liên thông mạnh hoặc không liên thông mạnh.

Input:	Output:
1 6 9 1 2 2 4 3 1 3 2 3 5 4 3 5 4 5 6 6 3	YES

15. **Định trụ 1.** Cho đồ thị vô hướng liên thông $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS, hãy đưa ra tất cả các đỉnh trụ của đồ thị?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh trụ của mỗi test theo từng dòng.

Input:	Output:
1 1 2 1 3 2 3 2 5 3 4	2 3

16. **Định trụ 2.** Cho đồ thị vô hướng liên thông $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy đưa ra tất cả các đỉnh trụ của đồ thị?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các đỉnh trụ của mỗi test theo từng dòng.

Input:	Output:
1	2 3

1 2 1 3 2 3 2 5 3 4	
---------------------	--

17. **Cạnh cầu 1.** Cho đồ thị vô hướng liên thông $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS, hãy đưa ra tất cả các cạnh cầu của đồ thị?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các cạnh cầu của mỗi test theo từng dòng.

Input:	Output:
1 1 2 1 3 2 3 2 5 3 4	2 5 3 4

18. **Cạnh cầu 2.** Cho đồ thị vô hướng liên thông $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy đưa ra tất cả các cạnh cầu của đồ thị?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các cạnh cầu của mỗi test theo từng dòng.

Input:	Output:
1 1 2 1 3 2 3 2 5 3 4	2 5 3 4

19. **Cây khung 1.** Cho đồ thị vô hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS, hãy xây dựng một cây khung của đồ thị bắt đầu tại đỉnh $u \in V$?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào ba số $|V|$, $|E|$, u tương ứng với số đỉnh, số cạnh và đỉnh $u \in V$; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các cách cây khung của đồ thị bắt đầu tại đỉnh u của mỗi test theo từng dòng; đưa ra -1 nếu đồ thị không có cây khung?.

Input:	Output:
1	5 2 2 1 1 3 3 4 4 6
1	
6 9 5	
1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	

20. **Cây khung 2.** Cho đồ thị vô hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy xây dựng một cây khung của đồ thị bắt đầu tại đỉnh $u \in V$?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào ba số $|V|$, $|E|$, u tương ứng với số đỉnh, số cạnh và đỉnh $u \in V$; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách các cách cây khung của đồ thị bắt đầu tại đỉnh u của mỗi test theo từng dòng; đưa ra -1 nếu đồ thị không có cây khung?.

Input:	Output:
1	5 2 5 3 5 4 5 6 2 1
6 9 5	
1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	

11.4. Một số bài toán tiên tiến dựa vào đồ thị

Dưới đây là một số bài toán tiên tiến dựa vào đồ thị:

21. **Chu trình trên đồ thị vô hướng 1.** Cho đồ thị vô hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc “NO” kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Input:	Output:
1	YES
6 9	
1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	

22. **Chu trình trên đồ thị vô hướng 2.** Cho đồ thị vô hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc “NO” kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Input:	Output:
1	YES
6 9	
1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	

23. **Chu trình trên đồ thị vô hướng 3.** Cho đồ thị vô hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán **Union Set**, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$

dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.

- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc “NO” kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Input:	Output:
1 6 9 1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	YES

24. Chu trình trên đồ thị vô hướng 4. Cho đồ thị vô hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng **Disjoin Set**, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc “NO” kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Input:	Output:
1 6 9 1 2 1 3 2 3 2 5 3 4 3 5 4 5 4 6 5 6	YES

25. Chu trình trên đồ thị có hướng 1. Cho đồ thị có hướng $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán DFS, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc “NO” kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Input:	Output:
1 6 9 1 2 2 4 3 1 3 2 3 5 4 3 5 4 5 6 6 4	YES

26. Chu trình trên đồ thị có hướng 2. Cho đồ thị có hướng $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Sử dụng thuật toán BFS, hãy kiểm tra xem đồ thị có tồn tại chu trình hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra YES hoặc “NO” kết quả test theo từng dòng tương ứng với đồ thị tồn tại hoặc không tồn tại chu trình.

Input:	Output:
1 6 9 1 2 2 4 3 1 3 2 3 5 4 3 5 4 5 6 6 4	YES

27. Đồ thị Euler/ nửa Euler 1. Cho đồ thị vô hướng liên thông $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy kiểm tra xem đồ thị có phải là đồ thị Euler, nửa Euler hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100; 1 \leq |V| \leq 10^3; 1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra 1, -1, 0 kết quả mỗi test theo từng dòng tương ứng với đồ thị là Euler, nửa Euler, hay không phải Euler hoặc nửa Euler.

Input:	Output:
1	1

6 10	
1 2 1 3 2 3 2 4 2 5 3 4 3 5 4 5 4 6 5 6	

28. **Đồ thị Euler/ nửa Euler 2.** Cho đồ thị có hướng liên thông yếu $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy kiểm tra xem đồ thị có phải là đồ thị Euler, nửa Euler hay không?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra 1, -1, 0 kết quả mỗi test theo từng dòng tương ứng với đồ thị là Euler, nửa Euler, hay không phải Euler hoặc nửa Euler.

Input:	Output:
1	1
6 10	
1 2 2 4 2 5 3 1 3 2 4 3 4 5 5 3 5 6 6 4	

29. **Chu trình Euler 1.** Cho đồ thị vô hướng liên thông $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy đưa ra một chu trình Euler của đồ thị bắt đầu tại đỉnh $u \in V$?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào ba số $|V|, |E|, u \in V$ tương ứng với số đỉnh, số cạnh và $u \in V$; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V, v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra chu trình Euler của đồ thị bắt đầu tại đỉnh u theo từng dòng, đưa ra -1 nếu đồ thị không có chu trình Euler.

Input:	Output:
1	1 2 3 4 5 2 4 6 5 3 1
6 10 1	
1 2 2 4 2 5 3 1 3 2 4 3 4 5 5 3 5 6 6 4	

30. **Chu trình Euler 2.** Cho đồ thị có hướng liên thông yếu $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh. Hãy đưa ra một chu trình Euler của đồ thị bắt đầu tại đỉnh $u \in V$?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào ba số $|V|$, $|E|$, $u \in V$ tương ứng với số đỉnh, số cạnh và $u \in V$; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra chu trình Euler của đồ thị bắt đầu tại đỉnh u theo từng dòng, đưa ra -1 nếu đồ thị không có chu trình Euler.

Input:	Output:
1 6 10 1 1 2 1 3 2 3 2 4 2 5 3 4 3 5 4 5 4 6 5 6	1 2 4 3 2 5 6 4 5 3 1

31. **Đường đi Euler 1.** Cho đồ thị vô hướng liên thông $G=(V, E)$ được biểu diễn dưới dạng danh sách cạnh. Hãy đưa ra một đường đi Euler của đồ thị?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.
- T, $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra một đường đi Euler của đồ thị theo từng dòng, đưa ra -1 nếu đồ thị không có đường đi Euler.

Input:	Output:
1 6 9 1 2 2 5 3 1 3 2 4 3 4 5 5 3 5 6 6 4	2 1 3 2 5 3 4 6 5 4

32. **Đường đi Euler 2.** Cho đồ thị có hướng liên thông yếu $G=(V, E)$ được biểu diễn dưới dạng danh sách cạnh. Hãy đưa ra một đường đi Euler của đồ thị?

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh, số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ đôi $u \in V$, $v \in V$ tương ứng với một cạnh của đồ thị.

- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra một đường đi Euler của đồ thị theo từng dòng, đưa ra -1 nếu đồ thị không có đường đi Euler.

Input:	Output:
1 6 9 1 2 2 4 2 5 3 1 3 2 4 3 5 3 5 6 6 4	5 3 1 2 4 3 2 5 6 4

33. Biểu diễn đồ thị vô hướng có trọng số. Cho đồ thị vô hướng có trọng số $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh trọng số. Hãy viết chương trình thực hiện chuyển đổi biểu diễn đồ thị dưới dạng danh sách kề có trọng số.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ ba $u \in V, v \in V, w$ tương ứng với một cạnh cùng với trọng số cạnh của đồ thị.
- $T, |V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra danh sách trọng số của các cạnh kề với đỉnh tương ứng theo khuôn dạng của ví dụ dưới đây.

Input:	Output:
1 6 10 1 2 3 1 3 3 2 3 3 2 4 5 2 5 5 3 4 5 3 5 5 4 5 2 4 6 2 5 6 2	1: 3->3 2: 3-> 3->5->5 3: 3-> 3-> 5-> 5 4: 5-> 5-> 2-> 2 5: 5-> 5-> 2-> 2 6: 2-> 2

34. Thuật toán Kruskal tìm cây khung nhỏ nhất. Cho đồ thị vô hướng có trọng số $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh trọng số. Hãy viết chương trình tìm cây khung có độ dài nhỏ nhất bằng thuật toán Kruskal.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ ba $u \in V$, $v \in V$, w tương ứng với một cạnh cùng với trọng số cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra độ dài cây khung nhỏ nhất của mỗi test theo từng dòng; đưa ra -1 nếu đồ thị không liên thông.

Input:	Output:
1	15
6 10	
1 2 3	
1 3 3	
2 3 3	
2 4 5	
2 5 5	
3 4 5	
3 5 5	
4 5 2	
4 6 2	
5 6 2	

35. Thuật toán PRIM tìm cây khung nhỏ nhất. Cho đồ thị vô hướng có trọng số $G=(V, E)$ được biểu diễn dưới dạng danh sách cạnh trọng số. Hãy viết chương trình tìm cây khung có độ dài nhỏ nhất bằng thuật toán PRIM.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|$, $|E|$ tương ứng với số đỉnh và số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ ba $u \in V$, $v \in V$, w tương ứng với một cạnh cùng với trọng số cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 200$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra độ dài cây khung nhỏ nhất của mỗi test theo từng dòng; đưa ra -1 nếu đồ thị không liên thông.

Input:	Output:
1	15
6 10	
1 2 3	
1 3 3	

2 3 3	
2 4 5	
2 5 5	
3 4 5	
3 5 5	
4 5 2	
4 6 2	
5 6 2	

36. **Chu trình âm.** Cho đồ thị có trọng số $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh trọng số âm hoặc dương. Hãy viết chương trình xác định xem đồ thị có chu trình âm hay không.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai số $|V|, |E|$ tương ứng với số đỉnh và số cạnh của đồ thị; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ ba $u \in V, v \in V, w$ tương ứng với một cạnh cùng với trọng số cạnh của đồ thị.
- T, $|V|, |E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra 1 hoặc 0 theo từng dòng của mỗi test tương ứng với đồ thị có hoặc không có chu trình âm.

Input:	Output:
2	1
3 3	0
1 2 -1	
2 3 4	
3 1 -2	
3 3	
3 3	
1 2 -1	
2 3 2	
3 1 -2	

37. **Thuật toán Dijkstra.** Cho đồ thị có trọng số không âm $G=\langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh trọng số. Hãy viết chương trình tìm đường đi ngắn nhất từ đỉnh $u \in V$ đến tất cả các đỉnh còn lại trên đồ thị.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.

- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai ba số $|V|$, $|E|$ tương ứng với số đỉnh và $u \in V$ là đỉnh bắt đầu; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ ba $u \in V$, $v \in V$, w tương ứng với một cạnh cùng với trọng số cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra kết quả của mỗi test theo từng dòng. Kết quả mỗi test là trọng số đường đi ngắn nhất từ đỉnh u đến các đỉnh còn lại của đồ thị theo thứ tự tăng dần các đỉnh.

Input:	Output:
1	0 4 12 19 21 11 9 8 14
9 12 1	
1 2 4	
1 8 8	
2 3 8	
2 8 11	
3 4 7	
3 6 4	
3 9 2	
4 5 9	
4 6 14	
5 6 10	
6 7 2	
6 9 6	

38. **Thuật toán Bellman-Ford.** Cho đồ thị có trọng số có thể âm hoặc không âm $G = \langle V, E \rangle$ được biểu diễn dưới dạng danh sách cạnh trọng số. Hãy viết chương trình tìm đường đi ngắn nhất từ đỉnh $u \in V$ đến tất cả các đỉnh còn lại trên đồ thị.

Input:

- Dòng đầu tiên đưa vào T là số lượng bộ test.
- Những dòng tiếp theo đưa vào các bộ test. Mỗi bộ test gồm $|E|+1$ dòng: dòng đầu tiên đưa vào hai ba số $|V|$, $|E|$ tương ứng với số đỉnh và $u \in V$ là đỉnh bắt đầu; $|E|$ dòng tiếp theo mỗi dòng đưa vào bộ ba $u \in V$, $v \in V$, w tương ứng với một cạnh cùng với trọng số cạnh của đồ thị.
- T , $|V|$, $|E|$ thỏa mãn ràng buộc: $1 \leq T \leq 100$; $1 \leq |V| \leq 10^3$; $1 \leq |E| \leq |V|(|V|-1)/2$;

Output:

- Đưa ra kết quả của mỗi test theo từng dòng. Kết quả mỗi test là trọng số đường đi ngắn nhất từ đỉnh u đến các đỉnh còn lại của đồ thị theo thứ tự tăng dần các đỉnh.

Input:	Output:
--------	---------

CHƯƠNG 5. ĐỒ THỊ VÀ ỨNG DỤNG

1	0 -1 2 -2 1
5 8 1	
1 2 -1	
1 3 4	
2 3 3	
2 4 2	
2 5 2	
4 2 1	
4 3 5	
5 4 -3	

ĐỒ THỊ

TÀI LIỆU THAM KHẢO

1. Nguyễn Đức Nghĩa., “*Cấu trúc dữ liệu và giải thuật*,”. Nhà Xuất Bản KHKT Hà Nội, 2013.
2. Đỗ Xuân Lôi. “*Cấu trúc dữ liệu và giải thuật*,”. Nhà Xuất Bản KHKT Hà Nội , 2009.
3. Donald E. Knuth., “The Art of Computer Programming”, Addison-Wesley, Vol 1, 2, 3, 4, 5, 5.
4. Niklaus Wirth, Data Structures and Algorithms, Prentice Hall, 2003.

PDF